Полный статический аудит Proteus

Core Modules Desktop Extensions

Приватный технический отчёт для владельца проекта
7 ноября 2026 года

Назначение: зафиксировать полное текстовое покрытие четырёх репозиториев, перечислить все подтверждённые находки, условия и границы доказательств и определить порядок исправлений для модульного конструктора агентов.

Прочитан 2921 текстовый файл и 805803 строки. Код не запускался. Шесть Desktop binaries остаются только hash-only.

Состав отчёта

• Вывод и точное покрытие
• Приоритеты и архитектура конструктора
• Core runtime contracts persistence CLI tests и docs
• Modules tools workflows policy models и integration tests
• Desktop client UI catalog packages и terminal
• Extensions SDK Git production Pele и experimental native
• Полный каталог Desktop vendor findings
• Fixtures measurement oracles и план верификации

1 Вывод и границы аудита

Во всех четырёх закреплённых деревьях прочитаны все tracked текстовые файлы, включая исходники, тесты, скрипты, документацию, примеры, lockfiles и поставляемый vendor. Итог: **2921 текстовый файл и 805803 строки**, без непрочитанного текстового остатка после объединения диапазонов. Это результат чтения, а не test coverage.

Основной вывод для конструктора агентов: существующих границ Core достаточно, чтобы исправить найденные ошибки без новых behavior slots. Наиболее важны согласованная отмена и владение процессами, изоляция протокольного stdin, безопасный read-only Git, сохранение плана после checkpoint и сквозные ограничения очередей. Каталог Desktop уже объединяет разные типы пакетов в интерфейсе, но ещё не является атомарным установщиком одной составной capability.

Не все бинарные данные исследованы содержательно. В Desktop шесть бинарных файлов только сверены по Git blob. В Core PNG просмотрен визуально, а 11 текстовых members исторического tar.gz прочитаны отдельно. Поэтому утверждение «проверено всё содержимое всех файлов» было бы неверным.

Режим только чтение. Код репозиториев, тесты, сборки, fixtures, benchmark и предложенные сценарии не запускались. Исходники не изменялись. Найденные цепочки выведены из исходников. Не измерены RSS, CPU, задержки, UX или реальная эксплуатация. Чтение всех строк не доказывает отсутствие других дефектов и не сертифицирует безопасность.

Отчёт подготовлен для приватного рассмотрения. Уязвимости не публиковались. Все ссылки привязаны к SHA, а не к подвижной ветке main. HEAD мог измениться после фиксации снимка 7 ноября 2026 года. Перед исправлением нужно сверить diff до актуального HEAD, сохранив эти выводы как baseline.

Основной обзор дан по-русски. Подробные технические приложения сохраняют точные русские и английские формулировки доказательств, условия и отвергнутые гипотезы. Идентификаторы локальны разделу: например Core C1 не совпадает с vendor input C1. Model-находки Modules переименованы в MM1–MM5, чтобы не смешивать их с M1–M7.

2 Покрытие и закреплённые версии

Репозиторий	Tracked файлов	Текстовых прочитано	Строк прочитано	Прочий материал
Core	916	914 из 914	188630	PNG просмотрен; tar прочитан по members
Modules	399	399 из 399	69894	Всё tracked содержимое текстовое
Desktop	1165	1159 из 1159	407995	6 binary hash-only
Extensions	449	449 из 449	139284	Vendor включён
Итого	2929	2921 из 2921	805803	8 бинарных tracked paths

Диапазоны объединены по path и identity, повторное чтение не увеличивает итог. Hash, поиск и инвентаризация сами по себе не засчитывались как чтение. Symlink учитывается как tracked текст цели, без повторного обхода. Inline tests входят в строки содержащего их файла.

В Extensions используется LF-конвенция: первоначальный splitlines inventory давал 139285 из-за U+0085 внутри minified Pele. Прочитаны все 725719 Unicode-символов этого файла; корректный LF-итог 139284. Это различие определения строки, не пропуск.

Vendor и внешний код

Desktop: 583 vendor текстовых файла, 280950 строк; остальные 576 текстовых файлов, 127045 строк. В vendor входят Ghostty wrappers/resources, gpui-base, gpui-component и gpui-fast-wgpu. Два compiled terminfo файла остаются hash-only. Внешняя native Zig implementation Ghostty, получаемая build.rs, не находится в закреплённом дереве и не исследована этим чтением.

Extensions: 222 vendor файла, 102355 LF-строк; 11 notices и upstream patch documents, 1913 строк; остальные 216 файлов, 35016 строк. Production Node/Pele и opt-in native Rust renderers рассматриваются раздельно.

Core: явных tracked vendor directories нет. 658 файлов crates, 133783 строки прочитаны. Архив и research включены: 98 текстовых файлов, 24078 строк. Ещё 10429 строк в 11 members исторического tar.gz учитываются отдельно и не прибавлены к 805803 tracked строкам.

Modules: 399 файлов после dedup source, integration и model зон. Cargo.toml закрепляет **Core 23883bd3e43e482136c8dcbc40fa032ec6cc456d**, а общий Core-аудит выполнен на 9b07d03. Эти версии нельзя считать одной и той же runtime-сборкой.

Исходники внешних Cargo/npm/Python зависимостей, незакоммиченные и git-ignored файлы, пользовательские сессии и другие репозитории не становятся проверенными от чтения lockfile. Точечные проверки ureq и gpui-fast для Desktop указаны отдельно в доказательствах и не выданы за полный аудит этих библиотек.

3 Приоритеты исправления

P1 означает первоочередной риск на указанной границе. P2 означает воспроизводимый по исходникам дефект или существенный условный ресурсный риск. P3 означает локальную корректность, диагностику, документацию либо менее критичный API. Это не CVSS и не оценка доказанной эксплуатации. Vendor API и experiment не повышаются автоматически до production-инцидента.

Очередь	Работа	Почему и при каких условиях
Сначала	Core R1; Modules M1 и M2	Работающий child может попасть в idle pool; one-shot shell конкурирует за protocol stdin; context-pack обходит безопасные Git helper flags
Затем	Core C1 C2 I2 L2; Modules M3 M4	Сквозная отмена и deadlines; durable plan retention; одинаковая policy для эквивалентных путей

Очередь	Работа	Почему и при каких условиях
Затем	Core B2 B4 K2; JSON B1 L1 B3	Права credential history и точная цель записи; формат и актуальность профиля
Затем	Modules M5 M6 M7; Core I1; Extensions EXT02 EXT03 EXT04; Desktop DCAT04	Admission до создания thread, budgets до материализации, end-to-end queue и cache limits
Затем	Extensions EXT01; Modules MM1 MM2 MM3; Desktop DUI01 DUI02 DUI04	Revision/dedup истории, верная model semantics, видимость approval, сохранение настроек и cancel intent
Отдельный срочный library track	Ghostty VGHOST01–04	Safe API нарушает buffer/init/lifetime контракты; текущий Desktop exploit path не доказан
До включения experiment	Native Mermaid NATIVE01	Маленький вход может запросить 8 GB payload arrays; production Pele этим дефектом не объявляется
Параллельно	Остальные UI vendor defects, harness и docs	Точные границы приведены в каталоге; нужны targeted regressions, не только source assertions

4 Конструктор агентов без новых slots

Core уже имеет пять behavior descriptors: **Hook, Workflow, Context, Model, Policy**. Tool не является шестым behavior descriptor. Process contracts отдельно включают workflow, model, context, context_provider, policy, tool и hook: это другой уровень классификации.

Model optional, deterministic model-free workflow и detached tool execution уже предусмотрены. Services принадлежат root lifecycle и доступны через явные capabilities; Transport, AgentControl, persistence и presentation не нужно превращать в новые behavior slots. Для найденных ошибок достаточно укрепить существующие contracts, lifecycle, admission и storage boundaries.

Для установки одной capability в Desktop полезен необязательный bundle manifest с конкретными package references, проверкой совместимости и зависимостей, preview плана, staging и rollback. Такой слой должен вызывать существующие installers и отдельно активировать profile bindings. Установка module сама по себе сейчас не означает включение slot/tool. Agent instructions и ссылки не являются установкой Skill. Не вводить новый Core slot ради каталога.

Сначала следует сделать корректными отмену, профили, идентичность событий и бюджеты. После этого проверять единый capability install на сочетании module, extension, skill, AgentPlugin и MCP, включая отказ в середине и восстановление. Не считать совпадение protocol integer достаточной проверкой требуемых features.

Точные SHA снимков

Core
9b07d03199d929b4ea6f6b7c9f50fd0ff7b878c9

Modules
9899e02d97e0dd2e66d7fd864fd2603e6306baf1

Desktop
a608d93486440a4b38843745a5e16aae2d6f79aa

Extensions
d4d05d2e65a11053675be26399ccdda2df3faa3e

Шесть Desktop binaries только hash only

assets/fonts/noto-color-emoji/NotoColorEmoji.ttf
10730124 bytes; Git blob e605743d05f261634f0b3f68a439d9b557d8f335

assets/icon.png
1982 bytes; Git blob 7a05482b164527df23a56ffe44b3eae0f107d608

src/ui/chat/images/decode/testdata/animated-40.webp
1964 bytes; Git blob f46bfeafecd1bc78fa70f632e45d5612345ca437

src/ui/chat/images/decode/testdata/overlong-257.webp
12380 bytes; Git blob b2a02c177a15c8bcad09014b046810e4ee191b61

vendor/ghostty-vt/resources/terminfo/67/ghostty
3884 bytes; Git blob f41a6d051c0101ac85c751ffbaaf49176beb635a

vendor/ghostty-vt/resources/terminfo/78/xterm-ghostty
3884 bytes; Git blob f41a6d051c0101ac85c751ffbaaf49176beb635a

Данные этих шести файлов не декодировались и не проверялись визуально либо покадрово. Их hashes подтверждают identity, не содержательную проверку.

5 Core все подтверждённые находки

Proteus-core 9b07d03

Подтверждённые исходниками дефекты

Приоритеты предварительные и условные. Точное описание условий, существующих защит и сценария воспроизведения находится в соответствующем подробном отчёте. P2 не означает, что дефект воспроизводится во всех конфигурациях.

Lifecycle и отмена

1. **R1, P1: race отмены может вернуть ещё работающий child в idle pool.** agent_control/process/turn.rs сохраняет исходный Send ID A; принятый Send B может начать следующий turn после завершения A. Cancel направляется A; старый terminal response A принимается за подтверждение чистой остановки, хотя B уже работает. При включённом idle pool достаточно OS liveness, чтобы вернуть child как Cancelled/resumable. Нужен сценарий A terminal unread → B started → Cancel A. Существующие тесты terminal race и targeted cancel не комбинируют эти события. Исправление внутри existing turn/lifecycle:

подтверждать завершение всех соответствующих admitted работ либо уничтожать неоднозначного peer, а не добавлять slot.

- turn.rs
- cancel settlement

2. **R2, P2: malformed успешный Send response превращается в Completed с пустым текстом.** Decode через .ok() и unwrap_or_default() скрывает protocol error при ok=true, но неверном output. Нужен malformed/custom peer, стандартный producer таким не объявляется. Соседний messaging path уже валидирует строго.

- turn.rs:429–435

3. **C1, P2: blocked presentation sink задерживает наблюдение model cancellation.** После получения delta presentation ждёт sink, сравнивая его только с optional deadline. При timeout=0 отмена не пробуждает await; ненулевой timeout ограничивает ожидание своим сроком, а не моментом cancel. Outer workflow может отдельно drop operation, поэтому не утверждается, что любой пользовательский Cancel зависнет.

- bound_model.rs:197–206

4. **C2, P2: provider cancel-on-drop ждёт durable error recording.** Cancellation branch сохраняет model stream живым во время record_failure; timeout branch уже делает drop(stream) раньше. Медленная/зависшая запись откладывает сигнал cancel компоненту и может продолжать генерацию ресурсов.

- bound_model.rs:448–459

5. **I1, P2: очередь broker сохраняет cancelled/expired IDs при насыщении.** Pending budget освобождается, но queued_roots не удаляет historical IDs, пока не появится active capacity. Повтор enqueue/cancel при занятом единственном root накапливает IDs сверх pending bound. Это retained ID memory, не весь payload; generation failure очищает очередь.

- failure.rs:23–61
- runtime.rs:430–443

6. **I2, P2: sequential ProcessSession timeout не охватывает передачу request.** Synchronous send_frame ждёт completion без timeout до timed response phase. Живой child, не читающий stdin, и frame больше pipe capacity могут зависнуть на записи. V3 handshake имеет отдельную timed dispatch защиту и не включён в это утверждение.

- session.rs:86–95

7. **L2, P2: CLI shutdown timeout начинается после безграничного ожидания ACK.** Живой/остановленный child держит stdout открытым и не отвечает на Shutdown; двухсекундный child.wait ещё не достигается. EOF обрабатывается правильно, поэтому exited child с закрытым stdout не является сценарием.

- cli_app.rs:221–232

Конфигурация контракты и filesystem

8. **B1, P2: Builder не сохраняет writable JSON profile.** Loader читает JSON по extension, Builder оставляет тот же destination, но persistence безусловно парсит его TOML. Save реального изменения падает. Это отдельный путь от init ниже.

- persistence.rs:202–212

9. **B2, P2: config history может ослабить права приватного credential-bearing profile.** Full ConfigBuilderState сохраняется обычным fs::write без mode 0600. При umask 022 и traversable ancestors история становится 0644, даже если profile был 0600. Закрытый 0700 home ограничивает последствия; фактических секретов не читали и утечку не воспроизводили.

- config_history.rs:38–64

10. **B3, P2: HTTP new/cold-resume стартует из server-start config после Save.** Immutable launch snapshot используется до старта asynchronous profile watcher. Немедленная новая сессия видит старые model/tools, а недоступный старый executable может сорвать startup до watcher. Source-session creation берёт её current config и не является этим fallback.

- lifecycle.rs:54–79
- lifecycle.rs:148–163

11. **B4, P2: workspace Ignore следует symlink .gitignore и дописывает внешний файл.** Проверяются выбранные input paths, но destination открывается обычным append. Нужны существующий writable/readable UTF-8 target, symlink и авторизованный live workspace. Это ошибка цели записи, не обход HTTP auth.

- ignore.rs:24–69

12. **K1, P2: unknown fields вложенных canonical model controls молча игнорируются.** Strict outer DTO не распространяется на ReasoningConfig и другие вложенные structs. reasoning:{"budget_token":1024} принимается, но budget_tokens остаётся None. Это contract correctness, не доказанная privilege escalation.

- domain/model.rs:42–51

13. **K2, P2 conditional: workspace_write открывает существующий hardlink с O_TRUNC.** Descriptor-relative/no-symlink защита не различает shared inode. Предварительно существующий workspace hardlink на writable external inode позволяет изменить обе aliases. Это узкая гарантия helper, не доказанный обход restricted sandbox.

- workspace_write.rs:104–123

14. **L1, P2: init разрешает .json destination, но пишет TOML.** Следующий loader выбирает JSON и отвергает свой только что созданный profile. Нужно сохранять выбранный формат либо отвергать его до записи.

- cli_init.rs:169–185

Диагностика и benchmarks

15. **L3, P3: CLI footer использует выдуманный универсальный context window 200000.** Для 128k и 1M моделей один и тот же estimate показывается с одним процентом. Runtime budget bug этим не доказан.

- repl_render.rs:139–155

16. **L4, P3: doctor ищет trusted commands по parent PATH.** Resolved child environment не используется. Отдельная restricted read_executable ветка имеет защиту, но trusted child PATH диагностируется неверно. Также exists не доказывает executable regular file.

- cli_doctor.rs:256–293

17. **E1, P2: history-scale теряет subject CPU/IO totals после zombie sample.** Sampler overwrites classification, /proc exe недоступен → controller, summary исключает ранее backend/module identity. Общие tree counters остаются, ошибка именно subject summary.

- proc_sampler.py:101–120
- measurement.py:79–82

18. **E2, P2 conditional: legacy multi-agent sampler может принять reused root PID и завершить посторонний процесс.** После reaped driver новый sample заново присваивает текущий PID/start identity, затем cleanup сигналит её как owned. Нужны узкий PID-reuse race и OS permission. Новый common sampler уже pins root start, но old multi-agent helper остаётся отдельным.

- proc.py:29–48
- run.py:115–129

19. **E3, P3: отказ benchmark overwrite происходит после перезаписи binaries.json.** Старые case results остаются, standalone manifest уже описывает новые attempted binaries. Embedded manifest в прежнем report сохраняет часть provenance.

- run.py:235–246

Уже реализованное и вывод для модульного конструктора

- Пять behavior descriptors: Hook, Workflow, Context, Model, Policy. Tool не behavior descriptor. Process contracts отдельно включают workflow/model/context/context_provider/policy/tool/hook. Нельзя смешивать эти два счёта и объявлять инфраструктурные services новыми behavior slots.
- Model optional. Core не подставляет скрытого provider; model-free deterministic workflow и detached tool execution существуют. Отсутствующие обязательные behavior defaults представлены явно ограниченными stubs, а не магическим agent stack.
- AgentControl является root-owned lifecycle service. Context preparation, tool selection, memory/search/patch behavior могут жить в существующих tools/providers/hooks; отдельные новые слоты по результату аудита не нужны.
- PreparedAssembly связывает plan/registry в immutable epoch; admission фиксирует execution context. Catalog проверяет формы/дубли до factory, inspector не должен запускать несвязанные behavioral components.
- Typed execution lineage, strict version/export/feature negotiation, generation fencing, callback budgets, bounded admission/output, targeted cancellation, cancellation grace и process-group lifecycle уже реализованы. Найденные queue/deadline/cancel gaps не означают отсутствия этих механизмов целиком.
- Trusted local components по умолчанию имеют права пользователя. Opt-in Linux restriction ограничивает конкретный worker/export с явными grants, но mixed profile не является полной sandbox всей агентной системы; доверенные host callbacks и sibling identities имеют документированные пределы.
- Strict journal/read/replay semantics отделяют durable facts от presentation. Unknown tool effect не воспроизводится автоматически. Workflow replay использует recorded results, но реальный workflow/policy остаётся исполняемым кодом; prompt replay является реальным provider request с hosted-tools opt-in.

- Raw tool arguments нормализуются до hooks/schema/policy; гипотеза raw-vs-args bypass отвергнута. Child TurnStarted/SteeringDelivered не входят в whitelist forwarded runtime events, поэтому предполагавшееся загрязнение root transcript этим producer path не подтверждено.

Практический порядок ремонта: settlement/отмена и end-to-end deadlines; credential-history modes и filesystem destination checks; JSON lifecycle и nested DTO strictness; diagnostics/measurement; затем документация. Это локальные исправления существующих границ, не основание добавлять Search/Memory/Planner/Transport slots.

Документация и исторические материалы

Текущие docs содержат 8 групп проверенных противоречий: model/v12 против v13; removed aggregate binary в install/portable инструкциях; mandatory provider против model-free; disabled tool inventory; semantic profile polling; interrupted_turn producer; doctor handshake side effects; старые ACP/steering/reconnect описания. Подробные строки и оговорки приведены далее в разделе о текущей документации Core. Это отдельная группа maintenance defects, не ещё 8 production багов.

Research и archive читаются как исторические данные. Старые 12 slots, Rust dylib/ABI планы, Tauri/Leptos UI, memory/v2, A2A SDK эксперимент, внешние upstream vulnerabilities и speculative новые contracts не являются действующим контрактом Core. Некоторые research документы содержат неразрешаемые вне исходного сеанса citation markers; их текст прочитан, но внешние исследования не повторялись и их утверждения не сертифицированы. Количественная производительность и качество live agent задач здесь не измерялись.

Core подробная цепочка отмены и существующие защиты

Proteus-core 9b07d03

R1 Отмена может вернуть в idle pool peer у которого продолжает работать turn из сообщения

Приоритет: P1. Условия: peer process, сообщение в момент окончания предыдущего turn и ненулевое удержание idle processes. Это дефект lifecycle/cancellation, а не предположение о враждебном модуле.

Причина

1. process/turn.rs:216–254: active_send_id хранит основной Send A. В ветке message_delivery.queue отправляется B, но active id не меняется.
2. process/messaging.rs:30–47,77–109: B. обычный Send, а не отдельный гарантированно steering-only метод. Когда A уже завершён, B начинает новый turn. Это прямо предусмотрено существующим кодом и комментарием; его response обрабатывается как полный AgentOutput.
3. process/turn.rs:224–226,258–268: отмена всё ещё передаёт A в finish_cancelled.
4. process/turn.rs:451–477: cancel_child_turn отправляет Cancel(A), а затем считает отмену чистой по **любому Response с id A**. Он не учитывает незавершённые receipt B и не доказывает отсутствие нового turn.
5. У штатного producer stdio.rs:179–210 Send либо сразу подтверждает очередь, либо асинхронно публикует результат нового turn. runs.rs:208–234 снимает A с active и отменяет только текущий совпадающий run_id. Cancel(A) не отменяет B.

6. При clean=true turn.rs:328–340 не убивает процесс. process/mod.rs:474–515 проверяет только OS liveness, возвращает процесс в pool и может публиковать Cancelled, resumable=true.

Воспроизводимый порядок событий

- A закончен на стороне peer, его response ещё не обработан родителем.
- B уже отправлен из mailbox и на стороне peer начинает отдельный turn.
- Отмена родителя срабатывает раньше обработки response A.
- Родитель отправляет Cancel(A); peer не отменяет активный B.
- Родитель читает старый response A и принимает его за подтверждение чистой остановки.
- Родитель сообщает завершённую отмену и удерживает peer как idle, хотя B продолжает работать.

Последствия: возможны действия после сообщённой отмены, неверное состояние resumable/idle и ошибочный следующий reuse/resume. При max_idle_processes=0 последующее освобождение уничтожает процесс, поэтому основной опасный эффект связан с удержанием peer.

Минимальное исправление

Не считать response старого A достаточным доказательством quiescence при неразрешённых message Send. Учитывать все соответствующие запросы/receipts в cancellation settlement; если установить фактический активный turn и остановку нельзя, инвалидировать connection и завершить peer, а не возвращать его в pool. Это изменение существующего process lifecycle, без нового slot.

Regression

Расширить существующую terminal-race fixture: A завершён, ответ задержан; B стартует и остаётся активным; после барьера B-started отменить generation; peer отклоняет Cancel(A) и отдаёт задержанный response A. Проверить, что B остановлен либо peer гарантированно удалён, и при неопределённом состоянии resumable=false.

Уже существуют отдельные tests terminal-race success и targeted cancel: process_agent_control.rs:308–408. Их сочетание данным сценарием не проверяется. Проанализированы также pool admission, output limits и rotation tests.

R2 Невалидный успешный ответ основного Send молча считается Completed с пустым текстом

Приоритет: P2. Условия: ошибочный/custom peer или несовпадающий wire producer. Штатный stdio producer в проверенном снимке возвращает корректный AgentOutput; произвольный штатный turn не объявляется сломанным.

process/turn.rs:429–435 принимает ok=true, затем выполняет from_value::<AgentOutput>(value).ok() и unwrap_or_default(). Отсутствующий output или ошибочная форма DTO не возвращают protocol error: получается TurnEnd::Completed("").

Сценарий

После валидного SessionStarted fixture отвечает на текущий Send, эхо-повторяя его id, с ok=true, output=null, error=null. Ответ валиден как внешний StdioOutput, но не содержит обязательного результата Send. Parent завершает задачу как Completed с пустым summary и может сохранить peer для повторного использования.

Последствие: сломанный контракт скрывается под успешным пустым результатом. Это особенно вредно в модульном конструкторе, где явная ошибка границы помогает выявлять неверную сборку.

Минимальное исправление и тест

Обязательно извлечь output и десериализовать AgentOutput с контекстом ошибки; не преобразовывать ошибку в пустой success. Для сообщений уже есть нужный строгий образец: process/messaging.rs:98–108.

Проверить отсутствие output, неверный тип text и receipt {queued:true,accepted:true} вместо результата основного Send. Ожидается явная ошибка и invalidation peer через существующий fatal path. Из inspected suites ни один не проверяет эти формы основного successful response.

Реализованные меры которые нельзя выдавать за отсутствующие

Все пункты ниже прочитаны в коде и соответствующих тестах; тесты в рамках аудита не запускались.

1. **Generic execution без обязательного чата.** runtime/execution.rs создаёт detached ExecutionScope и проходит existing policy/approval/grants/recorder. execution_boundary.rs подтверждает архитектурный контракт отдельными сценариями без agent attribution. execution_boundary/workflow.rs проверяет model-free standalone workflow и явную ошибку обращения к отсутствующей модели.
2. **Согласованная смена сборки.** runtime/reload.rs, runtime/settings.rs, runtime/execution_binding.rs используют цельный runtime snapshot; admitted execution сохраняет выбранные registry/settings. runtime/tests/snapshot_atomicity.rs, reasoning_reload.rs и интеграционный memory reload test проверяют разные точки этой границы.
3. **Жизненный цикл turn и сохранение сделанных действий.** runtime/turn.rs, checkpoint.rs, failed_history.rs, terminals.rs различают cancellation, timeout и error, сохраняют уже зафиксированный progress. reviewed_workflow.rs сохраняет кандидат перед review и ограничивает продолжения. Hook interruption tests проверяют, что завершённый эффект не исчезает из следующей истории.
4. **Idle residency без потери writer ownership.** runtime/residency.rs, resume.rs освобождают decoded history, сохраняя lease; resume_cache.rs проверяет восстановление авторитетного состояния перед записью. Это не безусловное удержание полной истории в RAM.
5. **Ограниченная steering queue.** runtime/steering.rs имеет лимиты количества и суммарного текста, revisioned snapshots и финализационную границу. steering/weave.rs связывает доставку с точным anchor; последующие turn получают свежую execution identity и согласованные настройки. Прочитаны command, integration и review tests.
6. **AgentControl не добавляет behavior slot.** agent_control/mod.rs, tool_host.rs, task/collaboration facades предоставляют root-owned управление целыми peers. Peer profile указывает собственную конфигурацию. Тест разных tool surfaces проверяет независимые конфигурации peers.
7. **Admission, pool и resume ограничены.** pending.rs, process/pool.rs, process/config.rs ограничивают pending/role execution/idle retention, привязывают resume к session/role/cwd и reserve до eviction. Inherited depth не может быть пополнен metadata; depth integration test проверяет self-cycle и A→B→A.
8. **Устойчивость обычной отмены и I/O.** Mailbox закрывается перед cancellation, delivery сериализован, stdin записи ограничены deadline/cancellation/health. Stdout ограничен 256 buffered outputs, 32 MiB buffered bytes и 8 MiB frame. Ошибки вывода прерывают ожидание approval/user input. Это уже существующая защита, но она не закрывает сочетание terminal race с cancellation из R1.

9. **Ограниченный collaboration control.** Установлены лимиты sessions/agents/outstanding completions/batch wait/summary/error. Generation reservation защищает followup, terminal records освобождают owner/context. Истёкший backend history token не возрождается поздним completion.
10. **Worktree lifecycle сохраняет работу.** Detached owner доводит cleanup при cancellation; свежая не начавшаяся задача отличается от resume, неопределённо остановленный процесс сохраняет checkout. Нет автоматического merge и force branch deletion. Tests проверяют сохранение изменённых, missing/detached и имеющих уникальные commits worktrees.

Проверенные смежные вопросы

- Child `TurnStarted` и `SteeringDelivered` **не пересылаются** в родительский runtime event stream: whitelist process/turn.rs:192–205. `TurnStarted` применяется локально для session binding. Поэтому гипотеза о перезаписи root thread в Desktop именно этими forwarded child events не подтверждена.
- `runtime/builder.rs` строит runtime из переданного AppConfig; config_path не означает синхронную повторную загрузку AppConfig. Этот факт учтён в находке Core B3 об HTTP lifecycle. Он сам по себе не является отдельной находкой.
- `workflow_host/model_stream.rs` ограничивает backlog канала, не общее количество tool calls за sampling request. Один backlog bound не доказывает сквозной model/provider limit; глобальный лимит не заявляется ни существующим, ни отсутствующим на основании одного участка.

Core подробности persistence composition contracts и host

Proteus-core 9b07d03

Findings

B1 P2 writable JSON profiles cannot be saved by Config Builder

The regular loader accepts a `.json` file as JSON. Config Builder marks a file-backed profile as writable and retains that same filename as the persistence target. Its persistence implementation always parses an existing file as TOML. Thus a valid JSON profile can launch the server, but an otherwise valid changed Save fails with a TOML parse error. This affects profile and addon saves through `set_profile_config`, not just a particular UI.

Conditions: explicit valid existing JSON config file, writable path, a real configuration change and otherwise successful assembly preparation. No runtime reproduction was performed; the parser mismatch is direct control-flow evidence.

Evidence:

- JSON parser selection.
- Writable snapshot.
- Target keeps the JSON path.
- Existing target goes to TOML reader and unconditional TOML parse.

Recommended fix: preserve the selected format in persistence, or explicitly mark unsupported JSON-backed profiles read-only before presenting Save. Suggested regression: launch from valid JSON, change permission mode/addon, save and reload, assert the same intended config is loaded.

B2 P2 security config history can make formerly private credentials readable by other local users

record_replaced_state serializes the replaced full ConfigBuilderState using fs::write, without private file mode or redaction. That state includes MCP server environments and arbitrary module configuration. With a normal UNIX umask 022, new history files are mode 0644 and newly created history directories 0755. A source profile protected by mode 0600 therefore does not preserve that protection in its automatic history copy. Main profile persistence does explicitly preserve source file permissions, which the history writer does not.

Conditions: a UNIX installation with traversable config-store ancestors, a source profile restricted to its owner, secrets in MCP env or module_config, and a Save that records the old state; umask must allow group/other reads (e.g. 022). This is not a claim that an installation under a mode-0700 home directory exposes data to other users. No secret values were accessed or copied during this audit, and no runtime permission test was executed.

Evidence:

- History serialization and creation.
- State includes MCP servers and module_config.
- MCP env serialization.
- Main writer preserves mode.

Recommended fix: create history files exclusively with mode 0600 (and history directories privately), or carry forward an equally restrictive original permission policy. Redaction is an alternative only if restoring credential-bearing config is intentionally unsupported. Suggested regression: source mode0600 under mode0755 temporary ancestor, umask022, Save, assert no group/other bits on history.

B3 P2 HTTP new resumed sessions use the server start profile snapshot after a successful profile Save

HttpAppState captures the initial AppConfig in an immutable launch context. Creating a new session without a source session and resuming a cold session both reuse this snapshot, even when the shared profile has subsequently changed. AgentAppServer constructs the runtime from the supplied config before starting its asynchronous profile watcher. The watcher needs two stable observations with a 300ms interval. Consequently the new runtime initially admits the old model/tools/profile; if the old component executable/config is no longer viable, startup can fail before the watcher exists.

Conditions: server remains alive across a changed profile, then /new-session with no source_session_dir or resume of a session not already live. A new session with a source_session_dir uses that source's current config and is not the specific path described here. The exact duration of stale operation depends on scheduling and assembly preparation; 300ms is the polling interval, not a guaranteed upper bound. Static control flow confirmed; no running model or tool was invoked.

Evidence:

- Capture initial configuration.
- New session fallback.
- Cold resume.
- Runtime constructed before watcher.
- Watcher is spawned, two stable reads, poll period.

Cross-file source evidence: runtime/builder.rs:172–340 uses its passed config; PreparedAssembly::from_config in assembly/mod.rs:222–240 does not reread AppConfig from config_path. Both ranges are included in the complete Core coverage.

Recommended fix: obtain a validated current profile under the existing profile-path lock before constructing a new/resumed runtime. Suggested regression: Save then immediately new/resume, assert published module/model config matches the new profile without sleep; also remove the old component executable before the new session to distinguish startup failure from eventual correction.

B4 P2 Ignore action follows a repository gitignore symlink and appends outside the repository

The /workspace/ignore route delegates to git::ignore. It validates the paths to be ignored, but not the destination ignore file. The Gitignore branch selects <repository-root>/.gitignore; both read_to_string and the later append-mode OpenOptions::open follow a pre-existing symlink. An ordinary Ignore action can therefore modify an unrelated writable file outside the repository, rather than a repository-owned ignore file. No race is needed.

Conditions: a valid live-session Git workspace, root .gitignore already symlinked to an existing readable UTF-8 and writable external file, and at least one requested pattern not already present. The request must pass normal HTTP authentication/session checks. This is an unintended write target, not a token-authentication bypass. The append is completed before the final changes(root) call, so an error during that subsequent refresh would not undo it. Static standard-library and call-path evidence only; no repository test or reproduction was executed.

Evidence:

- HTTP route.
- Selected input paths are validated, destination is resolved and read normally.
- Destination opened with symlink-following append.

Recommended fix: obtain the ignore destination through a directory-relative, no-follow regular-file open, and apply equivalent checks to exclude-file parents. A pre-open canonical path check alone retains a TOCTOU window. Suggested regression: symlink .gitignore to a synthetic file outside a temporary repository, request Ignore for an ordinary workspace file, assert rejection and unchanged external bytes.

Findings

P2 Presentation can mask model cancellation indefinitely when model timeout is disabled

Primary location: bound_model.rs:197-206. Call site: bound_model/presentation.rs:29-47.

present_stream receives an item from the cancellation-aware recording stream, then awaits event publication before yielding that item. emit_delta_before_deadline races the sink only against the optional deadline, not against the bound/effective cancellation token. If the sink blocks and model_timeout_ms == 0, this await has no way to finish when the execution token is canceled, so the inner stream's cancellation select cannot be polled. A nonzero timeout delays cancellation until that deadline, which is three hours by default.

Concrete regression scenario using existing test fixtures: bind DelayedStreamAdapter producing one immediate delta and then pending, use StalledSink from bound_model_tests/deadline.rs, configure model timeout 0, start complete, wait until sink append starts, cancel the execution token. Completion does not return a canceled failure, because the pending sink is the only active awaited branch. This is a static control-flow finding, not an executed reproduction.

Impact is on the bound model capability and any host that waits for it to settle cooperatively. Outer workflow cancellation may independently drop the whole operation and limit the root-run impact; this finding does not claim that every user-facing Cancel hangs.

Suggested correction: thread the effective invocation cancellation token into presentation waits, and on cancellation release presentation so the inner recording stream can record cancellation; preserve already-recorded terminal outcomes. Add a stalled-sink cancellation regression distinct from the existing deadline-only regression.

P2 Cancel waits for durable error recording before signaling the provider s cancel on drop path

Primary location: bound_model.rs:448-450. Provider cancellation: process_adapters/model.rs:117-130 and 270-272.

The timeout branch explicitly drops stream before awaiting record_failure (lines 453-459). The cancellation branch does not. The process adapter sends CancelCause::User from CancelOnDrop; while model_error_recorded is slow or blocked, the model stream remains alive and the component has not received that cancel signal. A request can therefore keep generating/spending resources after Core has observed the canceled execution token.

The repository already tests the needed invariant for timeout in deadline_drops_provider_stream_before_waiting_for_error_recording (bound_model_tests/deadline.rs:325-362). The same scenario with an explicit canceled token and the same blocking recorder exposes the missing cancellation case: error_started occurs while PendingDropStream.dropped is still false. Static finding only; no tests run.

Suggested correction: drop the provider stream before the cancellation error-recording await, mirroring the timeout branch, and cover explicit cancellation with the existing drop-observed fixture.

Checked and not reported as defects

- Tool hooks mutate arguments through apply_hook_response, which revalidates against the same spec; retaining the pre-hook tool_spec is valid because a hook cannot change call identity through its response contract.
- Auxiliary model callbacks skip recursive model hooks, preserve operation owner/lineage via the delegating host, and pass the same bound tool authority validation.
- Arbitrary metadata.granted_permissions on the approved result path is explicitly documented generic grant semantics, not an undocumented bypass introduced in this review.
- Terminal model suffixes and embedded failure progress use the history-safe validation path; the inspected tests cover forged tool results, reused part/message IDs and duplicate completion normalization.
- Assembly planning does not start declared components; omitted environment/args/module_config values are deliberate diagnostic redaction covered by tests.

No P0/P1 finding was established in this slice. Both findings are static, conditional on blocking I/O, and should not be presented as runtime-reproduced defects.

Finding malformed known model control fields silently accepted

Priority: P2 (contract correctness, not established privilege escalation).

CanonicalModelRequest does not deny unknown fields, and neither do ReasoningConfig, SamplingConfig, ModelLimits, CacheHints, InstructionBlock, CanonicalMessage, or ContentPart. Strictness on the containing process DTO does not propagate to these nested structs. In particular, ReasoningConfig gives all three known fields defaults.

Static reproduction: take a valid serialized WorkflowCompleteModelRequest; replace request.reasoning with {"budget_token":1024}. Deserializing the typed envelope accepts it, discards budget_token, and yields effort=None, summary=false, budget_tokens=None. A misspelled supported control is therefore silently converted to disabled/default reasoning, rather than the explicit decode error required by AGENTS.md. Adding an unknown field directly to request is also ignored. This does not concern intentionally extensible metadata.

Sources:

- Источник crates/proteus-contracts/src/domain/model.rs#L42-L51
- Источник crates/proteus-contracts/src/model_standard/canonical_request.rs#L13-L29
- Источник crates/proteus-contracts/src/contracts/workflow.rs#L140-L144

Existing safeguards: the outer WorkflowCompleteModelRequest and ProcessModelInput deny unknown fields; canonical tool/spec/part records, model response, capabilities and failure DTOs are strict. Unknown enum variants fail. These do not reject unknown fields in the nested structs listed above. Recommendation is to make known DTO fields strict, preserving explicit opaque metadata. No additional slot is required.

Finding workspace write truncates out of workspace hardlink aliases

Priority: P2 conditional filesystem containment defect; not a sandbox bypass by an otherwise unprivileged remote caller.

The helper promises descriptor-bound writes below cwd. It rejects parent traversal and symlinks, and Linux final open uses BENEATH|NO_SYMLINKS. However it opens an existing inode with WRONLY|CREATE|TRUNC. Neither symlink checks nor BENEATH distinguish an ordinary workspace file from a hardlink to another path outside cwd.

Static reproduction (not run): on the same filesystem, prepare outside/target containing valuable bytes, create a hardlink workspace/link to outside/target, then call workspace_file_for_write(workspace, "link"). The opening itself truncates the shared inode; writing through the returned handle changes outside/target as well. Requires an existing hardlink and permission to write the inode. Does not require or demonstrate defeating Linux restricted sandbox rights. Even a subsequent write error cannot undo the initial truncation.

Sources:

- Источник crates/proteus-contracts/src/tool_support/workspace_write.rs#L31-L42
- Источник crates/proteus-contracts/src/tool_support/workspace_write.rs#L104-L123

Existing safeguards: canonical root, safe relative components, NOFOLLOW directory traversal, Linux openat2 BENEATH|NO_SYMLINKS, and descriptor rather than path writes protect against the tested symlink replacement cases. The tests cover symlinks and parent substitution, not shared hardlink inodes. Resolve in this existing file-write helper with explicitly chosen replacement/link semantics, not a new slot. The wider approval-cache contract only claims tool-name/cwd reuse for opting-in tools and does not itself promise per-argument path proof.

Rejected or bounded hypotheses

- Schema validation uses raw_arguments as authoritative while module helpers parse args. Source review traced BoundTools::execute_enriched at bound_tools.rs:301–305 and confirmed raw arguments are

normalized into args before recording, hooks, schema and policy, with the effective call then invoked. This is not a confirmed runtime bypass.

- Model response structure alone does not validate every canonical identity, but runtime stream/history layers can impose those constraints. No standalone duplicate-message exploit is claimed.
- WorkspaceWrite approval reuse is explicitly by tool name and cwd, with metadata opt-in: contracts/approval_transport.rs:167–169. That alone is not a defect; tool filesystem guarantees are a separate concern.

Architectural evidence

ExecutionScope contains identity and cancellation, not mandatory conversation/model services. ExecutionContext model is optional. WorkflowInvocationContext explicitly distinguishes detached execution from conversation state. AgentControl is documented as a root-owned service, not a behavior slot. Context preparation and tool selection use typed hooks, not reintroduced dedicated slots. Process protocol v3 export refs, invocation lineage, host-defined composition and per-export settings preserve modular boundaries in these contracts. No constructor redesign or new slots is indicated by the findings above.

Finding 1 Canceled queued roots accumulate outside the pending budget

Priority: P2, resource-bound violation in a long-lived saturated broker.

Locations: crates/proteus-module-protocol/src/v3/failure.rs:23-61,153-174; src/v3/runtime.rs:295-313,430-443; src/v3/pending.rs:111 (latter paths relative to the same crate).

finish_without_admission removes the invocation from pending but does not remove its ID from queued_roots. Cancellation and expiry of an inactive queued root reach this removal. admit_queued_roots pops IDs only while can_activate_root() is true. Pending-root capacity counts live entries in pending, not historical queue IDs.

Static reproduction: set max_active_roots=1 and max_pending_roots=2; keep one root active for a long period. Repeatedly submit a second queued root and cancel it (or let its short deadline expire) before the active root completes. Each cancellation releases pending capacity so another root can be submitted, while its old ID remains in the VecDeque. Retained IDs grow with total historical submissions, not configured pending capacity. When the blocker eventually completes, draining historical IDs also creates a burst of cleanup work.

Existing safeguards: live pending count remains bounded; IDs eventually drain when active capacity opens, and generation failure clears the queue. This is retained ID/deque memory, not retained whole request payload. tests/broker_v3.rs:292-381 covers individual queued cancellation/expiry, but does not hold saturation through repeated churn. src/v3/runtime_tests.rs:255-355 likewise does not exercise indefinite saturated churn.

Suggested repair: remove canceled/expired queued IDs immediately, or implement bounded lazy compaction independent of activation capacity. Add a source-level unit regression that repeatedly cancels queued roots while keeping the active root occupied and checks queue storage remains bounded. No new slot is needed.

Finding 2 Sequential request timeout does not cover request transmission

Priority: P2, potentially indefinite hang in the sequential Session/Host facade.

Locations: crates/proteus-process-host/src/session.rs:86-95,135-136; src/writer.rs:226-228,399-400,544; src/host.rs:147-155 (latter paths relative to the same crate).

ProcessSession::request synchronously calls send_frame before starting recv_response(timeout). send_frame waits for writer completion with dispatch.wait(), which uses wait_for_completion(None). The response timeout's clock begins only after that unbounded write completes. The Host facade keeps its session guard across this operation.

Static reproduction: start a live process that does not read stdin; submit a permitted JSON request larger than OS pipe capacity but below the configured outbound-frame bound, with a short timeout such as 50 ms. The writer can block in the pipe write indefinitely and the caller never reaches the timed response phase. Initialization through this sequential facade has the same exposure.

Existing safeguards: explicit lifecycle termination can release ordinary process-group children and unblock writes; response-phase timeout terminates the child. V3 initialization separately uses timed dispatch plus termination (crates/proteus-module-protocol/src/v3/handshake.rs:18-38) and is not implicated by this finding. tests/session.rs:697-706 tests a small transmitted request without a response, not blocked transmission. The external termination test at 644-667 does not establish that the request's own timeout bounds its write.

Suggested repair: use one end-to-end deadline for admission/write completion/response, with bounded dispatch wait and termination of the affected session on transmission timeout. Add a non-reading-child regression using a request above pipe capacity. No extra slot or architectural adapter is required.

Safeguards and boundary observations

- Existing authority remains slot/contract based, with seven declared process-capable slots. Ordered-many versus select-one composition and exact export/config/features handshake checks prevent inventing authority from a module name.
- V3 strictly validates frame envelope shape, directional/generation invocation IDs, callback replay intervals, parent/depth/active bounds, callback budgets, cancellation grace and generation failure. Dropped async terminal receivers trigger cancellation wakeup.
- Reader/writer/lifecycle separation and bounded outbound byte/frame admission limit blocked transport effects. Notification quotas remain held through RAII permits, including dequeued notifications; notification consumers do not block terminal routing.
- Feature dispatch is checked against the current generation's acknowledged export and feature set; generation fencing protects control dispatch.
- Unix lifecycle uses WNOWAIT to retain child identity before process-group termination/reap. Linux pidfd/eventfd monitoring has a portable polling fallback.
- The optional Linux bubblewrap sandbox explicitly pins executable/mount identities, rejects reserved roots and unsafe writable alias arrangements, sanitizes backend environment, closes inherited file descriptors except its mount pins, and isolates network unless enabled. Trusted default mode is not an OS sandbox.
- Multiple exports in one process share a trust/failure domain. The same-process sibling-parent behavior is explicitly documented/tested (tests/broker_v3.rs:163-202), so this audit does not misrepresent it as an unexpected security boundary bypass.

The above safeguards are verified from source and test definitions only, not runtime validation. No claim that the rest of this scope is defect-free is made.

P2 init accepts a JSON destination but writes TOML

is_config_file_path accepts both .toml and .json, so init_destination_path retains an explicit config.json filename. run_init always writes profile.config_body_for_init(), which emits TOML for every profile. The config loader selects serde_json::from_str for .json. Thus proteus --config /tmp/profile.json init coding reports successful initialization, then proteus --config /tmp/profile.json doctor fails to parse its generated file. An existing JSON profile is also overwritten with invalid-for-extension content. This scenario is statically established, not executed.

Sources:

- Источник crates/proteus-core/src/cli_init.rs#L71-L88
- Источник crates/proteus-core/src/cli_init.rs#L169-L185
- Источник crates/proteus-core/src/cli_init.rs#L217-L229
- Источник crates/proteus-core/src/core/config/loading.rs#L351-L364

Existing safeguards: TOML directory/file generation has loadability tests; generated peer configs use absolute locations and support files are created. No test covers a JSON destination. Reject JSON before writing, or serialize the generated config in its chosen format.

P2 shutdown deadline starts only after an unbounded response wait

CliAppClient::shutdown awaits request(Shutdown) before starting its two-second child.wait timeout. request's output loop and read_output's next_line have no deadline. A live child that retains stdout but never sends the correlated shutdown response leaves the parent waiting indefinitely, so start_kill and kill_on_drop are never reached. This is a silent/live or stopped child case; an exited child closing stdout is correctly detected and is not the reproduction condition. Main waits for shutdown after both interactive and one-shot work. Reproduction would pause the live server child before asking the parent to exit; it has not been run.

Sources:

- Источник crates/proteus-core/src/cli_app.rs#L118-L165
- Источник crates/proteus-core/src/cli_app.rs#L221-L232
- Источник crates/proteus-core/src/main.rs#L224-L249

Existing safeguards: request IDs are correlated strictly; EOF/errors propagate; child has kill_on_drop; child.wait has a two-second timeout once reached. The deadline needs to cover the shutdown request/write/read lifecycle, not only subsequent process exit.

P3 footer invents a fixed 200k context window

footer_context always divides metadata.context.token_estimate by 200,000, independent of selected model capabilities. A 100,000-token estimate is shown as 50% for a 128k or 1M model alike; missing usage is shown as 0% rather than unknown. This is presentation misinformation, not a demonstrated runtime budget bug. Keep the existing presentation surface and consume authoritative model/context information or omit unsupported percentage.

Source: Источник crates/proteus-core/src/main/repl_render.rs#L139-L155

Safeguards: unknown model identity is rendered as not selected; percentages are clamped. Those do not establish the denominator.

P3 doctor checks trusted component commands in parent PATH

check_external_commands computes resolved_environment but discards it. Except for sandbox.read_executable, it calls check_command with only command/cwd. command_in_path reads the parent's PATH. A valid trusted component with env.PATH pointing at a private executable directory can therefore be diagnosed as missing; conversely a parent-PATH executable can be reported available despite being absent from the child's PATH. Absolute/relative command checks only use exists(), so a directory/non-executable file also passes that check. The existing regression specifically covers restricted read_executable, leaving trusted child environments uncovered. No component was launched to reproduce this; this is a source-confirmed diagnostic mismatch, not proof that actual runtime launch fails.

Sources:

- Источник crates/proteus-core/src/cli_doctor.rs#L256-L293
- Источник crates/proteus-core/src/cli_doctor.rs#L331-L365
- Источник crates/proteus-core/src/main_tests/inspection.rs#L140-L167

Existing architecture and safeguards

- Product CLI delegates execution/session behavior to its own app-server process through canonical JSONL, rather than implementing a second runtime.
- Reserved namespaces are classified before config/runtime launch; malformed modules/tools/server/inspect/replay commands cannot fall through to inference.
- Permission flags conflict explicitly; new/resume and parent-session routing have validation.
- Noninteractive approval denies and user input returns an empty result rather than silently authorizing.
- HTTP CLI invokes HttpServerConfig validation; tests require a token on non-loopback bind.
- Inspection uses AssemblyPlan; invalid plans are surfaced, and tests show unrelated workflow/context/hook components are not started.
- Doctor session audit is explicitly current-workspace versus all-sessions; strict history loading errors are surfaced without migration fallback.
- Replay renderers preserve detached attribution and distinguish unavailable comparisons from false comparisons. Actual replay internals were outside this new scope.
- No finding calls for adding a module slot or changing the modular constructor architecture.

Findings

P2 Subject CPU IO counters disappear when the final sample sees a zombie

The common sampler refreshes executable and classification on each observation and replaces the retained row for the same PID/start identity. If /proc/PID/exe is no longer available, history-scale classifies the row as controller. Its summary then selects only rows still classified backend/modules. The already accumulated CPU/IO counters for this subject disappear from subject_observed_counters, despite belonging to the measured workload.

Scenario: sample the running backend with nonzero counters; sample the same PID/start identity during zombie teardown when exe is unreadable; summarize. Subject totals omit this identity. Existing test_clone_read explicitly models this transition, but protects the private trace's identity selection, not Metrics.summary's

counter filtering. Full-tree retained counters and earlier interval samples remain available; this is underreporting of the subject summary, not all metrics.

Sources: examples/performance/common/proc_sampler.py:101–120, examples/performance/common/proc_sampler.py:175–181, examples/performance/history-scale/measurement.py:15–24, examples/performance/history-scale/measurement.py:79–82, examples/performance/history-scale/test_clone_read.py:128–149.

Fix direction: preserve first established subject classification per stable process identity, or summarize over a separately retained set of subject identities.

P2 Legacy multi agent sampler can adopt and kill a reused root PID

The old sampler unconditionally inserts the current root PID and records its current start_ticks, rather than pinning the root's original identity. Cleanup samples again before signaling every currently matching owned identity. If the benchmark driver has exited and been reaped, and its PID is reused before cleanup's sample, the replacement process is newly adopted as owned and can receive SIGTERM/SIGKILL. Its descendants can be adopted too. This requires the narrow PID-reuse race and operating-system permission to signal the replacement process.

Existing start_ticks checks protect previously retained descendants, but cannot protect a root identity overwritten by the immediately preceding sample. The newer common sampler already pins root_start and has a PID-reuse regression test; the old multi-agent path still uses its independent helper. Journal-recovery also uses that helper for measurements, but the unrelated-process kill conclusion here is limited to multi-agent cleanup.

Sources: examples/performance/multi-agent/proc.py:29–48, examples/performance/multi-agent/run.py:115–129, examples/performance/multi-agent/run.py:176–190, examples/performance/common/proc_sampler.py:68–74, examples/performance/common/proc_sampler.py:130–138, examples/performance/common/test_proc_sampler.py:52–58.

Fix direction: use the identity-pinned common sampler or pin the root start_ticks in this legacy implementation before discovering descendants or accepting cleanup ownership.

P3 Refusing to overwrite a benchmark case still overwrites its binary manifest

main accepts an existing output directory and writes binaries.json before measure checks whether a case directory already exists. Repeating a completed command with the same output directory but different binaries overwrites the top-level manifest, then fails with refusing to overwrite evidence. The old case results/report remain but binaries.json now describes the new attempted binaries. The report's embedded old manifest helps recover the provenance, but the standalone manifest is no longer trustworthy.

Sources: examples/performance/multi-agent/run.py:134–140, examples/performance/multi-agent/run.py:235–246.

Fix direction: reject an occupied output directory before writing anything, or validate all planned case destinations and preserve manifests per run.

Existing safeguards and architecture

- Agent-worker uses existing Workflow callbacks, model/tool/context services and checkpoints. Its explicitly conversational example requires a model, without introducing a universal model requirement or an additional host slot.

- Context preparation uses the existing hook/v6 context_preparation phase; tgrep is an ordinary tool export. Neither requires a specialized core integration slot.
- Python component helper validates envelopes, direction, generation and active lineage, serializes stdout and bounds pending/active work. JS hook adapters reject unsupported actions/fields, preserve narrowing semantics and leave host validation authoritative.
- Rust tool is a separately pinned SDK workspace. Lockfiles were read as dependency metadata, not as an audit of downloaded third-party dependency source or live vulnerability advisories.
- Performance examples document sampled peaks, cache conditions and measurement limitations. Private clone-read artifacts use restricted output permissions, copies rather than hardlinks, symlink rejection, source/copied hash checks, redaction and measured-process identity checks.
- Session database comparison remains an isolated experiment, not a production storage migration; it checks histories/digests/tails and compares controlled durability/cache settings.
- Configurations retain ordinary component exports and explicit policy selections. The project-check configuration explicitly permits model-free deterministic operation; aggregate reference-component examples are marked fixture-only.

Core тесты документация и исторические границы

Proteus-core 9b07d03

Results

No additional production defect established from this test-only scope. Test source is evidence of intended assertions, not evidence tests pass at this HEAD.

- ACP test definitions cover initialize ordering, session separation, streaming text once, allow/reject/invalid approval choices, canceled unanswered approvals, nonblocking other sessions, EOF cancellation, unsupported content/methods, configured selectors and per-session model/reasoning persistence, timeout settlement and replay.
- MCP tests cover paginated discovery, correlated concurrency, persistent generations, ordinary RPC errors without restart, structured-only result persistence/model propagation, cancellation/drop/timeout teardown and explicit restart without replaying a mutation. Restricted MCP fixtures exercise setsid descendants, network toggle and invalid-grant failure without trusted fallback.
- Model process tests cover arbitrary export IDs, exact canonical request/response and opaque metadata preservation, long streams beyond callback-budget counts, backpressure and dropped-stream cancellation, generation-shared sibling preservation, optional-method feature acknowledgement, malformed catalog/quota shapes, and model-scoped hosted tools.
- Module-swap tests explicitly establish no extra slot is needed for search, context preparation or tool selection; arbitrary registered exports use the same contracts. Tool selectors may only narrow unchanged unique candidates and cannot reintroduce previously removed tools. Multiple exports share one process, callback-connected components need not be split, and dead components restart lazily on the next explicit request.
- Workflow checkpoint tests cover Rust/Python interoperability, revision fences, unbound tool results not changing root history revision, rejected-stale terminal success and protection against lost-ack rollback of committed preparation.

- Ownership tests cover live local/foreign writer deletion rejection, read-only coexistence, OS lock release on process death, no resurrection from cold handles, consumed creation permission after failed materialization, tentative lease release on rejected identity/recovery, and retention of legitimate owner leases.

Boundary qualifications

1. Restricted profile is not a sandbox for the entire agent. config_profiles.rs:101-148 explicitly keeps reference model and file-tools/git-tools/shell-tool/plan-tool/rust-lsp/skill-pack/memory-pack/codex-patch launch configuration equal to trusted defaults; policy-pack loses its policy export but remains otherwise unchanged. Lines 153-285 assert six restricted single-export processes (workflow, context, policy, hook, search tool, search context provider), with zero writable grants/network. This is an intended mixed-trust composition, not evidence of a newly discovered bypass.
2. Test declarations are not runtime qualification. Ownership slice header explicitly requires an outer whole-test process-group deadline. session_writer_lock.rs:69-78 synchronously waits for a helper marker without a local timeout, and its helper holds stdin indefinitely (41-44), unlike the newer finite/RAII ownership helper. Running this older test directly without an outer deadline can hang on a helper regression; this is a test-harness limitation, not a proven production lock deadlock.
3. Fixture mcp_roundtrip_model.py has a stale model/v12 docstring at line 1, while actual handshake correctly checks model/v13 at lines 15-16. Documentation-only discrepancy.

Concrete documentation defects

1. **Model contract version drift.** docs/architecture/modules.md:18 and process-module-architecture.md:430 advertise model/v12, while modules.md:77,525 and process-module-architecture.md:885,908,1010 specify v13. product/scope.md:47–49 confirms v13. A module author following the old table would implement the wrong strict version; existing strict handshake rejects incompatible exports rather than falling back.
2. **Installed binary instructions retain removed aggregate.** architecture.md:129–130 describes the portable proteus/proteus-reference-module pair; configuration.md:201–202 claims install.sh puts proteus-reference-module on PATH for its minimal example; second-pc-bootstrap.md:30–33 repeats this. README.md:85–88 instead documents standalone proteus-<id>-module executables and explicitly excludes the aggregate from delivery. testing.md:406–408 also refers to two executables. Existing aggregate test fixtures may legitimately use the aggregate; the defect is claiming it exists in the installed/portable package. Reconcile with the parent's installer source audit before calling an actual installation failure reproduced.
3. **Obsolete mandatory-model statement.** configuration.md:696–703 requires an active provider in all AppConfig despite describing a no-model-call project check. Its own :204–208, architecture.md:347–352 and runtime-and-events.md:445–451 allow no active provider and explicitly do not synthesize an adapter. This can mislead authors into adding a fake model to an otherwise model-free assembly. Absent-model calls fail explicitly, so optional selection is not silent model fallback.
4. **Disabled-tool builder inventory contradiction.** inspect.md:118–120 says disabling an ordinary tool removes it from the builder and re-enabling requires file edits. configuration.md:1552–1565 describes disabled process tools retained as read-only inventory with enabled=false, registered=false and a pack toggle. These are conflicting user instructions about the same builder capability. Disabled inventory does not itself authorize execution or expose tools to the model.
5. **Reload polling semantics drift.** runtime-and-events.md:2147–2153 says unchanged profiles are never reread at idle. configuration.md:1441–1449 says metadata is checked every 300 ms and semantic reread occurs after 3 s even if metadata did not change. The latter explicitly calls 3 s a preliminary scheduling tradeoff rather than

a hard deadline. Older statement encourages a false assumption that metadata-preserving edits remain invisible forever.

6. **Interrupted-turn producer inventory drift.** pack-contracts.md:19 says turn_aborted producer absent, whereas process-module-architecture.md:513–519 and runtime-and-events.md:1690–1697 describe Core's neutral interrupted_turns fact and coding.codex_loop's request-only marker. The distinction matters: Core does not synthesize a user message; the module formats the prompt marker, not durable conversation history.
7. **Doctor side-effect description inconsistent.** runtime-and-events.md:60–64 says handshake belongs to conformance/runtime, not read-only doctor. configuration.md:1630–1634 explicitly says doctor can start model/tool processes and bootstrap describe/list/handshake while making no model request or behavioral turn. The distinction is important for users who interpret 'read-only' as no process execution.
8. **Runtime guide retains superseded transport descriptions.** :700 calls ACP future despite :110–198 documenting implemented ACP v1. :1917–1923 describes UI queue updates from Steering events and reconnect history fetch, whereas :1238–1270 specifies versioned pending snapshots as authority, and :825–843 specifies SessionSnapshot reconnect without separate /history. These stale passages should defer to the newer detailed contract rather than encourage a second inconsistent client state machine.

Contract promises and explicit limits relevant to the constructor

- Core offers seven existing process slot families: workflow/model/context/policy selected singly; tools/context providers/hooks ordered collections. No new search/memory/patch/compactor/tool-exposure/subagent slots are needed. Search, memory, patches and preparation are tools/providers/hooks. Authority is slot plus invocation context, not module ID. The universal registration of new contract families remains planned, not implemented.
- Component Runtime v2/wire v3; workflow/v20, hook/v6, tool/v6, model/v13, context/v3, context_provider/v4, policy/v2. Strict current schemas and no hidden legacy reader are promised. SDK alpha stability is not future-version compatibility.
- PreparedAssembly couples plan and registry in one immutable epoch. Admission captures effective model/reasoning/permissions with the registry. A running execution retains the capture across configuration changes.
- Model-optional standalone process Workflow and detached execute_tool exist. This does not imply public arbitrary typed task admission, durable generic ExecutionSettled, program-counter checkpointing, or automatic crash continuation. Workflow still uses AgentTask/AgentOutput; a separate registry call does not provide full task lifecycle.
- Trusted workers share user OS rights. Opt-in Linux restricted launch limits one configured export with explicit filesystem grants and network disabled by default; setup fails closed. MCP has an explicit generation-wide variant. The restricted profile is mixed trust, not whole-agent isolation. Allowed host callbacks can have effects beyond a worker's direct OS grants. Cooperative invocation cancellation does not guarantee descendant cleanup.
- process-module-architecture.md:53–57 must be read together with :724–727: correlation IDs are not secret capabilities and malicious code in a shared trusted process can impersonate an active sibling identity. Separate processes/grants are required for distrust boundaries. This limitation is disclosed, not a newly discovered isolation bypass.

- Journal v21 and config snapshot v8 are strict. Durable effects have recorded requests/outcomes and transactional history semantics; a side effect without recorded result stays unknown. A lost ACK does not undo committed history. No automatic replay of unknown tool effects is promised.
- Workflow replay substitutes model/tool/context/hook recorded outcomes but runs real workflow/policy processes. It supports complete Success/Error turns, including model-free turns; rejects incomplete records, delivered root steering and external Cancel/Timeout timing. Prompt replay is a real provider call, with hosted tools opt-in. Neither should be described as blanket sandboxing of arbitrary selected code.
- AgentControl manages complete local stdio peers with bounded messaging and lifecycle. No durable agent tree, attach/reconnect, A2A migration or universal remote backend is claimed. Collaboration restricts role/isolation eligibility.
- Process conformance/module swaps show technical replaceability, not arbitrary module semantic compatibility or live agent quality. Product docs explicitly say full differential/live Codex task-and-cost parity remains unproven and future work is not automatically authorized by limitations.

Historical conclusions not current defects

- desktop-app-2026-10-12.md (679 lines) and ui-extensions-2026-10-12.md (865) describe the former Tauri/Leptos/WebKit client, JavaScript UI API v4, shared DOM extension lifecycle, ZIP packaging, retained settings, authenticated server transport and old aggregate delivery. These are historical clients, not evidence of defects or current GPUI functionality. Their performance/UI tests are reported historical evidence, not this audit's executed tests. DOM/fixture observations are explicitly distinguished from native GPU measurements in the text.
- roadmap-through-2026-08-31.md (1,615) preserves staged ExecutionScope/BoundModel/BoundTools/admission migration, ownership separation and eventual process peer cutover. Old memory/v2, BoundMemory and direct-user /remember (987–1026) are superseded by the current tool-only memory path. Old instructions to keep schema-incompatible sessions outside active storage (664–668) likewise must not override current skip-without-delete behavior. Historical model adapters, compactor/search/memory/renderer slots and phase stop-gates are not today's work list.
- subagent-web-ui-handoff.md (220) expressly marks its task-card and mid-turn readback defects as fixed (3–13,194–204). The old in-process child/thread model and workflow-owned task events cannot be applied to the current AgentControl implementation without rereading current source.
- codex-parity-audit-2026-07-14.md (235) records historical P0/P1/P2 findings at older hashes. It explicitly closes native-tool ownership/cancellation findings in the July 18 update (121–125) and ordered-phase loss in its September update (18–23). Its 12-slot/native ABI inventory is obsolete. None of its old severity labels establishes an open defect at the pinned HEAD.
- dogfood-freeform-tool-loop-2026-07-22.md (60) describes a concrete prior 42-request loop caused by custom/function surface mismatch; records explicit capability validation and function-profile correction. It states missing raw SSE prevented attribution between model and proxy. No fresh reproduction was performed.
- dogfood-readiness-checkpoint-2026-07-23.md (75) records fixed cold transcript omission of non-success settlement and an intentional cancel causing an eval failure classification despite a successful control/readback gate. Its historical tool/plugin counts do not describe current coverage.

- pi-vs-proteus.md (523) proposes a 30-day product-pivot experiment and then explicitly marks that experiment canceled in the header. Its prescribed freeze/pivot dates, numeric thresholds, native ABI and SubagentRunner design are not current requirements or authorization.
- extensibility-cost-model-2026-07-16.md (88) offers qualitative time estimates, not measured current implementation cost. It uses the old dylib/process migration state.
- memory-research.md (117) is an older FFI/MemoryPolicy blueprint. Its backend latency figures explicitly are rough engineering estimates, not measured SLA. Scoped callbacks and curated facts versus history are reusable concepts; removed MemoryPolicy, carry-forward and ABI machinery are not live implementation.
- pi-extension-composition-2026-08-07.md (85) separates composition from process transport and acknowledges direct OS authority of trusted modules. It argues for ordered contributions with validation and branch-aware state; current typed hooks are a later implementation, not justification to add unrestricted hooks now.
- deepseek-harness-lessons-2026-08-21.md (207) preserves superseded R1 sequencing and subsequent completed v2 cutover status. Typed boundaries, canonical request evidence, cancellation and sandbox-versus-process distinctions remain useful historical rationale. It does not authorize a new provider/peer/scheduler migration.
- platform-expressiveness-after-runtime-v2-2026-08-22.md (205) puts algorithmic freedom inside components and strict authority/lifecycle at the host boundary. It explicitly distinguishes private implementation, tool, profile and genuinely new host semantics, and rejects treating research as authorization. Its five-agent matrix and remote/background directions remain proposals.
- agent-runtime-reconstructions.md (47) distinguishes compatible from inspired and mechanical conformance from behavioral evidence.
- prime-agent-process-lessons-2026-08-06.md (52) explicitly depends on a supplied report, with no independent upstream verification in that historical slice. Useful lessons concern scoped callbacks and capability-level health checks, not copying another runtime.

DeepSeek research document reading audit

No new current Proteus production defect established by this historical document.

Evidence boundaries:

- Lines 3–7 explicitly classify it as upstream research, dated 2026-08-21 at DeepSeek commit b150a551b8d465e31e418e1b2eaf5e79bbb7d28e, not the current Proteus contract.
- Dependency direction, event-sourced state, adapter normalization, reversible lifecycle and fail-closed behavior are comparative design observations. The proposed module/package list in lines 837–894 is advisory and does not justify adding Proteus host slots.
- The P0 label for upstream HTTP fetch SSRF at lines 672–682 is the research author's context-dependent severity assessment, not a verified current Proteus vulnerability. Trusted in-process plugin and node:vm limitations at lines 684–702 likewise do not establish that Proteus promises hostile-plugin confinement.
- Lines 803–807 explicitly withhold quantitative performance and cluster-scaling conclusions; no benchmark numbers can be imported from this document as measured evidence.
- Lines 827–835 already qualify community reports as signals from older upstream releases, not proven defects on the pinned upstream revision.

- The upstream source map is mostly pinned but its repository README link is a moving repository landing page. This reading audit confirms the checked-in document contents, not correctness or current status of all cited upstream facts.

6 Modules инструменты workflows и policy

Proteus-modules 9899e02

Приоритетные дефекты shipping кода

M1 P1 one shot shell наследует протокольный stdin

tool/shell-tool/src/lib.rs260–297 создаёт процесс shell/bwrap, переназначает stdout/stderr278–281, но не stdin. Для std::process::Command::spawn стандартное поведение stdin. inherit. В standalone process module fd0 занят JSON-RPC. Поэтому обычный shell вызов с cat/read либо программой, читающей stdin, конкурирует с SDK за протокольные байты: может поглотить следующий RPC/cancel frame или зависнуть до внешнего timeout. Это не необходимость интерактивности: one-shot shell не предоставляет interactive handle; exec_command pipes отдельно явно используют Stdio::null() в unified_exec/process.rs273.

Исправление: явно закрывать stdin one-shot пути; интерактивность оставлять PTY пути. Проверка: реальный worker, shell с чтением stdin, параллельный RPC и cancel; ни один protocol frame не должен попасть в дочернюю команду. Integration suite проверяет закрытый stdin только exec_command/write_stdin, не one-shot shell.

Источник: Источник tool/shell-tool/src/lib.rs#L260-L297

M2 P1 context pack запускает Git без защит ReadOnly Git

context/context-pack/src/lib.rs320–324 делает прямой Command::new("git").args(["status",…]).output(); git_diff_chunks355–365 вызывает git_output, который382–383 делает такой же прямой запуск git diff. Нет protect_readonly_git, core.fsmonitor=false, --no-ext-diff/--no-textconv. Код tool/git-tools/src/lib.rs180–206 намеренно включает все эти защиты. Следовательно, автоматический сбор контекста может исполнить настроенные fsmonitor, diff/textconv или активные clean/process filters, хотя read-only git tool такой путь отключает/отклоняет. Для codex_context git_diff включён по умолчанию (config.rs183–191), git_status также включён в repo_aware.

Условие: Git checkout с соответствующей активной конфигурацией/атрибутами и подходящим изменённым файлом. Не утверждается, что любой git status запускает произвольный код; проблема. конкретные поддерживаемые Git helper paths до tool policy/approval.

Исправление: один защищённый Git helper для context и tool, без скрытого запуска content filters; сохранить явный отказ на неподдержанную фильтрованную семантику. Проверка: synthetic filter/textconv/fsmonitor с marker, context build не должен оставлять marker.

Источники: Источник context/context-pack/src/lib.rs#L320-L397 Источник tool/git-tools/src/lib.rs#L180-L206

M3 P2 plan_execute_review теряет внутренний план после durable ACK

plan_execute_review.rs102–118 добавляет финальный текстовый план только в turn.model_messages и выходит до persistent_messages.extend на121. with_workflow_phase (metadata.rs61–89) добавляет только metadata, не Request scope. После первого execute tool call checkpoint_tools189 приводит к durable acknowledgement. scaffold/preparation.rs69–83 принимает ack.messages как durable history; merge_request_context104–110 сохраняет из прежнего model context лишь сообщения, у которых все parts

имеют Request scope. Обычный plan response. Conversation scoped; его нет ни в ACK, ни в request-only сохранении. Следующий execute request, а затем review, уже не содержат план.

На detached ACK revision=None ошибки не видно. Unit FakeHost использует именно default detached ACK; integration model_failure_recovery содержит warm/cold plan_execute_review tool checkpoints, но не проверяет наличие текста плана в последующих HTTP requests.

Исправление: явно сохранять workflow-private plan при canonical ACK, не публикуя его как финальную историю пользователя. Согласовать его scope с compaction. Проверить plan → tool → tool result ACK → next execute request и compaction на этой границе.

Источники: Источник workflow/coding-workflow/src/plan_execute_review.rs#L102-L122 Источник workflow/coding-workflow/src/scaffold/preparation.rs#L69-L113

M4 P2 path policy зависит от написания одного файла

policy/policy-pack/src/opencode_policy.rs198–219 берёт сырые path arguments и передаёт relativize. Эта функция245–255 делает только slash replacement и строковый strip_prefix(cwd); не убирает ./ и не проверяет границу компонента пути.

Конкретный сценарий с уже использованной в тестах формой rules: global allow; edit с pattern src/* → ask. write_file path "src/main.rs" требует Ask, а эквивалентный workspace-relative "./src/main.rs" не совпадает и получает Allow. Файловый инструмент открывает тот же файл: pinned Core23883bd tool_support.rs147–162 safe_relative_path явно пропускает Component::CurDir (строка152). Дополнительно "/ws-other/file" ошибочно режется относительно cwd="/ws", хотя это другой каталог. Это не утверждение о полноценном shell AST; это ошибка path representation.

Исправление: согласовать нормализацию path policy с файловым tool и применять component-aware prefix. Не нормализовать этим путём command strings. Проверки: src/x, ./src/x, абсолютный путь в cwd, sibling-prefix и traversal формы должны давать корректное одинаковое решение там, где указывают на один файл.

Источник: Источник policy/policy-pack/src/opencode_policy.rs#L198-L255 Источник crates/proteus-contracts/src/tool_support.rs#L147-L162

M5 P2 gate ограничивает выполнение но не созданные OS threads

workflow/coding-workflow/src/codex_stream/in_flight.rs24–43 создаёт scoped thread для каждой пришедшей batch, а gate.enter вызывается уже внутри потока37. tasks растёт до drain47–64. Gate81–116 сериализует несовместимые вызовы, но ожидающие batches уже занимают OS thread/stack и каждые20ms проверяют cancellation. Быстрый stream с большим числом завершённых tool items и медленным первым tool накапливает threads, вплоть до spawn failure; с parallel=true нет численного cap и на активные threads.

Это локально доказанный admission gap; полный upstream/global cap числа completed items не установлен. Наличие mpsc(64) в прочитанном Core runtime ограничивает backlog, а не число уже снятых из канала и принятых batches. Не выдавать отсутствие найденного global cap за математически бесконечный вход.

Исправление: admission до thread spawn, ограниченный executor/queue с backpressure и сохранением порядка commit/results. Проверка: burst намного больше текущих пяти вызовов, медленный первый tool, cancellation и bounded thread count.

Источник: Источник workflow/coding-workflow/src/codex_stream/in_flight.rs#L24-L43

M6 P2 Ptyxis bounded preview сначала читает весь output

tool/shell-tool/src/ptyxis.rs104–108 tee пишет полный stdout/stderr в capture files без byte cap. read_bounded_file268–274 затем делает fs::read всего файла и только после этого BoundedBuffer::from_bytes. Таким образом 64KiB preview не ограничивает временный RAM или disk usage; гигабайтный output сначала окажется в Vec. В отличие от обычного pipe reader, который получает небольшие chunks, этот путь создаёт полный buffer до truncation.

Условие: явно выбранный внешний Ptyxis path, не обычный exec source_capture. Его bounded source capture с quota. другой, более аккуратно реализованный механизм.

Исправление: читать bounded head/tail по seek/metadata либо chunked; отдельно задать disk cap/обозначить ограничение capture. Проверить большой output при маленьком preview и measured peak RSS, не делать performance claims без замеров.

Источник: Источник tool/shell-tool/src/ptyxis.rs#L260-L275

M7 P2 context diff budget применяется после полной материализации

context/context-pack/src/lib.rs355–375 собирает четыре полных Git outputs; git_output383 использует Command::output; sections.join также создаёт полную строку, только затем truncate_to_bytes. Малый git_diff_max_bytes/max_context_bytes ограничивает передаваемый bundle, не peak RAM и не объём чтения dirty repository. Может быть особенно заметно до первого model request.

Исправление: bounded output collection с явным truncation и cancellation/deadline; учесть статус процесса и stderr. Связано с M2 общим helper, но отдельная correctness/resource причина.

Источник: Источник context/context-pack/src/lib.rs#L350-L395

Меньшие несоответствия и дизайн риски

1. Context schema сужает расширяемость: runtime lib.rs167–176/codex.rs11–21 принимает arbitrary external provider ID, но config_schema.rs33–61 описывает providers как enum только builtins. В schema-driven UI нельзя выбрать внешний provider, поддерживаемый runtime. Это schema/runtime mismatch; конкретный UI effect должен подтвердить UI worker. search_provider/memory_provider отдельно правильно объявлены text.
2. package.py13 импортирует tomllib,63 использует hashlib.file_digest, оба требуют Python3.11+, но README39/54 и catalog говорят только Python3. На Python3.10 импорт завершается до CLI. Нужен явный minimum или совместимый fallback. Severity P3 документация/установка; не считать это аварией основной Rust runtime.
3. write_file67–80 и edit_file165–175 открывают workspace_file_for_write и write_all; pinned Core helper использует CREATE|TRUNC. После ENOSPC/EIO исходный файл может остаться усечённым/частичным. Нет staging/rollback. Это реальный риск сохранности, но API явно truncating и обещания atomic write здесь не найдено; не называть нарушением обещанной атомарности. Подход staging+rename потребует сохранения symlink/descriptor safety.
4. OpenCode policy split_command227–242 намеренно наивен: нет shell AST/quoting/substitution semantics. Это явно записанное ограничение порта, не полная upstream parity и не доказательство универсальной command sandbox. Не смешивать с конкретным path bug M4.
5. Codex patch overwrite/move-to-self/частичное применение явно задокументированы как pinned upstream semantics и подтверждены тестами. Не выдавать их за случайную новую regression. Подозрение о CRLF несоответствии отклонено после чтения pinned OpenAI Codex file_update.rs.

Research не shipping дефекты

research/dcp README прямо исключает эксперимент из стандартных профилей, runners и portable. Его собственный worker.mjs имеет unbounded used IDs / readline framing, createDcp держит session state в Map без eviction; это нужны условия для production readiness, не дефект обычного SDK runtime. Node minimum README22.18+ не совпадает с engines transitive ini7 (^22.22.2…), но runtime достижимость этой peer ветки не проверялась; не утверждать, что эксперимент обязательно не запускается.

research/tool-output-artifacts/src/lib.rs194–211 проверяет symlink только у parent directory, после чего tokio::fs::write следует существующему leaf symlink. Например заранее существующий .proteus/tool-outputs/shell/call-output.txt, указывающий наружу, выводит запись за workspace без гонки. Есть также collision у sanitize_path_segment (call/1 и call_1). Это исключённая research library, не source_capture production implementation. Исправлять перед возвращением в runtime; test сейчас проверяет directory symlink, не leaf.

Расширяемость что уже соответствует цели

- Самостоятельные components/<id>-module: узкие main wrappers, implementation зависимость только своего пакета, общий module-runtime не знает implementations.
- exports.json ограничивает bindable exports; registry/dispatch соблюдают slot/module identity. Model aliases отдельно разрешены wildcard.
- Aggregate proteus-reference-module. test fixture, исключён из default build/delivery; не нужно добавлять новый module в обязательный монолит.
- package.py и catalog берут metadata из component packages: discovery без центральной hardcoded карты. Добавление готового slot implementation не требует изменения Core.
- Hook adapter Claude использует публичный JS SDK и hook/v6, честно фиксирует strict_subset; inspection не исполняет scripts, unknown selected output fail-closed, explicit execution opt-in.
- Workflow checkpoint ownership, typed failures, request-only context, tool result bindings и cancellation не спрятаны в UI.
- Source capture exec особенно аккуратен: quota/admission до spawn, bounded queue/read, owner+generation checks, prefix completeness отдельно от process exit, нет false complete после storage error.
- Architecture gap не в количестве abstractions: повторяющиеся direct Git paths и разные представления временной/долговечной истории уже дают наблюдаемые ошибки. Нужны точечные общие helpers и contracts, не новый universal module framework.

Тесты и границы уверенности

Рекомендованный порядок исправлений: M1/M2, затем M3/M4, затем admission и bounded collectors M5–M7. Добавлять regression на реальной boundary, а не source-text assertion. После исправлений отдельно запустить repository gate и targeted synthetic cases с разрешения на исполнение.

Modules адаптеры моделей

Proteus-modules 9899e02

Подтверждённые дефекты и риски

MM1 P2 Anthropic понижает canonical System message до user

- anthropic.rs:161–170 объявляет supports_system_role=true.
- request.rs:174–214 преобразует в assistant только Assistant, все остальные роли в user.
- request.rs:138–147 формирует top-level system исключительно из request.instructions; System messages туда не попадают.
- request.rs:309–315 дополнительно объединяет соседние сообщения одинаковой преобразованной роли.

Минимальный статический пример: canonical messages [System("system instruction"), User("question")] без instructions превращаются в один user message с двумя text blocks, без top-level system. Это не отказ от неподдерживаемой роли, а незаметная потеря приоритета инструкции. Обычный путь через request.instructions сохранён правильно; не утверждается, что все текущие workflow создают System messages. Нужен тест именно публично заявленной System-role ветки и перенос в provider system либо честный отказ/изменение capability.

MM2 P2 Anthropic незаметно повторяет уже начатый inference после ошибки SSE

- anthropic.rs:233–255 сначала отдаёт дельты, затем при любой ошибке чтения SSE безусловно вызывает complete_response с исходным запросом. Нет проверки уже выданного содержимого и нет переключателя opt-in.
- anthropic.rs:187–204 выполняет новый POST /v1/messages. Это второй inference, а не дочитывание первоначального ответа.
- Первоначальные дельты несут ID из anthropic/stream.rs:27–32,195–199; fallback создаёт другое CanonicalMessage в response.rs:88.

Сценарий: сервер принял запрос, вернул часть текста, затем поток оборвался с transport error. Адаптер отправит платный запрос второй раз и представит новый ответ после уже виденных дельт первого. Статически доказан повтор сетевого inference и отсутствие стабильной identity между ними; двойное выполнение локального инструмента или конкретный UI-эффект не утверждаются. Общий regression adapters/stream_fallback_tests.rs:15–79 проверяет replay, но первый поток в нём содержит лишь data: {} и не выдаёт пользовательские дельты. В OpenAI тот же риск хотя бы отделён явным диагностическим stream_error_fallback с default=false (openai.rs:83–86,127–135,312–330).

MM3 P2 Anthropic SSE подменяет неизвестное отсутствующее завершение успехом и не сохраняет семантику tool use

- anthropic/stream.rs:292–318 собирает все ToolUse независимо от stop_reason; некорректный JSON аргументов превращается в Null. Для любого нераспознанного/отсутствующего stop_reason возвращается ToolCalls, если вызовы есть, либо Stop, если их нет.
- anthropic/response.rs:21–28,67–83 в non-stream пути сохраняет Unknown и принимает вызовы только при явном stop_reason="tool_use".

- anthropic/stream.rs:58–60 игнорирует повреждённый JSON события; после утраты message_delta оставшийся message_stop тоже станет Stop/ToolCalls.

Минимальные статические трассы: message_stop {} сразу возвращает успешный пустой Stop; text delta + message_delta {stop_reason:"future_reason"} + message_stop возвращает Stop, хотя эквивалентный non-stream ответ даёт Unknown. Структурная проверка downstream уже не видит исходный provider status. Также max_tokens с ToolUse возвращает Length **с вызовами**, тогда как non-stream их удаляет. Regression anthropic/tests.rs:334–365 защищает только non-stream путь.

Ограничение важное: НЕ утверждается выполнение оборванного max_tokens-вызова. На той же ревизии Modules single_loop.rs:81,93–113 и plan_execute_review.rs:115–124,177–191 исполняют только FinishReason::ToolCalls. codex_stream.rs:54–60 валидирует terminal response до accept; в доступном, но не идентичном pinned Core canonical_response.rs:102–104 Length отвергается. У Anthropic нет промежуточных MessageCompleted для обхода этой terminal-ветки. Главный дефект M3. ложная нормализация статуса и расхождение адаптерного контракта, а не доказанный обход разрешений.

Дополнительные конкретные наблюдения для triage

MM4 P2 кандидат OpenAI API теряет SamplingConfig

openai/request.rs:30–117 не читает request.sampling и не отправляет temperature/top_p ни для обычного API, ни для подписочного Codex. openai/tests.rs:495–543 даже задаёт SamplingConfig(0.2,0.9), но не проверяет эти поля. Наблюдение подтверждено полным чтением mapper, однако намеренное Codex-соответствие может объяснять подписочную ветку. Для обычного openai/openai_compatible остаётся silent omission публичного запроса. Перед исправлением следует явно развести transport/model support и не добавлять неподдерживаемые sampling-поля в Codex без согласования.

MM5 P3 кандидат OpenAI ToolCallDelta использует provider item ID как call ID

openai/stream.rs:27–45,47–65 записывает item_id в canonical ToolCallDelta.call_id, а response/item.rs:73–87 строит финальный ToolCall по настоящему call_id. Реальная пара id="fc_1", call_id="call_1" даёт разные identities у дельт и завершённого вызова; никакого сопоставления added-item→call_id в stream_state.rs нет. openai/tests.rs:1024–1037 закрепляет именно item ID при наличии обоих полей. Это подтверждённое различие данных, но пользовательский ущерб зависит от потребителя ToolCallDelta; конкретный UI failure не установлен, поэтому не повышать severity без проверки consumer.

Проверенное без новой существенной находки

Весь Codex OAuth путь прочитан, включая lock/атомарную запись, PKCE/state, однократный refresh после 401, отмену refresh и ограничения endpoint/redirect. Прочитаны quota/catalog coalescing/deadline/cache, request identity/cache headers, hosted tools, SSE idle/retry, opaque thinking и cold-history tests. В этих частях новых подтверждённых существенных ошибок сверх перечисленного не установлено. Это не результат запуска тестов и не гарантия отсутствия дефектов.

Modules интеграционные тесты и недостающие oracles

Proteus-modules 9899e02

Конкретный дефект теста

T1. Неограниченное ожидание ответа handshake в model_exports.

components/process-module/tests/model_exports.rs:16–33: worker запускается через std::process::Command, затем синхронный BufReader::read_line без timeout. child.kill() и child.wait() вызываются только после завершения read. Если регрессия initialization оставит процесс живым без строки/EOF, тест не провалится по локальному deadline, а зависнет до внешнего таймаута. Это устанавливается непосредственно по управляющему потоку. Требуется ограничить handshake ожидание и гарантировать cleanup child также на timeout/panic. В этой работе тест не запускался.

Пробелы regression проверок

Это ограничения доказательной силы suite, не самостоятельные утверждения о наличии production-багов.

G1. Нет oracle сохранения плана после durable checkpoint.

В codex_model_resume/model_failure_recovery.rs:75–101, 371–417, 507–545 действительно есть warm/cold сценарии coding.plan_execute_review, включая tool execution и model failure. Однако initial planning reply использует тот же success_response(), что и последующие ответы, а assert_requests/assert_resume_request (104–209) проверяют только tool call/result, порядок continuation и equality retries. Наличие planning context после tool checkpoint не проверяется. codex_compaction/recovery.rs:203–233 также включает этот workflow, но проверяет terminal compaction failures, не retention плана. Для production-подозрения проверки исходников нужен отдельный уникальный plan sentinel и assertion его сохранности в следующем execute/review request после ACK с durable revision. Не следует писать, что plan_execute_review вообще не тестируется.

G2. Нет проверки, что one-shot shell не читает stdin протокола worker.

codex_model_resume/terminal.rs:20, 95–126, 271–278 проверяет pipes/TTY и closed stdin только для exec_command/write_stdin. One-shot shell вызывается в других тестах с printf, sleep и локальными mutation-командами; ни одна не пытается читать обычный stdin. Следовательно suite не ловит source-подозрение о наследовании worker stdin в one-shot spawn. Нужен process-level сценарий stdin-reading shell плюс живой sibling RPC/cancel, а не только unit test команды.

G3. Проверка parallel ordering не проверяет ограничение ресурсов очереди.

codex_model_resume/stream_recovery/parallel_execution.rs:14,63–81,141–163,182–214 задаёт ровно пять вызовов: a, b, write, serial, c; одновременно активны два до exclusive fence. Oracle хорошо доказывает ordering, overlap, approval и ordered drain; parallel_execution/cancellation.rs доказывает отсутствие dispatch queued calls после cancel. Но burst из большого числа completed tool items, число waiting threads/queued allocations, backpressure и их возврат после cancellation не проверяются. Это не доказательство bounded RAM у workflow; source-аудитор отдельно проверяет spawn-before-gate.

G4. One-shot cancellation oracle не доказывает уничтожение потомков.

Тест codex_model_resume/terminal/cancellation.rs:143–201 называется cancel_one_shot_shell_kills_descendants_and_records_canceled_turn, но команда имеет вид printf started > started; sleep 1; printf late > late-marker. Проверяется только отсутствие late-marker через 1200 ms. Если убить родительский shell, а sleep оставить живым, marker тоже не появится: этот oracle

пропускает утечку потомка. В соседнем long-poll тесте (45–99) проверяется PID ребёнка через /proc, но для one-shot такой проверки нет. Нужен PID/termination oracle либо marker, который записывает сам дочерний процесс.

G5. Повреждение/частичная запись обычных file tools не моделируется.

codex_model_resume/direct_tool_surface.rs:53–100,293–367, model_failure_recovery.rs и context_overflow.rs проверяют обычную успешную запись и её journal/recovery. patch_transaction.rs:59–112,155–181 проверяет отклонение invalid patch plan до частичных эффектов и сохранение различий алгоритмов. Эти patch-обещания не распространяются на write_file/edit_file. В рассматриваемой integration-зоне нет fault injection короткой записи/ENOSPC/write error после truncate для file-tools. Это связано с отдельным source-подозрением; сам production-путь здесь повторно не аудировался.

Ограничения среды и независимости

- conformance.rs:443–485 исполняет rg-search round trip только если rg --version успешен; иначе остальной тест проходит. Это явное условное покрытие, не баг поиска. Положительный результат данного теста без подтверждения наличия rg не доказывает search interop.
- independent_modules.rs:67–148 проверяет все 18 реальных пакетов, manifests, tool lists, model descriptors; реальное tool invoke в этом тесте только plan_tool. Отдельный independent_modules/source_capture.rs проверяет shell capture на standalone и aggregate, а context_profile_swap.rs:34–71 использует настоящие prepared artifacts для profile. Большая recovery matrix через direct_tool_surface::fixture_config (172–179) заменяет production binary paths aggregate reference executable. Поэтому её нельзя представлять как полную runtime-проверку всех 18 shipping executables.
- Внешние fixtures из pinned Proteus-core задаются PROTEUS_TEST_HARNESS_ROOT; в ledger этой работы включены только файлы snapshot Modules. Core SDK/runtime и внешние profiles не подменялись и не считались прочитанными здесь.
- Unix crash cases и Linux descendant assertions защищены cfg; их наличие в исходниках не доказывает выполнение на другой платформе.

Сильные имеющиеся проверки

- Cross-process canonical DTO, policy visibility/escalation, hook scope, nested same-component callback, concurrent sibling и сохранение generation/PID после targeted cancel.
- Реальная cold process restart, не только drop/rebuild; non-idempotent effect log и два crash windows до/после durable ToolResultRecorded; неизвестные effects не превращаются в выдуманные durable results.
- JSON/SSE и subscription flows; exact raw tool arguments, encrypted reasoning, commentary/final phases, message IDs и cold transcript.
- Completed vs unfinished SSE items, duplicate item delivery, early tool execution до terminal SSE, disconnect/idle/deadline, retry budget/provider delay/quota, cancellation без позднего retry.
- Parallel/exclusive/serial fences, approvals и denial, out-of-order durable results с model-order history; cancellation queued work.
- Compaction scopes, explicit render mode, input retention, request-only versus durable state, overflow/retry failure semantics, cold recovery, replay и source journal immutability.

- Patch interception сохраняет model call и effective execution, проверяет workdir, allow/ask/deny/unavailable policy и отсутствие повторного эффекта на replay.
- Terminal pipe/TTY input, nonzero exit, long polling >30s, export override deadline и cold journal/replay.

7 Desktop catalog packages и настройки

Proteus-desktop a608d93

Conclusion on one installation per capability

The existing separation is a useful foundation. Desktop already supplies one catalog UI and reusable installers for five kinds: process modules, client extensions, Agent Plugins, Skills and MCP configurations. Source export/build, ZIP validation, staged publishing, install receipts and profile reconciliation are present. It is not yet a single capability/bundle installer.

`Kind` describes five independent types. Each `Entry` has one kind/path and optional one source workspace/build command/artifact. There is no dependency graph, version solver, component plan or transaction spanning a module plus extension plus agent instruction. A client extension's `dependencies` describes Core/tool availability, while packaged `agent_instructions` links do not install a Skill or add it to the agent prompt. Installing a module registers its component but intentionally does not select tools/hooks/active slots.

Sources:

- Catalog schema, lines16–73
- Per-kind installation, lines75–156
- Packaged instructions UI, lines192–195
- Documented activation boundary, lines141–147

A narrow extension of the existing architecture is preferable to replacing it: capability manifest → explicit resolved component plan → pinned materialization → validation/build → staged installation → profile activation. Resolve all components to immutable revisions before work begins. Preserve the distinction between installing files and choosing policy/tools/hooks. Define rollback and recovery for a partial multi-component operation, rather than calling a sequence of current single-item installers atomic.

Findings

D CAT 01 Tools parameter editors are not initialized on direct navigation P2 high confidence

`visible_modules()` creates editor scopes for Model, Slot, enabled Hooks and Modules, but has no Tools branch. `prepare_editors()` only initializes the returned scopes. The Tools page nevertheless renders the pack parameter disclosure. Its form renderer returns an empty div when the text control is missing; JSON mode also has no state to render.

Reproduction inferred from source: open fresh Settings → Tools → parameters of a pack with text/numeric/JSON settings. The editors are blank/missing. Visiting Modules first masks the problem because that page still eagerly initializes all tool/hook exports, although its own UI moved individual parameters away. Disabled hooks have a similar mismatch: their parameter disclosure is offered, but only enabled `draft.hooks` are prepared.

Impact: supported settings cannot be edited through their actual page, depending on prior navigation history. Boolean/select controls that do not need a retained editor are not necessarily affected.

Sources:

- Scope selection and preparation, lines169–242
- Tools renders parameters, lines143–150
- Missing form control, lines101–117
- Missing JSON control, lines118–124

Fix/test direction: initialize exactly the modules whose parameter forms Tools/Hooks may display. Add a direct-navigation test from fresh Settings, without pre-calling ensure_json or visiting Modules.

D CAT 02 Ready file catalog installation can mix stale metadata with a refreshed commit P2 high confidence

The normal non-source installer keeps the supplied Entry. git::with_files reads the latest mutable cache State and materializes that state's commit, without accepting the Catalog revision. A second Settings window can refresh the same source while the first still displays its earlier Catalog. The first window's install then combines its old entry/version/path metadata with files from the newer cache revision. The receipt records the old Entry version. The source-build/export path is better protected: it re-resolves against the selected snapshot.

Impact: wrong bytes under a displayed/recorded version, stale install offer operating against changed source layout, or installation failure for an entry which existed in the visible revision. This does not require treating the catalog as malicious; ordinary concurrent refresh is enough.

Sources:

- Cache publication, lines217–233
- Materialization uses current State, lines244–288
- Only source path re-resolves entry, lines60–74
- Receipt version, lines157–167
- UI retains selected Catalog, lines114–155

Fix/test direction: materialize all kinds by immutable selected revision; reject or explicitly re-resolve a stale offer. Add an interleaving test: load A, refresh B, install entry from A, verify bytes and receipt both identify A or the operation explicitly refuses.

D CAT 03 Extension ZIP import performs archive and filesystem work on the UI thread P2 high confidence

The extension page iterates selected/dropped archives and invokes the installer synchronously inside the UI update callback. The installer reads/decompresses/validates, stages files, replaces the package and rescans/reloads extensions before returning. Supported limits are 64MiB compressed, 128MiB expanded and 4096 records per archive; multiple archives are processed serially. Removal also recursively deletes/rescans synchronously.

Impact: normal large extension installation can make the window unresponsive. No duration or native reproduction is claimed. The catalog path already demonstrates a background preparation pattern that can be reused.

Sources:

- Import and chooser callback, lines84–128
- Synchronous install/rescan, lines136–143

- Common archive processing

Fix/test direction: perform read/decompression/validation/staging on a background executor; publish lifecycle changes on the UI thread after completion. Exercise a large legitimate archive while checking that keyboard/window events continue to run.

D CAT 04 Session setting requests escape the bounded wire queue P2 conditional workload risk

Every incoming set_session appends to Runtime.settings: VecDeque. Only one HTTP setting update runs at a time. Incoming wire reservations are released when message handling finishes, so the bounded transport does not bound this secondary queue. A fast trusted producer or repeated setting changes behind a slow API can retain arbitrarily many old requests and values. The SDK independently retains pending commands until ACK, so the same backlog can exist at both ends.

Sources:

- Queue field, line29
- Enqueue and serialized processing, lines49–104
- SDK peer-confirmed: Proteus-desktop-extensions d4d05d2e65a11053675be26399ccdda2df3faa3e, sdk/src/host.rs459–475, pending.push at468–469.

This is not an untrusted-process security boundary claim. It is an end-to-end resource-control gap, separate from the SDK stdin-reader queue in the opposite direction. No production RSS growth measurement was performed.

Fix/test direction: a finite command budget with explicit rejection, or semantics-aware coalescing for replaceable same-session/same-setting values while completing superseded request IDs. Test a blocked API and a burst of settings, including generation changes.

D CAT 05 Partial build compatibility checks protocol integer but not required features P2 conditional compatibility gap

Partial portable build and runtime preflight compare only the numeric extension protocol. Runtime manifest validation also requires every required_features item to be supported. Because additive features keep protocol4, a newer extensions-only build can pass preflight against an older ready Desktop, then be rejected by Desktop when loaded. A Desktop-only build against a newer ready extension set has the symmetrical risk.

Sources:

- Partial portable protocol checks, lines67–85
- Runtime preflight, lines19–41
- Runtime feature validation, lines380–385

The current official pinned release is not claimed incompatible. This specifically concerns selecting components from different compatible-integer revisions, a documented supported workflow.

Fix/test direction: record Desktop's supported feature set in portable provenance and compare the required set during partial-build/preflight validation; test equal protocol with a missing additive capability.

D CAT 06 Empty graph schedules an endless redraw loop P3 high confidence

Graph starts with fit=true. fit() returns immediately for no nodes, leaving the flag true. Every render therefore schedules another next-frame notification. Empty graphs are valid under the protocol validator. The loop stops only when a nonempty graph is fitted or the view disappears.

Sources:

- Initial fit and empty return, lines31–98
- Repeated notification, lines101–107
- Empty graph is accepted, lines655–670

Impact is unnecessary UI CPU/battery use for a valid empty state. No concrete shipped producer or native CPU measurement was verified. Clear/defer fit appropriately for an empty graph and re-enable fitting when data arrives.

Cross repository and peer owned findings

- Duplicate session splice: Desktop sends SessionUpdate once per context (runtime/scopes.rs128–145), while SDK keeps entries per session_dir. Shared runtime can own distinct scopes for one session, so SDK without revision/dedup can apply one splice multiple times. Exact production Plan failure is NOT proven: its activity and popup reuse the same Host, and its embedded settings are isolated. Scope should be described as the public multi-surface extension contract. SDK reviewer owns the integrated finding.
- SDK unbounded stdin reader: Desktop outbound reservation ends after stdin write/flush, not after SDK processing, so Desktop's transport bound does not bound an SDK mpsc receiver that drains stdin ahead of synchronous handlers. SDK reviewer owns this opposite-direction finding.
- Shortcut reset collision: UI reset directly calls preferences.shortcuts.reset(command), src/ui/settings/shortcuts.rs53–63, with no conflicting-binding cleanup. Desktop-client reviewer owns the headless reset/validation/quarantine chain.
- Terminal exit clears retained screen: independently read declarative/terminal.rs; Signal::Exited replaces Running(Terminal) at109 and render266 shows only notice. DESIGN.md914–916 explicitly expects a completed terminal to retain its final screen. Desktop-client reviewer owns this finding.

Strengths observed in source

- Buildable-source export is pinned to a selected commit, uses direct Git blobs instead of filters, preserves valid file modes/symlinks, checks limits and stages publication. Build uses explicit argv, an external Cargo cache, a timeout, process-group cleanup and bounded logs.
- Shared archive handling rejects traversal, .git paths, duplicates and symlinks, applies count/size limits, validates staging before replacing the old package, and has rollback paths. Plugin versions use immutable directories rather than destructive in-place overwrite.
- Catalog source refresh retains the previous successful state on failure. Source and per-entry diagnostics remain distinct. UI and profile receipts preserve enabled state and stable plugin data_key across version replacement.
- Save acknowledgments use edit_revision and rebase newer drafts. Catalog profile updates reconcile late independent edits and report conflicts instead of silently replacing the current draft; dedicated tests cover this logic.
- Extension wire transport has explicit application-wide budgets and per-frame limits. Outgoing overflow fails explicitly instead of blocking UI or silently dropping traffic. Content surfaces use one active request plus latest replacement with source_revision checks.
- Settings/Preview runtimes have explicit lifetime and capability scopes. Nested content cannot regain Core credentials or workspace through rehosting. These are capability/lifecycle guarantees, not a process sandbox.

- SVG raster and source-key caches are bounded, visible-paint admission avoids offscreen rasterization, retained scene leases have explicit ownership, and tests distinguish cache limits from total CPU/GPU memory.
- Release sources and contracts are pinned. Development sibling/env override resolution is intentionally different from release exact-SHA preparation. Manual/tag-only CI is an explicit repository policy, not an accidental missing PR trigger.

Minor documentation inconsistency

examples/catalog/plugins/review/README.md:4 says catalog installation adds a draft requiring Save. Current catalog installation writes the profile immediately; examples/catalog/README.md87–92 and root README580–581 correctly distinguish catalog installation from ZIP draft import. Update the sample README to avoid misleading the user. Root README750 still describes only alacritty as the terminal engine although README293 correctly states default Ghostty.

Verification boundaries and rejected candidates

- No build/test/native window execution, external catalog network exercise, package installation or persistent user setting change was performed.
- Tests in this zone were read, including full Python release/build tests and Rust inline tests; their existence is not proof of passing at this SHA.
- External dependency internals and vendor code are outside this assigned zone; the overall audit must state separate coverage rather than inherit this zone's completeness.
- The stable-ID callback hypothesis was rejected after cross-checking Extensions PROTOCOL.md411–412: interactive controls explicitly require stable id/action/type/purpose. Do not report it as an API bug.
- Direct extension execution, Core credentials for Core-dependent trusted packages, developer sibling source selection, and immediate catalog profile installation are documented architecture choices. They are not presented as vulnerabilities.
- No attempt was made to claim safety, race freedom or full correctness from a static complete read.

Additional crates tabs review

All 24 tracked files under crates/tabs were read in full, including all tests. The crate keeps tab, pane, preview, pin/history, split normalization, fixed-size lock, document retarget and independent ChatSpaces state separate from native rendering. Tests explicitly cover MRU, close/reopen, empty room beside locked panes, serde validation, document collisions, child-chat context, window merging, root deletion and hidden areas. No repository tests were run.

A client-owned P3 UI finding was confirmed at this boundary: automatic split eligibility uses the entire viewport (src/ui/tabs/mod.rs177–188), while actual geometry uses tab_area (view.rs31–35). crates/tabs/src/open.rs46–73 trusts the callback and, with one chat pane, returns Split when it is true; open.rs27–29 inserts immediately and tree.rs192–201 creates two equal weighted panes. Thus at scale1, a1000px window but660px actual tab area can split into approximately329.5px panes despite the760px threshold. This is a narrow sizing/UX defect, not a data-loss claim, and no native UI reproduction was executed. The client report owns this item to avoid duplicate findings. Fix by calculating eligibility from the actual tab area and divider-adjusted geometry, with a sidebar-visible regression case.

Desktop UI client transcript и terminal

Proteus-desktop a608d93

Результат

10 дефектов, выводимых из исходников: 5 P2 и 5 P3. Это статический аудит, а не утверждение о воспроизведении в запущенном приложении. Сценарии ниже являются рекомендуемыми регрессионными проверками. Код, тесты, сборки и скрипты репозитория не запускались; исходники не менялись.

D UI 01 P2 Сброс одной клавиши создаёт невалидные preferences

Цепочка. Shortcuts::add, 172–188 переносит chord от прежнего владельца. reset, 201–203 лишь удаляет override и тем самым возвращает default, не отбирая chord у нового владельца. Реальная кнопка «Как по умолчанию» вызывает этот reset напрямую: settings/shortcuts.rs, 53–63. validate, 232–237 затем отвергает такой конфликт, но save_at, 672–679 его сохраняет без валидации. При следующем старте load, 583–585 перемещает файл в .broken-*, а prefs::init, 74–76 применяет весь Preferences::default.

Сценарий. Назначить Ctrl+N другой команде (например, NewTerminal), затем сбросить NewSession. Обе команды получают Ctrl+N. После сохранения и перезапуска пользователь теряет применение всех настроек, а не только этих двух shortcuts. Файл сохраняется в резервной копии: это не безвозвратная потеря данных.

Исправление/проверка. Reset должен согласованно разрешать владение возвращаемыми default chords. Добавить add → reset → save → load тест с проверкой всех остальных preferences. Не ограничиваться блокированием сохранения без понятного сообщения.

D UI 02 P2 Approval не даёт посмотреть полное разрешаемое действие

requests.rs, 22–23 ограничивает preview 24 строками и 6000 байтами. render_approval, 201–202 передаёт тело через clip_block; 264–270 рисует только полученную обрезанную строку. Scrollbar прокручивает обрезанный текст, не оригинал; expand/full-view здесь нет. Дополнительно clip_block, 65–79 для единственной длинной строки возвращает обрезанный префикс без маркера: после lines=1 rest=0.

Сценарий. Запросить разрешение длинной команды (>6000 байт в одной строке) или большого patch (>24 строк). Хвост действия недоступен для просмотра в approval перед «Разрешить один раз»; для одной строки даже не видно, что часть скрыта.

Исправление/проверка. Сохранить полный payload для отдельного полного просмотра/копирования, оставить bounded preview, всегда сообщать об обрезке. Проверить одно- и многострочные команды/patch, Unicode на границе и доступность полного тела с клавиатуры.

D UI 03 P2 HTTP команды и история могут ждать бесконечно после connect

server_agent, 11–16 устанавливает только connect timeout 10s. get_with, 314–337, command, 340–361 и save_config_builder, 267–283 не задают request/response/body deadline. Это не предположение о библиотеке: закреплённый Cargo.lock содержит ureq3.4.2; verified upstream tag3.4.2 → 2e9ef24a80e1e7ecd0e6604f98ba49aceb7ec322; Timeouts::default оставляет global/per_call/send_request/send_body/recv_response/recv_body в None.

Сценарий. Сервер принимает TCP, но не заканчивает HTTP headers или body. История/операция остаётся в ожидании, background blocking call не заканчивается и UI не получает ошибки. Не распространять этот вывод на все запросы: image и remote-terminal API задают собственные тайм-ауты.

Смежный lifecycle. spawn_pump, 161–181 замечает закрытый receiver только после следующего SSE event/heartbeat; stalled соединение может удерживать старый pump thread/socket и после закрытия UI. Для штатного живого сервера heartbeat даёт завершение, поэтому это условие отказа, не нормальный постоянный leak.

Исправление/проверка. Конечный deadline обычных API-запросов; отдельный cancellable SSE/read policy. Fixture, который принимает соединение и зависает в headers/body, должен приводить к ограниченному ожиданию и восстановимому UI состоянию. Команду с неизвестным результатом нельзя автоматически повторять.

D UI 04 P2 Stop игнорируется до появления Codex turn id

RunState::running/begin, 21–34 считает starting активной работой; composer, 125–132 показывает Stop. В то же время send, 355–375 резервирует current_turn как пустую строку до ответа turn/start. stop_codex, 466–478 при отсутствии непустого id просто выходит, не сохраняя намерение отменить.

Сценарий. Замедленный turn/start, нажатие Stop сразу после Send и до turn/started/response. Запуск после этого продолжается без отмены; пользователь должен нажать Stop повторно.

Исправление/проверка. Сохранять cancel intent для текущего запуска и отправлять interrupt после получения его id; защищать intent от переноса на новый turn. Проверить Stop-before-id, поздний ответ и завершение до ответа.

D UI 05 P2 Локальный терминал теряет весь экран при выходе shell

State, 23–28: Terminal хранится только в Running. Signal::Exited, 104–119 заменяет его пустым Exited, освобождая terminal state. Render, 261–277 для Exited показывает только сообщение о завершении, а snapshot получает только для Running.

Сценарий. В локальной terminal-вкладке выполнить вывод, затем exit (либо shell сам аварийно завершается). Последний экран и scrollback больше нельзя прочитать/скопировать. Это finding относится к локальному terminal widget, не ко всем удалённым терминалам. DESIGN.md914–916 содержит правило сохранения последнего экрана завершённого терминала, но соседний раздел обсуждает также инструментальные блоки; непосредственное доказательство здесь поведение локального widget.

Исправление/проверка. У Exited должен оставаться terminal/snapshot и доступный readonly scrollback без ввода/курсора до restart/close. Проверить normal exit и failure после уже показанного вывода.

D UI 06 P3 Home End и Enter из фильтра выбирают неверный конец списка

session_list.rs, 108–110 передаёт MIN/MAX для First/Last. Но при отсутствии текущего видимого элемента 169–173 использует только знак offset: MIN выбирает last, MAX выбирает0. Enter из фильтра, 180–185 также вызывает MIN, хотя обещает первую строку.

Сценарий. Фильтр исключил активный чат, ручного cursor нет, совпадений несколько. Enter/SelectFirst выбирает последнюю строку, SelectLast первую. При наличии видимого current ветка работает правильно.

Исправление/проверка. Разобрать команды First/Last до fallback для отсутствующего current; отдельные tests для пустого курсора и отфильтрованного active.

D UI 07 P3 Несколько Core окон дублируют системное уведомление

У каждого Workspace свой attention tracker (workspace.rs). chats.rs, 504–514 вызывает attend до active-chat filter; комментарий подтверждает, что каждый session stream несёт каждое обновление. attend, 22–30 дедуплицирует только через self.attention и вызывает notify::show, когда ни одно окно приложения не активно.

Сценарий. Два Workspace с Core streams, приложение в фоне, один чат завершается/ждёт ввода. Одинаковый status transition независимо проходит оба tracker и вызывает два уведомления. В одном окне повторные копии корректно подавляются.

Исправление/проверка. Общий tracker/notification owner на приложение или WindowGroup с ключом session+transition. Проверить два окна, несколько streams одного окна и background→foreground.

D UI 08 P3 Инерция прокрутки смешивает разные окна

wheel.rs, 32–35 содержит один thread_local GLIDE без WindowId. 108–115: пока running=true, второе окно добавляет delta в общий remaining и не планирует свой callback. step, 137–168 отправляет накопленное движение через Window первого callback. Аналогично fling.rs, 52–89 и 123–141.

Проверена точная GPUI семантика: gpui-fast0.1.12 tag→07b42ebf1f43088b8b74a7d4fca6b95e44e04265; Window::on_next_frame, 2746–2752 хранит callback в self.next_frame_callbacks и планирует именно self.platform_window.

Сценарий. Начать wheel glide в A, быстро переместить указатель и прокрутить B до завершения glide A. Движение B попадает в A, а B его не получает. Для Linux touchpad тот же недостаток FLING имеет более длинное окно инерции.

Исправление/проверка. State и callback должны принадлежать WindowId; смена окна должна отменять/переназначать исходную инерцию. Проверить чередование двух окон и закрытие первого во время glide.

D UI 09 P3 Один tooltip overlay ошибочно разделён всеми окнами

tip.rs, 185–192 возвращает один глобальный Entity<TooltipOverlay>. Он монтируется в каждом Launcher (launcher.rs, 323–325, 347–355). Vendor прямо определяет TooltipOverlay как per-window (tooltip.rs, 169–184); render, 301–326 не проверяет владельца окна и рисует общее content с общими trigger_bounds.

Сценарий. Показать hover-tooltip в A при двух видимых окнах. При redraw B оно получает тот же tooltip в координатах A. Для keyboard-help tip.rs, 201–211 дополнительно позволяет фоновому B счесть focus A устаревшим и скрыть общий tooltip. Вендорная реализация изучена, это не предположение о скрытой per-window фильтрации.

Исправление/проверка. Отдельный overlay и owner/suppression state на окно. Проверить hover/focus A при redraw B, закрытие A и смену активного окна.

D UI 10 P3 Auto split считает ширину окна вместо доступной области tabs

can_split, 177–188 вычисляет rects по window.viewport_size. Реальный renderer использует измеренный tab_area (view.rs, 31–35), из которого уже вычтены rail/sidebar. Headless open_target, 46–73 доверяет callback и не перепроверяет размер; tree.rs, 192–201 создаёт два равных Weight(1).

Сценарий. Окно1000px при scale1, sidebar+rail оставляют tab_area660px. Открытие документа рядом с единственным chat-pane автоматически разрешает split, хотя доступная область ниже порога760px;

получаются панели примерно329.5px. Неправильный autosplit, не data loss и не нарушение ручной команды split.

Исправление/проверка. Использовать tab_area и тот же spaced layout, что renderer; до первого измерения выбирать безопасный fallback. Проверить sidebar open/closed, scale и nested splits.

Не превращённые в подтверждённые defects риски

- Core SSE pump использует unbounded async_channel в connection.rs155. Ограничение UI batch не ограничивает накопленные события/байты между producer и GUI. Это структурный ресурсный риск при быстром producer/медленном GUI; RSS, достижимую скорость и практический предел в этом аудите не измеряли.
- Remote terminal Retry после ошибки Display::new лишь очищает error и вызывает ensure_read (view.rs63–65,266–270); отдельного recovery-test для создания Display нет. Это дополнительный кандидат, не включённый в10 findings.
- Большой diff синхронно подготавливается на UI, а сжатые draft images не проходят те же явные decoded limits, что отправленные изображения. Без измерения и полной конкретной decoder-семантики не называю это доказанным freeze/OOM.

Проверенные и отклонённые гипотезы

- **UserInputDrafts после detach:** не bug. В точном gpui-fast0.1.12 Context::subscribe_in341–363 не сохраняет старый WindowHandle; helper 952–968 вызывает cx.with_window(subscriber.entity_id()). InputState render также выполняет rehost. Отсутствие явного пересоздания этой subscription само по себе не доказывает ошибку.
- **Child TurnStarted/SteeringDelivered routing:** не bug по проверенному producer path Core. Producer на Core9b07d03 не пересылает эти события в родителя (whitelist process/turn.rs199–213).
- **Archive→live сохраняет старую высоту одинаковых row keys:** scroller.set_archived(false) уже вызывает remeasure.
- **Codex deletion не доходит в другие окна:** windows watcher обрабатывает ThreadDeleted, гипотеза отклонена.
- **Terminal Failed мгновенно затирается Exited:** не подтверждается, signal consumer завершает цикл после Failed. Не включать.

8 Extensions SDK Git и production packages

Proteus-desktop-extensions d4d05d2

Результат

Подтверждены четыре самостоятельные проблемы: повторное применение session delta при нескольких subscribing contexts, отсутствие сквозного ограничения входящей очереди SDK, превышение предела Desktop одним агрегированным Git diff и удержание working-tree diff без отдельного лимита памяти. Первая относится к поддерживаемой комбинации поверхностей пользовательского расширения; её проявление в штатном Plan не доказано. Остальные затрагивают общий SDK или штатный Git.

EXT 01 Одна session delta применяется повторно в общей проекции SDK

Приоритет: высокий для целостности данных расширения. Доказан конфликт producer/consumer; живой процесс не запускался.

SDK намеренно хранит один SessionContext с entries на сессию и перемещает его при смене контекста, а не создаёт отдельную историю на окно: scopes.rs:15–57. Однако Desktop создаёт отдельную подписку для каждого context и отправляет копию одного SessionUpdate из каждой подписки: runtime/scopes.rs:108–147. Фильтр подписки проверяет только session_dir: host.rs:123–139.

Для snapshot Desktop уже делает дедупликацию по сессии, scopes.rs:182–197. Для update такой проверки нет. SDK каждый раз выполняет splice без revision/event id: host.rs:681–702.

Достижимый сценарий: пользовательское расширение с session_events: true, одновременно badge и panel, root session совпадает с command target. Badge использует for_command_chat, panel обычный workspace host; identity общий, scope различается. См. badges/mod.rs:38, host.rs:141–157, declarative/mod.rs:202–238. Аналогичная граница существует для panel/document с различными scopes одного runtime.

Минимальный логический пример: начальная история [A], update append с start=1, removed=0, entries=[B]. Доставка через два context даёт [A,B,B]. Замена или удаление тоже не являются идемпотентными. Это доказательство поведения splice, а не результат выполненного integration test.

Ограничение вывода: штатный Plan activity и его popup получают один Host clone. Две подписки штатного Plan не подтверждены; утверждать его фактическую порчу истории нельзя.

Исправление: одна подписка/доставка на (runtime, session) либо монотонная revision и гарантированная дедупликация перед общей проекцией. Тест должен создавать два scopes одной сессии и проверять append, replace, remove, rehost, закрытие одного scope и смену session.

EXT 02 Ограниченный pipe Desktop не ограничивает очередь SDK

Приоритет: высокий для устойчивости при медленном потребителе. Рост очереди доказан устройством кода; RSS/OOM не измерялись.

SDK использует неограниченный std::sync::mpsc::channel<Value>; отдельный поток непрерывно читает stdin, парсит JSON и складывает сообщения в канал: host.rs:802–817. Обработка идёт последовательно в другом потоке: 818–844. Синхронный HTTP может удерживать handler до 15 секунд по умолчанию: 304–325. Git явно использует 120/130 секунд для hooks/remote: main.rs:22–24, вызов 417–422.

У Desktop действительно есть bounded writer: 128 frames / 32 MiB. Но его reservation освобождается после write/flush в stdin, не после применения сообщения SDK: process.rs:315–325. Быстрый reader SDK снимает pipe backpressure, пока handler занят. Поэтому ограничение Desktop не является ограничением pending Values расширения.

Исправление: лимит числа и encoded/retained bytes в SDK queue плюс обратное давление до handler; не выбрасывать упорядоченные splice/actions. Coalescing допустим только для действительно заменяемых state/render сообщений с учётом ordering.

Регрессия: медленный обработчик, быстрая серия крупных snapshots/updates, явно проверяемый предел pending bytes, восстановление обработки и точный порядок событий.

Смежный, отдельно найденный в Desktop дефект: очередь настроек VecDeque<Setting> не ограничена после снятия transport permit. SDK тоже не ограничивает outstanding set_session: host.rs:459–475. Не следует выдавать это за доказательство, что каждый штатный виджет генерирует произвольный поток

команд; архитектурный API такой поток допускает. Этот обратный путь учитывается в отчёте Desktop отдельно, не как второе доказательство той же входящей очереди SDK.

EXT 03 Все изменения Git может превысить frame cap и остановить расширение

Приоритет: средний. Достижимый численный конфликт лимитов, без запуска.

Git ограничивает агрегированный документ 40 файлами, но не суммарным размером. Каждый patch целиком дописывается в одну строку и затем один diff root: document.rs:164–199; CHANGES_LIMIT=40 в строке19.

Core допускает отдельный diff до 512 KiB: workspace.rs:17–18, git/command.rs:152–153, diff.rs:30–47. Превышающий отдельный patch корректно становится too_large; проблема в сумме допустимых ответов.

40 patches по 500 000 bytes дают 20 000 000 bytes ещё до JSON escaping. Desktop принимает максимум 16 777 216 bytes на frame: process/io.rs:11–37. При превышении он вызывает stop: process.rs:303–307. Это не просто долгий рендер большого дерева, а ошибка транспортного контракта с завершением текущего процесса расширения.

Исправление: общий бюджет сериализованного frame с запасом под envelope и files, постраничный просмотр или явный частичный результат. Не повышать cap без оценки памяти обоих процессов. Регрессия: много допустимых отдельных patches, сумма вокруг16MiB, escaping и крупные имена файлов.

EXT 04 Working tree diff cache удерживает все просмотренные актуальные patches

Приоритет: средний. Это недостаток политики удержания, а не доказанная неограниченная утечка всех данных процесса.

Git.diffs. обычный HashMap по path, отдельный от исправленного history cache: main.rs:59–64. Любой ответ с revision копируется туда: document.rs:105–115. Poll оставляет запись, пока revision присутствует в changes: main.rs:385–407. Закрытие preview не удаляет запись: Git не переопределяет default closed, sdk/lib.rs:83–85. Session instance остаётся жив, пока существует владелец/контекст, instances.rs:19–27.

Последовательный просмотр многих изменённых, но затем не меняющихся файлов оставляет все patches в памяти при живом badge/panel. Core возвращает до1000 entries и до512KiB на patch, поэтому речь потенциально о сотнях MiB payload, не о фиксированном небольшом кэше. Это оценка допустимого retained payload, не прогноз RSS и не замер конкретного проекта. Исчезновение path, смена revision или окончательное освобождение session ограничивают жизнь записей.

Важно: immutable commit/history cache уже исправлен, имеет общий LRU 16 записей / 8MiB с объединёнными full/short aliases: cache.rs:1–16. Повторно заявлять старую проблему истории нельзя.

Исправление: отдельный entry/byte LRU working diffs и/или учёт владельцев открытых документов, сохранив revision invalidation. Регрессия: последовательный просмотр множества current paths при живом badge, закрытие каждого preview, bound и корректный повторный HTTP после eviction.

Тесты CI и доказательность

Статически найдены 211 Rust #[test] в SDK/production,46 в first-party experimental Rust,7 вызовов Node test() в worker.test.mjs,76 Python test definitions в основном tests и26 в experiments. Это количество определений, не выполненных/успешных тестов и не процент branch/line coverage. Тесты локального vendor patch считаются отдельно от123 файлов исходников; Vendor и содержащиеся в нём тесты дочитаны в итоговой фазе; они не запускались.

В репозитории есть полезные тесты shared session ownership, rehost, retained allocations, stale messages, revision invalidation, history cache budgets, renderer lifecycle, packaging artifact selection и13 standalone package wire flows. Однако перечисленные четыре сценария полноценно не закрыты: повторная доставка одной delta в общий session state, медленный handler с быстро растущей SDK queue, агрегированный Git frame около16MiB, бюджет working-diff cache.

Единственный workflow намеренно ручной, в соответствии с AGENTS.md. Само отсутствие push/PR trigger не является ошибкой. check.yml:23–28 запускает Python discovery, проверку каталога и сборку пакетов. Он не вызывает Cargo unit tests, Node worker tests или Python experimental tests. Bundle test действительно строит и запускает13 пакетов, test_bundle.py:34–47, поэтому «CI вообще не проверяет процессы» было бы неверно.

Отдельный no-Core/no-session wire test Terminal имеет skipUnless на packages/terminal/bin/terminal: test_terminal_local.py:22–23. Чистый checkout и staging сборка bundle этого файла не создают; обычный manual workflow пропускает именно эту проверку. Manifest-проверка и общий terminal wire остаются.

Python tests/schema.py документирован как mirror закрытой Desktop-схемы, но устарел: rich_text допускает только html, отсутствуют inlines; picker не знает title/back_action/search; stack не знает transition, svg не знает typography. См. schema.py:1–22. Это test-maintenance gap, не доказательство отказа production renderer: tests/wire.py не валидирует roots через этот mirror, authoritative conformance находится в Desktop. Желательно один источник schema либо conformance-корпус положительных примеров новых features.

Архитектура и границы

- SDK владеет JSONL stdin/stdout, routing по context/view, общими session projections, instance per session, HTTP через Core, pending settings и построением декларативных widgets. UI, terminal и rasterization исполняет Desktop, не SDK.
- 13 production packages: analysis, context, files, git, json-render, markdown-render, mermaid-render, mode, model, plan, quota, terminal, usage. Все manifests и package definitions прочитаны.
- Markdown выдаёт нативные rich-text/content nodes; вложенный renderer живёт по контракту Desktop. JSON renderer не получает разрешение на произвольный privileged UI от невалидированного source: используется свой catalog и budgets.
- Production Mermaid использует отдельный Node worker/Pele с ограничениями source/SVG и lifecycle/cache. Rust Mermaid и LaTeX находятся в отдельном opt-in experimental workspace; подменять ими стандартную поставку нельзя.
- Полностью прочитан собственный Mermaid patch 2292 строки: orthogonal candidates, diamond ports, bounded micro-detour cleanup, rounded outline clipping и тесты. Это существенный локальный слой поверх upstream; В итоговом покрытии vendor также прочитан полностью; собственный patch учитывается отдельно. Synchronous upstream render не получает hard wall-clock timeout от одного source-size budget.
- Packaging использует Cargo JSON artifact selection, selected package/develop/archive paths, atomic publication и self-contained bundle. Каталог проверяется отдельным скриптом. Shelved architecture находится вне production delivery и не должен оцениваться как активный UI.
- Исторические performance reports за29октября–2ноября прочитаны. Они явно ограничены synthetic ready processes, pinned binaries и sampled PSS/RSS; данные о снятии copies и bounded history cache не доказывают bounded whole-app RAM, долгосрочное отсутствие leaks или текущий throughput HEAD.

Extensions production Pele и experimental native renderer

Proteus-desktop-extensions d4d05d2

Четыре подтверждённые статическим чтением ошибки production Pele

P2 PELE 01 Gantt X start считает секунды миллисекундами

getStartDate для числовых форматов x и X вызывает new Date(Number(n)). Общий parser tC для X использует множитель 1000; getEndDate проходит через него. При dateFormat X задача с началом 1700000000 и концом 1700003600 получает начало в январе 1970 и конец в ноябре 2023 вместо одного часа. Источник: LF 86–87; codepoints 369746 (tC), 375197 (getStartDate), 375742 (getEndDate). Не запускалось. Исправление: единая интерпретация единиц измерения и тесты X/x.

P2 PELE 02 положительные pie сегменты меньше 1 вообще не рисуются

FN заменяет долю на 0 при fraction×100<1. Затем цикл выходит до построения пути и изменения угла. Для 999 и 1 маленький сектор исчезает, крупный занимает 99,9% круга, а легенда содержит обе категории. Это не только скрытая процентная подпись. Источник: LF 104, codepoints 450090 и 450737. Исправление: не отбрасывать геометрию, отдельно управлять видимостью подписей.

P2 PELE 03 autonumber 0 заменяется на 1

Sequence parser принимает Number(0), но renderer pk применяет i=l.start||i,s=l.step||s при начальных значениях 1. Явный start 0 теряется; step 0 также заменяется прежним шагом. Источник: LF 126, codepoints 545766–545789. Исправление: проверка undefined/null вместо truthiness; отдельно определить допустимость нулевого шага.

P2 PELE 04 treemap 1_000 превращается в 1

NUMBER2 принимает [0-9_\.\,]+, а qA удаляет только запятые и использует parseFloat. Подчёркивание завершает частичный разбор, поэтому соседние значения 1_000 и 1000 дают разные размеры в 1000 раз. Источник: LF 138, codepoints 624978–625014 и 625817–625863. Исправление: нормализовать разрешённые разделители или отклонять всю некорректную числовую лексему.

Все четыре примера выведены из исходников, не являются запущенными воспроизведениями. Полное доказательство и координаты suffix находятся в отдельном Pele report.

Экспериментальный native Mermaid приоритетные проблемы

P1 NATIVE 01 columns создаёт массивы без ограничения памяти

parser.rs:3852–3858 принимает положительный usize без верхнего предела. layout/block.rs:35–48 создаёт два массива f32 длиной columns, а 83–85 проходит по всем столбцам. Короткий block-beta с columns 1000000000 и одним узлом требует 8000000000 байт полезного содержимого массивов, несмотря на лимит входа 50000 байт и по 500 узлов/рёбер в adapter. Это размер, рассчитанный по исходнику, не измерение RSS. Возможны отказ выделения, abort или длительная обработка; ordinary catch_unwind не является защитой от allocator OOM. Требуются лимит columns до выделения и общий budget. Production Pele эту конкретную схему выделения не использует.

P2 NATIVE 02 Sankey независимые компоненты накладываются

layout/sankey.rs:132–138 задаёт всем y=0, 191–215 меняет только rank>=1. Для независимых равных связей A→B и C→D исходные узлы совпадают по координатам, соответствующие целевые тоже. render.rs:2006–2018 использует rectangles напрямую; 2023–2097 двигает лишь подписи. Нужна упаковка rank-zero узлов и disconnected components.

P2 NATIVE 03 XY bar строится от минимума а не от нуля

layout/xychart.rs:34–42 и 92–105 использует height=(value-min)/range×250. Для [-10,10] отрицательный минимум исчезает, а положительный максимум занимает весь plot. render.rs:3801–3806 напрямую выводит эти прямоугольники. Нужен общий zero baseline и отдельные направления положительных/отрицательных bars.

P2 NATIVE 04 обычный C4 Person конфликтует с SVG policy

layout/c4.rs:497–503 безусловно задаёт image_y=Some(y) для Person/ExternalPerson, даже без sprite. render.rs:4725–4743 создаёт image. Experimental adapter render.rs:71–115 отвергает все image. Поэтому обычный Person попадает в конфликт renderer/validator. Это отказ совместимости, не image-policy bypass и не доказанная XSS.

Другие source proven проблемы vendor engine parser

Следующие подробные доказательства, статические входы и ссылки сохранены в vendor-engine-report.md. Нельзя считать каждый parser-level дефект отдельно воспроизведённым через strict adapter: его валидация может остановить ввод раньше.

- ENG-01 P2: строгий validator ошибочно считает+ частью recipient y sequence activation shorthand и отвергает поддерживаемый parser ввод. validator:491–500, 324–356 против parser:1192–1222.
- ENG-02 P3, только dump: layout_dump:120–138 восстанавливает ID по первому совпавшему display label, теряя различие одноимённых subgraphs.
- ENG-03 P2, только standalone CLI: cli:378–405 принимает открывающий fence из 4 backticks, запоминает 3 и не закрывает блок; 349–375 отбрасывает его при EOF.
- PAR-01 P2: architecture malformed delimiters создают reversed string slice, parser:4186–4199.
- PAR-02 P2: flowchart A[/] создаёт slice 2..1, parser:6206–6217.
- PAR-03 P2: GitGraph branch/merge options становятся частью branch identity, parser:2738–2793.
- PAR-04 P2: Gantt dependency lowercased, исходный task ID нет, parser:2371–2389.
- PAR-05 P2: standalone class member не создаёт свой class node, parser:1408–1418.
- PAR-06 P2: block label с пробелом разбивается на phantom nodes, parser:3910–3936.
- PAR-07 P2: timeline continuation events теряются, parser:2159–2182.
- PAR-08 P2: sequence deactivation shorthand записывает recipient вместо sender, parser:5253–5263.
- PAR-09 P2: standalone flowchart label с двойным дефисом исчезает, parser:5474–5482.
- PAR-10 P2: quoted C4 label x=y после снятия кавычек становится named argument, parser:3402–3416.

Два malformed parser inputs доказывают panic path, но **не** crash всего процесса в интеграции: SDK host.rs:539, 848–856 содержит catch_unwind вокруг render. Отдельно allocator OOM из NATIVE-01 этим не

ограничен. Renderer escaping candidates не превращены в XSS-утверждения: XML/SVG adapter отбрасывает опасные/неподдерживаемые элементы.

Что не засчитано как доказанный end to end дефект

- Глубокая рекурсия native text math, Gantt fallback double-origin и sequence integer overflow требуют дополнительных проверок достижимости/границ. Оставлены кандидатами.
- Raw quadrant Y-label и pie outside label без escaping подтверждены локально, но сохранение конкретного raw input до renderer не проверено. Не считаются доказанным XSS или полным input→failure воспроизведением.
- Подозрение на state left-note clipping снято: общий finalize_graph_label_bounds учитывает notes и компенсирует отрицательные координаты.
- C4 zero boundary-row source path снят: parser требует num>=1.
- Pele usecase edge-0 collision снят: lexer/parser не допускают такой explicit relationship ID.
- Иконки из callback вставляются как SVG markup по API. Без подтверждения недоверенного provider это контракт доверия, не самостоятельная уязвимость.

Тесты и качество доказательств

Весь checked-in baseline 31 467 строк прочитан. Это исторические 215 fixture/metric records, а не текущий benchmark. Нет duplicate JSON keys или non-finite metric values. Абсолютные пути developer checkout требуют нормализации. Generic metrics неполно описывают pie/Gantt/quadrant/sequence: нулевые counts и score не доказывают пустой или корректный SVG; perfect ratio при нулевом знаменателе не доказывает покрытие. Layout иSVG metrics могут сильно расходиться. Все 215 records имеют layout_anchor_label_count 0.

Пример port_quality.rs:419–454 не включает parse failures в threshold_failures, а collector пропускает отсутствующие roots. Для строгого evidence gate надо требовать ненулевое ожидаемое число fixtures и отсутствие parse errors. Запусков этого примера не было.

Все 233 vendor/notices files дочитаны полностью. Проверка лицензий здесь ограничена чтением текста, не является юридическим заключением. Ни одно найденное ограничение или успешно прочитанный тест не доказывает runtime memory safety либо полное динамическое покрытие.

Pinned source links

- Pele Gantt
- Pele pie
- Pele sequence
- Pele treemap
- Native parser columns
- Native block allocation
- Native Sankey
- Native XY
- Native C4 layout

- Native C4 SVG
- Experimental adapter

Extensions доказательства native parser и renderer

Proteus-desktop-extensions d4d05d2

Integration severity and boundaries

The adapter checks source <=50,000 bytes before calling parse_mermaid_strict, then checks <=500 nodes and edges after parsing (adapter render.rs:12–29). Therefore the small parser panic inputs below reach the parser. The SDK wraps extension.render in guard (host.rs:539); guard uses catch_unwind and converts ordinary unwind panics to an error (848–856). PAR-01/PAR-02 are proven panic paths, but **not proven process crashes** in this integration. Their integrated priority is P2 (parser robustness/diagnostics), rather than the standalone reader's P1 label. No panic-abort build or runtime behavior was tested.

Generated SVG undergoes XML parsing and rejection of script/image/a/foreignObject and unsafe attributes (adapter render.rs:71–115). Missing renderer escaping therefore supports an invalid-output/render-rejection concern, not an established XSS finding.

Additional source proven findings

ENG 01 P2 Strict validation rejects supported sequence activation shorthand

For `sequenceDiagram\nparticipant A\nparticipant B\nA->>+B: hi`, the validator splits only the plain arrow and returns recipient `+B` (validator.rs:491–500), then rejects it against declared `B` (324–356). The parser explicitly supports `->>+`/`->>-` and consumes the marker as part of the arrow (parser.rs:1192–1222). Since the experimental adapter uses parse_mermaid_strict, this valid supported input fails before layout. Use the parser's arrow tokenization in validation.

ENG 02 P3 dump only Equal subgraph labels get the first subgraph s ID

layout_dump.rs:120–138 resolves each output subgraph ID by searching the original graph for the first equal label. Two distinct subgraphs with different IDs and the same display label produce the same ID in the dump, misidentifying the second group. Preserve graph identity/index rather than joining on display text. This affects layout dump metadata, not the adapter's normal SVG route.

ENG 03 P2 standalone CLI Four backtick Mermaid fences are not closed

cli.rs:378–405 accepts any run beginning with three backticks, strips the entire run to find `mermaid`, but records only a three-backtick fence. A matching four-backtick closing line leaves one backtick after slicing the recorded prefix, so is_fence_end returns false. extract_mermaid_blocks drops the unfinished block at EOF (349–375). A Markdown document with a normal four-backtick Mermaid fence yields no diagram. Retain the opening delimiter length and accept a closing run at least that length.

C4 verification limited to already read evidence

The proposed `UpdateLayoutConfig(..., 0)` source-driven modulo-zero path is **not established**: parser.rs explicitly requires `num >= 1` before storing c4_boundary_in_row_override (3247–3263). Thus do not report this as a Mermaid-source exploit of the experimental adapter. External config loaders can assign zero, but no additional execution or end-to-end verification was performed. Duplicate boundary IDs and recursive layout were observed

during completed reads, but no further source-path confirmation is included here. C4 Person emission is confirmed across layout, renderer and adapter in NATIVE-04 above.

Parser findings from the complete single file read

PAR 01 P2 integration priority Architecture malformed delimiters panic instead of returning a parse error

parser.rs:4186–4199

architecture-beta\nservice a][reaches node_part[start + 1..end] with start=2 and end=1, yielding the invalid range 3..1. Independently, architecture-beta\nservice a)(reaches the icon slice with reversed parenthesis positions. The dispatcher reaches this parser (78–82), preprocessing preserves these lines, and 4092–4093 passes them to parse_architecture_node. No delimiter-order check precedes either slice. This proves a Rust panic from short untrusted diagram text; caller/process impact depends on panic containment and is not claimed here.

PAR 02 P2 integration priority Overlapping flowchart shape delimiters panic

parser.rs:6206–6217

flowchart TD\nA[/] passes node-only parsing (447,5474–5482), split_id_label (6170–6177), and the [/ + /] checks. Raw [/] has length 3, so line 6210 slices 2..1 and panics. A[\] has the same problem in the alternate parallelogram branch at 6214–6216. Require non-overlapping delimiter lengths or checked slices and return a syntax error.

PAR 03 P2 GitGraph branch and merge options become part of branch identity

parser.rs:2738–2793

For branch feature order: 1, line 2739 uses the entire suffix feature order: 1 as the branch key despite separately extracting order at 2741. A subsequent checkout feature creates a separate branch with no head (2757–2773). Separately, after ordinary main/feature commits, merge feature id:"M" looks up feature id:"M", not feature, at 2779–2783; the resulting merge records only the current parent even though id is explicitly supported at 2796. Parse the branch token apart from supported attributes.

PAR 04 P2 Gantt dependency references are lowercased while task IDs are not

parser.rs:2371–2389

gantt\nTask A : A1, 2020-01-01, 5d\nTask B : after A1, 3d stores the first task ID as A1 (2388–2389), but extracts the dependency from lower, producing a1 (2371–2375). Lines 2311–2315 then create an extra node a1 and point the dependency edge at it. This also writes the incorrect dependency into gantt_tasks at 2295. Preserve the original token when removing the case-insensitive keyword.

PAR 05 P2 Standalone class member declarations never create their class

parser.rs:1408–1418

classDiagram\nAnimal : +String name only inserts a member into the members map (1408–1413). No graph.ensure_node occurs on this branch, and finalization iterates only graph.nodes (1418 onward). Therefore the returned graph is empty and the implicitly declared class and member disappear. Ensure the referenced class when accepting a member declaration.

PAR 06 P2 Block node labels containing spaces split into phantom nodes

parser.rs:3910–3936

block-beta\nA["Hello world"] is split by whitespace into A["Hello and world"] at 3910. Neither token has a complete bracket pair, so parse_node_token falls back to the entire first whitespace token (6129); the loop inserts two malformed IDs rather than A with its label. This contradicts supported labeled block parsing, including the neighboring named-group path which preserves spaces (3815–3824 and test 7112–7116). Tokenize outside brackets/quotes.

PAR 07 P2 Timeline continuation events are silently discarded

parser.rs:2159–2182

timeline\n2020 : Launch\n : Follow-up accepts Launch, but the second line has an empty time_part. All event collection is inside if !time_part.is_empty() (2164–2181), so Follow-up is discarded rather than appended to the pending 2020 event. Maintain pending_time while accepting event-only continuation lines.

PAR 08 P2 Sequence shorthand deactivates the recipient instead of the sender

parser.rs:5253–5263

sequenceDiagram\nA->>+B: request\nB-->>-A: reply parses the second marker as Deactivate (1215–1218) but unconditionally selects the last edge.to at 5256, i.e. A. The active lifeline is B, which sent the reply. Activation (+) targets the receiver; deactivation (-) must target the sender. Current IR records Activate B followed by Deactivate A.

PAR 09 P2 Standalone flowchart labels containing double hyphens disappear

parser.rs:5474–5482

flowchart TD\nA["before -- after"] has no edge outside its bracketed label, so masked edge detection does not match. It then reaches parse_node_only, which rejects any raw line containing -- before parsing the label. No node is inserted and no error is returned. The same node works when referenced as an edge endpoint because add_flowchart_edge directly calls parse_node_token. Apply the hyphen check outside labels or remove the redundant raw check.

PAR 10 P2 C4 quoted labels containing equals signs are reinterpreted as keyword arguments

parser.rs:3402–3416

C4Context\nPerson(a, "x=y") loses its label: parse_function_call strips quotes from every argument at 3357–3359, then parse_c4_args splits the now-unquoted x=y as a key/value entry and removes it from positional arguments (3407–3412). process_c4_line falls back to label a (3270). The parser must preserve whether an argument was quoted, or only recognize actual named-argument syntax before stripping quotes.

Cross file confirmations for the combined audit

- Activation markers are explicitly supported by the parser at 1192–1222: A->>+B: hi produces recipient B, not +B. Validator participant checks must apply the same tokenization.
- Multiline init directives are rejected by this parser too, not a parser/validator disagreement: validate_init_directives processes input one line at a time (90–105), and parse_init_directive requires the complete init syntax on that line (16–17,235–250).

Renderer report from the complete single file read

Confirmed consumer evidence for cross file findings

1. Sankey node overlaps are not repaired in rendering. Lines 2006–2018 directly translate each rectangle by node.x/node.y and use node.height. Lines 2023–2097 adjust only separate label_y values. Lines 2145–2156 directly consume link.start/end and thickness. Combine with layout review's rank-zero placement finding; layout/sankey.rs is included in the complete coverage.
2. XYChart negative bar geometry is not repaired in rendering. Lines 3801–3806 emit each bar's x/y/width/height without zero-baseline calculation. Combine with layout review's (value-min)/range finding; layout/xychart.rs is included in the complete coverage.

Renderer local defects and integration verification required

- Missing XML escaping of quadrant Y-axis labels: Lines 3451–3473 interpolate the first label line directly inside tspan, unlike x-axis labels via text_block_svg and its escape_xml call. A TextBlock containing R&D produces malformed XML. Parser/layout preservation of raw ampersands must be established for a user-input repro. Adapter validation means expected impact is rejected rendering, not established XSS.
- Missing XML escaping of pie outside labels: Lines 3121–3126 take category text from joined label lines when outside; lines 3258–3266 interpolate it without escape_xml. A raw ampersand in an outside category produces malformed XML. Requires confirming parser/layout preservation and outside-label condition for an input repro. Ordinary percentage labels are generated numeric strings and unaffected.
- C4 Person integration confirmation: Lines 4725–4743 emit an embedded image for Person/ExternalPerson whenever image_y is Some. Cross-file review establishes adapter rejects every image. Layout c4.rs:497–503 assigns image_y for ordinary persons; this closes the source-level chain for NATIVE-04. Not an external-load or XSS claim.

No independent proven production defect, process crash, or XSS is claimed from this file. Its embedded tests were read, not executed.

Extensions production Pele подробности и quality baseline

Proteus-desktop-extensions d4d05d2

Findings

1 Pie rendering drops positive slices below 1 P2

Location: LF line 104, the FN renderer. Fraction construction starts at codepoint **450090**, column **2410**; the early skip starts at **450737**, column **3057**.

The renderer builds O from each value divided by the total, but substitutes zero unless H*100>=1 and (H*100).toFixed(0)!=="0". The slice loop then executes if(re===0)return; before adding any slice path or advancing angle g. Consequently this threshold removes the slice geometry, not only its percentage label. Remaining fractions are not renormalized.

Static example: a pie with "Large": 999 and "Small": 1 accepts both nonnegative values. The small fraction becomes zero, so the small slice has no SVG path; the large slice covers only 99.9% of a revolution. The legend still lists both entries. Keep all positive fractions for geometry, with a separate visibility decision for percentage text.

2 Sequence autonumber ignores explicit zero start P2

Location: LF line 126, pk; codepoints **545766–545789**, column **14000**.

pk initializes the running number and step to 1, then handles the parsed sequenceIndex with i=l.start||i,s=l.step||s. The parser constructs numeric start and step using Number(...), and its NUM lexer accepts zero. Thus an explicit zero is replaced by the previous value.

Static example: sequenceDiagram, autonumber 0, then A->>B: First produces number 1 in this control flow, rather than 0. An explicit zero step is likewise ignored. Use an undefined/null check rather than truthiness where zero is an accepted value, or reject a zero step explicitly if intentionally unsupported.

3 Treemap accepts underscore numeric tokens but silently truncates them P2

Location: LF line 138. NUMBER2 token definition at codepoints **624978–625014**, column **536**; leaf conversion at **625817–625863**, column **1375**.

The token pattern is /[0-9_\.\,]+/y, which consumes 1_000 as one numeric token. qA converts it using parseFloat(t.expect(13).replace(/,/g,"")), stripping commas but leaving underscores. JavaScript parseFloat stops at the underscore, so the accepted text 1_000 becomes 1. That finite positive value then feeds treemap sizing and labels.

Static example: treemap-beta with sibling leaves "A": 1_000 and "B": 1000 assigns values 1 and 1000, rather than equal values. Normalize a deliberately supported separator consistently, or reject unsupported numeric spellings instead of accepting a partial parse. The same permissive numeric token also admits malformed punctuation, which should be validated rather than partially parsed.

Architecture and contracts observed

- This suffix contains the continuation of gitGraph parsing/rendering and complete modules for info, ishikawa, journey, kanban, mindmap, packet, pie, quadrantChart, radar, railroad, requirement, sankey, sequence, state, swimlane, timeline, treeView, treemap, usecase, venn, wardley, and xychart. It also contains substantial specialized layout code and the final module-registration/export statement.
- Renderer results consistently have {svg,width,height,links}. Layout is calculated and serialized to SVG strings; this suffix contains no browser execution or network fetch path. Responsive dispatchers (vt, xn, fs, hs) are called here but defined in the parent's prefix scope.
- The tail registers the diagram module list with hm(...$1) and exports PeleError, detectType, enableZoom, forgetTextWidths, mount, parse, register, registered, render, and supports. Implementations of those public functions are in the prefix, so no broader claim about their behavior is made here.
- Source labels, IDs, titles, and text are escaped through helpers such as nt, We, and Ce. Styles flow through Mt and related helpers. Those helpers' definitions are outside this assignment, so their complete safety behavior must be assessed using parent coverage.
- **Icon-provider trust boundary:** the icons callback's return is inserted directly as nested SVG markup in Ea (starts at codepoint 431308), and corresponding treeView/usecase rendering paths. This is an integration contract, not an independently established vulnerability: callers must know whether the provider is trusted and whether the prefix enforces any broader policy.
- Kanban ticket links pass through Xn and Ro, are serialized with escaping, include Vr(n), and are returned in the links collection. State click links pass through Xn, qn, and an explicit _blank target, with tooltip text escaping. Helper definitions are separately reviewed.

- **Sequence link limitation:** links / link / details directives are parsed and stored on actors, but the sequence renderer read here neither wraps actors in anchors nor returns actor links; it returns links: []. This is an observed supported-syntax/rendered-feature distinction, not labeled a defect without the consumer's documented expectation.
- info reports Pele version **1.12.0**.
- Railroad supports native, ABNF, EBNF, and PEG notations, converting them to a common terminal/nonterminal/choice/sequence/optional/repetition model with iterative traversal. Requirement and state/usecase renderers use the shared graph layout engine with compound groups and specialized marker/port logic.
- Sankey validates finite parsed link values, rejects graph cycles, treats nonpositive values as zero for layout, and only emits gradient references when idPrefix passes /^[\w-]+$/. It validates node names for invalid XML characters before rendering.
- TreeView normalizes box-drawing input and translates syntax-error positions back to source line/column locations. Usecase retains spans, validates node/relationship kinds and metadata, preserves JSON property ordering, and clears parser-global buffers in finally.
- Explicit bounded-work controls appear throughout: packet row cap 10000, radar tick cap 32, requirement style-copy cap 1000000, bounded small-layout collision optimization, and bounded Venn iterations. These observations do not substitute for a full hostile-input complexity review.

Withdrawn candidate

An initially suspected usecase explicit/anonymous edge-0 ID collision was withdrawn after following the lexer/parser: hyphenated edge-0 is a CSS_IDENTIFIER, while explicit relationship IDs require IDENTIFIER. Do not report that collision as reachable or proven.

Stored data checks

The JSON parses, with no duplicate object keys and no non-finite metric values. Its 215 unique fixture paths exactly match its 215 metric keys. They comprise 46 test fixtures, 83 benchmark fixtures, and 86 documentation comparison sources. Metric records contain 124–163 fields, so consumers should not assume identical field sets. The config value is a historical absolute path, not embedded config content.

Findings and interpretation limits

1. **This file is historical metric data, not current validation.** Fixture and config paths use /home/jeremy/mermaid-rs-renderer/.... A consumer must establish how those identities match the present checkout and config. The file alone does not establish the generating code revision, current fixture equivalence, current pass/fail status, runtime safety, or visual quality. Missing comparisons must not be treated as coverage.
2. **Layout and SVG measurements differ.** tests/fixtures/flowchart/ports.mmd records 31 layout crossings (line 2712) versus 40 SVG crossings (2825). benches/fixtures/flowchart_path_occlusion_maze.mmd records layout overlap length 7.336459999999967 (14782) versus SVG overlap length 1182.5030000000002 (14898). These are distinct representations, not interchangeable measurements or proof of a new regression.
3. **Generic graph metrics have diagram-type limits.** Twenty-five records have zero extracted nodes. For example, gantt/basic.mmd has zero nodes/edges (3539–3540), zero content width (3594), aspect elongation 1000000 (3598), and score 512 (3632), alongside nonzero labels/canvas area. Pie, quadrant, and xychart cases

show similar patterns. Sequence and mindmap cases can show endpoint-off-boundary ratio 1.0. Without matching metric implementation and rendered artifacts, these values do not prove empty or broken diagrams. Earlier shorthand describing these as “defects” was too strong; they are stored penalties/overlap measurements.

4. **Zero observations can produce apparently perfect ratios.** All 215 records have layout_anchor_label_count zero. In 115 records, owned-label mapping ratio is 1.0 with zero owned-label candidates. Monotonicity can also be 1.0 with zero classified edges. Read counts alongside ratios; the baseline does not demonstrate independent layout-anchor validation.
5. **Nearest-edge, owned-edge, and anchor measurements are not equivalent.** In docs/comparison_sources/flowchart_opaque.mmd, the ordinary alignment bad count is zero (23636), while owned alignment bad count is one (23643). Other records show nonzero owned-anchor penalties while ordinary alignment bad counts remain zero. Selecting only one metric can miss a different measured condition.
6. **Fill ratios are not necessarily bounded union fractions.** Seven records have node fill ratio above one. tests/fixtures/treemap/basic.mmd records 1.8311111111111111 (6360), consistent with summed nested rectangles rather than union area. Radar examples also exceed one. This is a metric-semantic caveat, not a proven arithmetic bug.
7. **Zero overlap or a single aggregate score is not a usability pass.** The 200-node benches/fixtures/flowchart_large.mmd has width 30718.82242 (12896), aspect 33.05751250546272 (12899), and zero label overlap (12936). Separately, docs/comparison_sources/quadrant_tech.mmd has score 512 (27578) and one label overlap (27580), while other quadrant cases with no label overlap also have score 512. Do not infer from score alone that every recorded metric is represented in it; the actual score formula was not examined here.
8. **Numeric and schema comparisons need defined semantics.** Small floating-point residuals occur in endpoint errors and journey scores. Exact equality may be intentional for serialization snapshots, but would otherwise need a justified tolerance. Missing conditional fields must remain distinct from observed zeroes.

Conclusion

Literal coverage of the assigned vendor data is complete, and source integrity is unchanged. These observations concern the stored baseline and how to interpret it. No current runtime, security, rendering, regression-test, or score-formula validation was performed.

9 Desktop vendor terminal и WGPU

Proteus-desktop a608d93

Confirmed defects in public vendor APIs

These are not claims of reproduced Desktop vulnerabilities. Searches of current first-party src/ and crates/ found no calls to the affected graphemes_buf, alloc::Bytes constructors, public sgr conversion, standalone osc parser, Search, or PlacementIterator APIs. Desktop's Ghostty dependency disables default features, including Kitty graphics. Production snapshot rendering uses the length-carrying UTF-8 path instead.

V GHOST 01 safe buffer API can write out of bounds

P1 library soundness. vendor/ghostty-vt/src/render.rs:696–708 accepts any mutable char slice and forwards only its pointer to GRAPHEMES_BUF, with no capacity check. Pinned C contract vendor/ghostty-vt-sys/src/bindings.rs:4471–4474 requires at least graphemes_len elements and writes the whole cluster. A safe consumer passing a shorter slice permits a native out-of-bounds write. The allocating graphemes() method at render.rs:681–685 uses the correct size. Fix the public slice method to compare its length against graphemes_len before FFI; add short/empty-buffer tests.

V GHOST 02 fresh Bytes exposes uninitialized memory as initialized u8

P1 library soundness. alloc.rs:92–107 calls ghostty_alloc and constructs Bytes without initializing its memory. Deref/DerefMut at 135–149 and to_vec at 121–124 expose those bytes as ordinary initialized slices. Pinned bindings.rs:4836 explicitly states memory returned by ghostty_alloc is not zeroed. Thus safe Bytes::new(n)?.to_vec() or indexing immediately can read uninitialized memory. Initialize the allocation, or use a MaybeUninit-based initialization API before exposing u8. No runtime reproduction was performed.

V GHOST 03 safe From conversion accepts arbitrary raw pointers

P1 library soundness. sgr.rs:225–234 implements public safe From<ffi::SgrUnknown> for Unknown and constructs slices from both pointers without validation or a lifetime-carrying owner. ffi is publicly re-exported (lib.rs:48), and SgrUnknown has public pointer/length fields plus all-zero Default (bindings.rs:4635–4661). Safe client code can therefore construct an invalid pointer or default/null struct and invoke this conversion, violating from_raw_parts requirements. Make the raw conversion private/unsafe and tie returned slices to an actual parser borrow. The neighboring public TryFrom<ffi::SgrAttribute> route should be audited together with it.

V GHOST 04 borrowed native data is tied to the wrong lifetime

P1 vendor API contract; no production call found. Search::selected_match at search.rs:164–174 and selections at 194–237 accept an ignored _terminal solely to assign a return lifetime, without checking it is the terminal bound to the Search. A search for A can return Selection tied to unrelated B, after which safe mutation/drop of A invalidates the GridRefs while Rust still permits use. The pinned C contract calls these untracked snapshots (bindings.rs:5611,5670). Existing TrackedGridRef::snapshot in screen.rs:196–202 checks terminal identity, illustrating the missing guard.

Feature-gated related issue: kitty/graphics.rs:294–304 returns PlacementIteration with lifetime elided to &mut self, not to the Graphics/Terminal argument; the iterator holds only a mutable PlacementIterator reference. Native placement data is invalid after terminal mutation (bindings.rs:2021). Retain a borrow of the terminal/graphics for the entire iteration. This is not enabled by Desktop's current default-features=false dependency.

Allocator variant: alloc.rs:85–89 stores a raw pointer to Allocator, and new_with_alloc at 98–107 imposes no lifetime. A custom DecodePng implementation can retain allocated Bytes beyond its callback; Drop at 128–131 then uses an expired callback allocator. The built-in decoder transfers ownership on success and is not asserted to trigger this failure.

V GHOST 05 standalone OSC parser contradicts pinned native API

P2 vendor functionality. osc.rs:72–77 panics on null from ghostty_osc_end although bindings.rs:1949–1950 specifies null for invalid/cancelled OSC commands; the Rust docs at 52–58 promise CommandType::Invalid. Also title extraction at osc.rs:112–114 instantiates generic get<T> (148–156) with T=&str, while bindings.rs:1913–1914 documents const char** output. Native writes a C string pointer, not a Rust fat pointer/length. This cannot

correctly construct the title; on the common layout the zeroed length remains zero. Use an optional handle and explicit CStr-to-str conversion with UTF-8 validation. Desktop's Terminal title getter is a different, length-carrying path.

V GHOST 06 resource license summary is inaccurate

P3 documentation/redistribution metadata, not a legal compliance conclusion. vendor/ghostty-vt/README.md:53–54 describes Ghostty's terminfo and shell integration scripts as MIT. Bundled bash/ghostty.bash:1–16, zsh/.zshenv:1–16 and zsh/ghostty-integration:3–18 explicitly identify GPLv3-or-later derivation from Kitty. Preserve and accurately enumerate per-file licenses in package documentation. No claim of incompatibility with this GPL Desktop application is made.

Boundaries and candidates not confirmed application defects

- GridRef::style (screen.rs:77–82) and Terminal::cursor_style (terminal.rs:568–570 via zeroed generic getter) use Style with size=0, unlike sized initialization required by bindings.rs:1042 and used by render.rs:631. The native implementation's exact rejection/partial-write behavior is not inspected; keep as sized-ABI mismatch needing a focused test, not a demonstrated crash.
- Build provenance: ghostty-vt-sys/build.rs:85–88 emits the locked commit/version even when an unchecked local source override is selected at 98–112. The default fetched archive is SHA256 checked at 230–236; the finding is restricted to override provenance/ABI reporting, not the pinned download flow. build_info.rs:32–41 reports those constants as the built library identity.
- Pinned bindings document various native security/lifetime constraints; reading generated declarations does not prove the external native implementation follows them.
- No race, memory footprint, runtime latency, or user-input exploit has been measured in this read-only audit.

Additional confirmed WGPU source findings

V WGPU 01 translucent underlines apply alpha twice

P3 visual correctness. vendor/gpui-fast-wgpu/src/shaders.wgsl:501–504 computes output alpha as color.a * alpha_factor. fs_underline passes input.color.a for solid underlines at 1331, and coverage * input.color.a for wavy underlines at 1347, yielding alpha squared. A requested 0.5 alpha therefore produces 0.25 opacity in full-coverage pixels. The active renderer creates this pipeline in wgpu_renderer.rs:847–858 and uses it in fast/frame.rs:881–888. Pass coverage only (1.0 for solid), as the monochrome sprite path does at shaders.wgsl:1394. Actual theme-specific visibility was not run or measured.

V WGPU 02 vendored unit test inputs are absent from the pinned tree

P2 test reproducibility, not a production build failure. cosmic_text_system.rs:1295–1302 and 1327–1331 include_bytes! three fonts under ../../../assets/fonts/{lilex,ibm-plex-sans}; these resolve to root assets/fonts and all are absent from the full pinned tree. fast/layers/tests.rs:121–123 and rounded_clip_texture_tests.rs:7 reference ../../../../gpui/src/fast/rounded_clip_fixtures.rs, resolving to vendor/gpui/src/fast/rounded_clip_fixtures.rs, also absent. These are compile-time includes, so the crate's affected unit-test targets cannot be built from this repository snapshot alone. In addition, tests.rs:30–33,138–143 refer to gpui test types and rounded_clip_tests outside the feature guard that defines the fixture module. Supply pinned fixtures/fonts and align feature gates. No cargo test was run; this is a static missing-input finding. The first-party renderer_corners integration test is a separate source-based test and is not asserted broken by this.

WGPU candidates and boundaries

- Rounded overflow clip geometry in fast/rounded_clip.rs:317–338 remains a circle/ellipse, whereas the Proteus quad/image mask patch in shaders.wgsl:381–424 uses a fourth-power superellipse with enlarged extent. This is a source-level geometry mismatch; a concrete affected parent/child screenshot and intended contract still need verification before making it an application defect.
- Oversized atlas allocation may create an empty texture before etagere allocation rejects the requested size (wgpu_atlas.rs:163–181,199–253). Upstream gpui AtlasState size validation and the reachable image path must be checked before claiming a retention defect.
- handle_device_lost updates atlas device/queue/format but not its original max_texture_size (wgpu_atlas.rs:103–110); recover can create a new context (wgpu_renderer.rs:2239–2243,2264). A replacement device with a smaller texture limit is a conditional boundary, not a reproduced failure.
- WgpuRenderer::new documents a caller-managed native-window lifetime at wgpu_renderer.rs:292–294 despite being a safe constructor; replace_surface similarly stores a static raw-handle surface from a temporary borrow. Current platform ownership is external to this vendored renderer snapshot, so no current Desktop lifetime failure is claimed.
- GPU shader validation, pixel tests, driver recovery, allocations, and latency were not executed or measured.

Desktop vendor dock и base state

Proteus-desktop a608d93

Findings

V BASE C01 P2 vendor API resizing docks does not emit the documented persistence event

vendor/gpui-base/src/dock/dock_area.rs:40–41 documents LayoutChanged as the subscription point for persisting every edit. Programmatic set_dock_size emits that event at 348–363, but the interactive resize path DockContext::resize_to/end_resize at 1854–1863 calls the closures at 1519–1529, which reach resize_dock (1304–1339) and end_dock_resize (1344–1366). Both mutate dock layout and notify drawing without emitting LayoutChanged. For ordinary side resizing the end handler returns when live_size is None (1353–1355). A consumer that follows the documented event contract misses the new size until an unrelated later save. The setter test at 2088–2123 does not exercise drag resize. Static search found no first-party Desktop DockArea usage in src/ or crates/, so this is a vendor API defect, not an established Desktop persistence failure.

V BASE C02 P3 vendor API exhausted RopeLines reports an invalid remaining length

vendor/gpui-base/src/input/base/rope_ext.rs:64–74: nth(n) saturating-adds n to row without clamping to end_row, then next() returns None. size_hint subsequently computes end_row - row rather than saturating subtraction. For Rope::from("a").iter_lines().nth(2), the exhausted iterator's size_hint()/ExactSizeIterator::len() underflows: a debug build panics, a release build reports an enormous length. The iterator explicitly implements ExactSizeIterator and FusedIterator (77–78), whose exhausted length must be zero. Fix by clamping row or using saturating_sub; add a test calling len() after nth past EOF. No first-party trigger was established.

V BASE C03 P3 vendor API auto grow configuration can create an invalid clamp range

input/base/state.rs:12334–12335 accepts auto_grow(min_rows,max_rows) verbatim, while runtime set_auto_grow at 12287–12289 explicitly normalizes max_rows.max(min_rows). In auto-grow mode,

rows()/set_rows() at 12305–12312 and 12321–12327 can also lower max_rows below the retained min_rows. input/base/mode.rs:210–224 subsequently calls new_rows.clamp(min_rows,max_rows), which panics when min exceeds max. Examples are auto_grow(5,2), or auto_grow(5,10).rows(2) followed by layout/text growth. No runtime was executed and no invalid first-party argument was found. This is a public vendor configuration invariant gap.

Corrections and unconfirmed boundaries

- The original base report's move_panel invalid-target removal is deliberately asserted by the current tests in dock_area.rs:3484–3542, including on_removed. Do not call it a demonstrated Desktop regression. The public API is non-atomic on invalid target; any issue must be framed around that deliberate contract and a concrete caller.
- Candidate forward folding clamp is not a finding: state.rs:9173–9238 and 12423–12437 explicitly keep first/last fold lines visible, so targeting end_line is intended.
- Candidate readonly Escape bypass is not established: escape calls programmatic clean, but render only registers editing actions while editable at state.rs:5104–5125.
- Input LSP Position conversion uses Unicode scalar counts, not UTF-16, in rope_ext.rs:330–342. The final controls section confirms the LSP contract defect C2 with its stated reachability boundary.
- IME character_index_for_point at state.rs:5004–5017 passes the same point to each line without a running y offset. The final controls section confirms the geometry defect C1 with a current Desktop route.
- Native content-type map lifetime, long-line shaping, ghost-text geometry and masked caret concerns were not promoted without a complete trigger/caller chain.

Desktop vendor editor controls accessibility и copy

Proteus-desktop a608d93

Confirmed findings

C1 Native point to character lookup uses editor coordinates as line local coordinates P2 production text controls

The EntityInputHandler callback localizes the point only to last_bounds, then passes the same point to every visible logical line. LineLayout::index_for_position requires coordinates relative to that particular line, and wrapped_line_at starts y at zero each time. For two unwrapped visible lines and a point on the second line, y is at least one line height, so neither line accepts it and the callback returns None. With mixed wrapping, it can instead resolve the point against the wrong logical line. It also omits visible_top and line-number gutter translation. The adjacent bounds_for_range implementation explicitly adds the gutter and accumulates each line's y offset, demonstrating the missing inverse transform.

This affects the native text-input geometry callback, not the ordinary mouse hit-test path. Desktop's chat composer instantiates the shared TextareaState; readonly code fields instantiate EditorState with a gutter. The defect is reachable when the host input method requests character lookup at such a point; no OS/IME runtime reproduction was performed.

- Faulty callback
- Line-local y requirement

- Exact index entry point
- Opposite coordinate transform
- Desktop composer
- Desktop code editor/gutter

Fix direction: subtract gutter and visible_top, then subtract accumulated logical-line heights before calling the line-local lookup. Test an unwrapped second row, a wrapped row, a partially scrolled first row, and a nonzero gutter.

C2 LSP positions are converted as Unicode scalars not UTF 16 units P2 vendor LSP API

The exported Position is lsp_types::Position. position_to_offset treats character as a Rope character index. offset_to_position counts chars; its UTF-16-to-byte round trip does not change that scalar count. LSP's default UTF-16 character positions therefore fail after non-BMP characters. For text a😀bc, Position(0,3) should point to byte 5 before b, but this implementation produces byte 6 before c; byte offset 5 is returned as Position(0,2), not (0,3).

This is not just an internal Point convention: diagnostics, definitions, document colors, semantic tokens, text edits and completion edits consume real lsp_types positions through this conversion. No encoding-negotiation layer or concrete Desktop LSP-provider installation was established in src/crates, so the finding is not presented as a currently exercised Desktop server integration.

- Conversions
- Position type
- Diagnostics consumer
- Definition target consumer

Fix direction: use line-relative UTF-16 conversions by default, or explicitly thread negotiated encoding through every LSP boundary. Include astral characters, out-of-range columns and bidirectional round trips in tests.

C3 A list of LSP TextEdits is applied against successively mutated coordinates P2 vendor API

apply_lsp_edits iterates the provider's order, converts each range using current self.text, and immediately mutates that text. LSP edits in one list refer to the same original document. A valid ascending list becomes wrong after the first length-changing edit. ASCII example: for abcdef, insert XX at column 1, then replace original columns 4..5 with Y. Expected aXXbcdYf; this loop yields aXXbYdef because the second edit now targets c. This is independent of C2. No direct Desktop caller was found.

- Entire application loop

Fix direction: derive all offsets from the original text and apply as one validated batch, or use correct reverse ordering with explicit handling of same-position insertions. Test multiple non-overlapping edits with insertions/deletions before later ranges.

C4 Accepting an inline completion ignores its replacement range P2 vendor API

accept_inline_completion takes InlineCompletionItem but always replaces cursor..cursor. The returned item's range is never considered. A legal completion for pri, with range (0,0)..(0,3) and insert_text print, is appended as priprint rather than replacing the prefix with print. No Desktop inline-completion provider was established.

- Acceptance

Fix direction: honor the item's supplied range using the applicable LSP encoding and use cursor insertion only when no range is supplied. Add acceptance tests, not just suggestion-filter tests.

C6 Replacing fold candidates can leave removed folds hidden P2 vendor API Desktop sequence not established

FoldMap::set_candidates removes folded ranges no longer present but does not mark the projection dirty. DisplayMap calls rebuild, which returns immediately when needs_rebuild=false and the wrap-row count is unchanged. Concrete sequence: construct folds (0,3) and (5,8), fold both and build the projection, then replace candidates with only (0,3). The second fold is no longer in folded_ranges/is_folded_at, but its interior rows 6–7 remain hidden because a nonempty first fold keeps the rebuild path active and the cache returns early. Separately, a replacement candidate with the same start and a changed end retains the old folded end.

State exposes this through apply_highlighter_fold_candidates and its full tree-sitter extraction, but a concrete Desktop sequence that changes only these candidates while retaining active folds was not established. This is not the rejected next-boundary hypothesis.

- Candidate replacement
- Stale-cache early return
- Public setter
- State entry point

Fix direction: reconcile retained ranges with replacement candidates and invalidate the projection whenever folded ranges change. Test replacing two folded candidates with one while line count stays constant.

C7 A panel exactly at the 100px minimum never becomes unchanged P2 vendor resizable API

panel_size_changes returns true whenever the stored size equals PANEL_MIN_SIZE (100), even if bounds/range/fixed are unchanged. update_panel_size then writes the actual measured size back, which is still 100 for a panel genuinely at its minimum, and notifies. The on_prepaint callback repeats this on every frame, invalidating state readers despite an unchanged layout. The regression test for retained readers does not target the sentinel-equals-real-size case. No Desktop use of this resizable API was established.

- Sentinel and notification
- Every-frame callback
- Existing regression test

Fix direction: track uninitialized measurement explicitly rather than using a valid panel size as a sentinel. Test an unchanged 100px panel across redraws.

C8 Both axis scrollbar reserves a vertical bar corner when only horizontal overflow exists P3 vendor API

ScrollbarAxis::Both enumerates Horizontal before Vertical. Prepaint initializes has_both=true, computes the horizontal margin_end as track_width and builds that track before discovering that vertical content does not overflow. Setting has_both=false during the later vertical iteration cannot fix the horizontal geometry already produced. For a 100×100 viewport with 500×100 content and default 16px track, the horizontal track unnecessarily loses the rightmost 16px. The nearby comment claims the opposite behavior. No specific Desktop Both-axis visual regression was established.

- Axis order
- Margin before overflow check

Fix direction: determine actual overflowing axes first and reserve the corner only when both tracks will exist. Test Both with horizontal-only overflow as well as both axes.

C9 Root s keyboard Copy trims the selected text P2 Desktop Root path

Root::on_action_copy trims TextSelection::selected_text before writing it. Selecting text copies text; selecting whitespace alone does not replace the clipboard. Exact selection bytes are lost, including leading code indentation. Desktop's context-menu handler copies the same TextSelection result without trimming, so those handlers have inconsistent semantics. Desktop opens windows through gpui_kit::open_window and explicitly downcasts native windows to gpui_kit::base::Root; the accompanying comment states that every native window wraps its application view in Base Root. That Root installs this Copy listener. The gpui-kit wrapper itself is external and was not independently read, but Desktop's own source establishes the Root integration. The component touch-selection Copy handler contains the same trim operation.

- Trimming handler
- Desktop exact-copy menu
- Desktop window creation
- Desktop Base Root downcast
- Root listener installation
- Touch Copy counterpart

Fix direction: use trimming only to decide whether a semantic empty selection should propagate, if necessary; preserve the original selected text for clipboard output. Prefer allowing deliberate whitespace selections to copy too.

Rejected hypotheses and bounded observations

- C5 (downgraded, not counted as a confirmed defect): standard completion insert_text without text_edit discards the menu's fallback prefix range and appends literally (overlay.rs188–190, component completion_menu.rs278–286). This differs from label-only prefix replacement, but LSP explicitly leaves insertText interpretation to the client. Treat it as a compatibility/test concern rather than an unambiguous protocol violation.
- The proposed forward fold-boundary bug is rejected: fold_map explicitly preserves both first and last rows and hides only their interior. Moving forward to the start of end_line is consistent with these semantics. Source.
- DisplayMap::set_text calls an initialization-only wrapper path. Its documentation says “Initialize with text”; repeated setter semantics were not treated as a confirmed contract defect.
- Scrollbar touch startup checks raw thumb bounds, whereas mouse startup checks the hitbox to respect overlays. Occluded touch behavior deserves a targeted event-routing test, but was not promoted without checking the host touch dispatch contract.
- DiagnosticSet stores pushed entries without sorting and its summary overwrites rather than aggregates endpoints. Unordered/overlapping diagnostic lookup deserves an explicit regression test; the external sum-tree search contract was not fully inspected here, so this remains a bounded concern rather than a finding.
- No assertion is made that every vendor-only API is configured by Desktop or that inspected tests pass. Tests were read, never run. Full reading does not prove the absence of further defects.

Findings

W1 Assistive technology slider changes do not apply Desktop preferences P2 production path

Increment and Decrement call `state.set_value`, which changes the state and notifies rendering but emits neither Change nor Release. Pointer interaction emits Change. Desktop settings subscribe exclusively to those events to apply/persist chat text size, chat width and surface radius. Consequently an accessibility adjustment changes the slider's internal value without applying that preference. This is not merely an unused vendor API: Desktop renders the component Slider, and the component renders BaseSlider with the same state. Cargo patches gpui-component to this vendored version; the lockfile records gpui-kit depending on gpui-component.

- Accessibility callbacks
- Setter only notifies
- Desktop event subscriber and rendered slider
- Component delegates to BaseSlider

Fix direction: user-initiated accessibility edits should publish the same semantic change event as pointer edits. Keep programmatic setters silent if callers depend on that contract. Add an accessibility action test that asserts both state and event/application preference.

W2 Accessibility adjustment destroys a range slider s start value P2 vendor API only

The same callbacks take `value().end()`, calculate a scalar, and pass that f32 to set_value. From<f32> constructs Single, replacing a Range rather than updating its end. For Range(20,80), step=1, Increment produces Single(81), removing the start thumb/value. No Desktop range-slider consumer was established; its settings use Single values.

- Callbacks
- Scalar conversion

Fix direction: preserve variant and the untouched endpoint; ideally expose independently accessible range thumbs.

W3 Pointer step rounding can emit values outside configured min max P2 vendor API only

Pointer percentage is clamped, but the resulting value is rounded against zero using `(value / step).round() * step`, with no subsequent clamp. A min=1, max=10, step=3 slider emits 0 at its left endpoint; min=0, max=10, step=4 emits 12 at its right endpoint. Thumb percentage also remains the unrounded pointer percentage rather than the snapped value. This counterexample does not establish that Desktop's particular configured bounds trigger the out-of-range case.

- Clamping then unbounded rounding and event

Fix direction: snap relative to the intended grid origin, clamp the snapped value, and recompute the thumb position. Test non-step-aligned bounds.

W4 A default open popover skips its opening lifecycle P2 vendor API only

PopoverState::new(true) stores open=true while previous focus, dismiss subscription and deferred-context registration remain None. First render creates that state and does not transition it unless a controlled open value differs. Even sync_open(true) immediately returns because it is already marked open. Thus a default-open popup is rendered without the opening focus transfer, DismissEvent subscription or deferred-popover

registration that normal opening performs. Its visibility-only test does not cover these effects. No Desktop default_open(true) call was found in src/crates.

- Initial state
- Opening lifecycle and early return
- First render

W5 Initially absent Presence is reported as exiting renderable P2 vendor API only

On the first sample of Presence(false) with a positive-duration transition and reduce motion off, state initializes from=0,target=0 but started_at=now. It samples a running transition and returns Exiting until the duration completes, despite never having been present. PresenceSample::should_render returns true for Exiting. Consumers using that API to unmount absent content retain it during a spurious exit and schedule unnecessary animation frames. No direct Desktop consumer was found.

- Initialization
- Returned phase and frames
- Render predicate

W6 Reduced motion keyframes finish at the wrong endpoint for reverse playback P2 vendor API only

animate_keyframes returns keyframes.sample(1.0) unconditionally when reduce motion is enabled. Normal completion accounts for playback direction and final iteration. A 0→10 animation played Reverse for one iteration normally ends at 0 but reduced motion returns 10. Alternate with an even finite iteration count has the same mismatch. No direct Desktop animate_keyframes call was found.

- Reduced-motion branch
- Normal endpoint calculation

W7 Cached area path ignores baseline endpoint inputs P2 vendor API only

Area::path reads first/last x even when their y1 is None, and uses those x values to close the baseline polygon. paint_cached hashes only points for which both x and y1 are Some. Change the first datum's x while leaving its y1=None and the remaining valid points unchanged: the cache key stays identical even though a fresh polygon would close to a different first x. Reusing the same cache therefore retains stale geometry. No concrete Desktop caller supplying missing endpoint y values was established.

- Endpoint inputs
- Baseline closure and incomplete cache key

W8 Plot label font_weight setter has no effect P3 vendor API only

Text::font_weight stores the requested weight, but Label::paint explicitly discards it, and shape_label obtains the inherited window font. An explicit bold/normal weight on a label is never applied. No concrete Desktop visual regression was established.

- Setter
- Discarded value
- Inherited font

Additional review notes not promoted to production defects

- StepAfter intentionally omits the final vertical edge (line.rs201–209; area.rs156–168), so a two-point line does not reach its second y value. Its documented d3.curveStepAfter relationship merits a focused expected-geometry test; no Desktop StepAfter use was found.
- Grid dash_segments can fail to advance for a NaN-only dash list because its initial guard does not reject NaN and each normalized segment becomes zero. This is a malformed-input robustness issue, with no established Desktop input path.
- Positioner fit selection does not include the requested offset, which can choose a side fitting the popup but not popup+gap, followed by clamping. Treat as an offset/placement behavior to test, not a proven Desktop regression.
- Tabs explicitly documents its absent compound keyboard navigation; this is an acknowledged limitation rather than a newly hidden implementation failure.
- No additional concrete defects were established in the other fully read files. Full reading does not imply absence of defects, and the test code was inspected but never run.

Desktop vendor rich text и virtual widgets

Proteus-desktop a608d93

Findings reachable through Desktop s HTML rich text

D1 HTML cleanup corrupts attribute values before rendering P2

html5minify/mod.rs:537–570

The minifier serializes already-decoded html5ever attribute values. Its unquoted-value predicate excludes quotes, = and ASCII whitespace, but allows > and &. This branch writes the bytes directly rather than escaping them. HTML cleanup then parses the generated markup a second time: html.rs:95–131.

Static counterexamples:

- b">go first yields the value https://example.com/a>b; serialization becomes b>go. The next parse truncates the destination to /a and turns b> into visible text.
- go intends the literal path ©. The raw unquoted reserialization causes a second character-reference decode to ©, changing the destination.

The same defect applies to src, title and custom data-content values. This is a rendering/data-integrity issue, not a claim that the native widget executes JavaScript.

Desktop reachability is direct: rich_text.rs:17–31 creates TextViewState::html and updates it with the supplied HTML; 44–48 mounts the widget. Link destinations are consumed at html.rs:487–500.

Repair: use a correct attribute serializer in every branch, or bypass the serialize/reparse round trip. Regress with both examples and assert rendered text and exact destination, not merely parse success.

D2 Ordered HTML lists silently restart at one P2

html.rs:636–644

The <ol> conversion unconditionally sets start: None; it never reads the HTML start attribute. Rendering passes this value through node.rs:4369–4395, and utils.rs:11–28 defaults the ordinal to 1.

Thus a rich-text payload <ol start="3">
three
four displays 1 and 2, changing the content's numbering. The same directly reachable Desktop HTML path as D1 applies. The internal node comment documents this omission, but it still changes authored numbering. This report does not claim every list producer currently supplies a non-default start. Parse supported numeric starts and add an HTML start=3 rendering case; the existing layout test covers non-default starts only for Markdown, not HTML.

Confirmed vendor API defects not promoted to Desktop regressions

A targeted search in src/ finds Desktop's native rich-text construction using HTML, not direct TextViewState::markdown, SelectionFormat::Source, or selected_source_range. Chat Markdown goes through a contributing process: replies.rs:44–71. HTML forces plain copy: state.rs:432–445. Consequently the following Markdown-only defects are real in the shipped library but are not evidence of broken Desktop chat copying or reference links.

V1 Overlapping inline marks duplicate text during source copy medium

markdown.rs:309–331 and 342–423 preserve inner mark ranges and append an outer full-range mark. node.rs:1141–1168 treats those overlapping ranges as disjoint pieces and emits their intersections separately.

For **left *middle* right**, marks are italic 5..11 followed by bold 0..17. A drag selecting its whole rendered paragraph, in Source mode (not Select All), emits left *middle***left middle right**: the selected content is duplicated. A partial selection crossing these ranges also duplicates characters. SourceAcrossStyles can enter the same path. document.rs:134–180 reconstructs endpoint blocks even when a drag happens to cover their whole text; only view Select All returns the original source verbatim. Normalize mark boundaries into disjoint runs carrying the combined marks before serializing. The existing nested-mark test uses one combined full-range mark and misses partial overlap.

V2 Inline code delimited by three backticks loses source mapping medium

markdown.rs:655–699

The same helper maps inline and fenced code. Any leading sequence of at least three backticks is treated as a block fence. In valid inline Markdown before code after, the inline node has no newline, so body_start becomes raw.len() and the mapped source slice is empty. Selection over code cannot yield its exact source range, and source-to-rendered conversion misses it. Pass node kind or use distinct inline/fenced mapping. Regress triple-backtick inline spans, including multiline inline spans. This does not prove ordinary Plain clipboard text is lost.

V3 TextView drops parsed reference link definitions medium

state.rs:1341–1394

parse_content creates a local NodeContext, the parser adds definitions to its link_refs, but the function returns content without copying that context into content.node_cx. Rendering later reads parsed_content.node_cx.link_refs (state.rs:1129–1139). Reference nodes intentionally start with empty URLs (markdown.rs:923–939); render-time resolution needs the lost map (node.rs:2841–2846). A complete [docs][r] \n\n[r]: https://example.com document therefore reaches rendering without that destination.

There is an additional streaming boundary: reparsing only the last block cannot resolve an earlier literal [r] when a definition arrives after a later paragraph, nor retain definitions outside the reparse slice for a new reference. Saving the local map alone does not solve that global dependency. Regress full reference links plus append-

after-parser-settled forward/backward definitions. Tests that coalesce the entire source into one parse do not cover the incremental case.

V4 Table width cache ignores explicit inline code typography box style medium

node.rs:3336–3379

The cache key includes row text styles, rem size, theme mono font and inline-code HighlightStyle, but not TextViewStyle::inline_code_style, even though cell code measurement applies that explicit font and horizontal box inset. Reuse the same parsed table, then change only with_inline_code_style font size or padding: normal rendering changes but intrinsic column widths are reused. The view remeasures its list, not this independent table cache. Add the effective code metric/box fields to the key. No production Desktop interaction that changes only these fields was established; this stays API-only.

V5 V7 Text selection widget tail findings medium vendor only

The separately completed tail report supplies detailed proof, suggested tests and scope:

- virtual_list.rs:530–551: horizontal Infer computes height from known_dimensions.width rather than height.
- virtual_list.rs:471–511: retained geometry rebuilds only when item sizes change, ignoring changed gap or axis. Three 20px rows changing gap 0→10 retain origins 0/20/40 and total60 instead of 0/30/60 and total80.
- tree.rs:378–438: keyboard navigation/expansion bypasses the pointer's disabled-entry guard. Down onto a disabled folder then Right expands it and emits Expanded.

No direct consumers of h/v_virtual_list, TreeState or TreeItem were found in Desktop src//crates/. That search is not proof of absence through an indirect facade, so no Desktop screen is claimed affected.

Candidates deliberately not promoted

- The raw MouseUp link path synthesizes its down event rather than retaining the original down target. An unintended release-over-link scenario needs native input validation. The initial concern that ordinary right-click opens a URL was **withdrawn**: text_view.rs:116–133 explicitly filters the default opener to left/middle. Custom handlers intentionally receive right-clicks.
- Source reconstruction also uses fixed backtick fences and selected-table reconstruction does not escape literal cell pipes. These are source-copy fidelity limitations; no production consumer of this mode was found. They are not extra Desktop findings.
- Limited 128-character word scans and script classification are documented behavior; no stronger requirement was established.
- Retained native layout cleanup, weak selection registration, atomic/custom object ownership, parser coalescing, range remapping, touch handles and selection-before-paint tests were reviewed. No measured performance, RSS or execution claim is made.

Desktop vendor component основной каталог

Proteus-desktop a608d93

VC 01 P2 unbounded code block highlighter cache retains every unknown language s text

- Source: vendor/gpui-component/src/text/mod.rs:92–105,112–124; vendor/gpui-component/src/highlighter/highlighter.rs:379–389,395–413,619–623.

- A process-thread-local HashMap is keyed by the raw CodeBlock language string. Every previously unseen string gets a SyntaxHighlighter, including an unregistered language which falls back to the plain-text parser. update_input then stores the entire code Rope even when no grammar exists. Entries are never evicted or cleared when the text view/document is destroyed. Unique unknown fence labels therefore retain distinct code buffers for the thread lifetime, with no cardinality/byte budget. The outer four-theme cache does not constrain this inner cache.
- Reasoned case: render code blocks with labels unknown-1, unknown-2, … and distinct large contents, then close their views. The independent map entries keep all corresponding Ropes. This is retention proven by ownership, not a measured RSS result.
- Scope: vendor Markdown/CodeBlock API. Desktop enables tree-sitter (root Cargo.toml:29–45), but this pass did NOT establish a current Desktop UI trigger: its direct TextView caller uses HTML (src/ui/extensions/declarative/rich_text.rs:18,45), while code previews use Editor. Do not promote this to a proven Desktop memory leak.
- Correction direction: skip cache creation for unresolved languages, canonicalize registered names, and use a shared entry/byte budget; dispose retained code/tree state when it is no longer useful.

VC 02 P2 code block cache fails to refresh same name grammar factory changes

- Source: vendor/gpui-component/src/text/mod.rs:102–110,163–218; vendor/gpui-component/src/highlighter/registry.rs:530–555; vendor/gpui-component/src/highlighter/highlighter.rs:419–449,518–535,615–622.
- register and register_parser_factory bump the registry generation and can replace an existing entry. The outer shared highlighter closure responds to the generation, but all closures access the same thread-local inner map. Its refresh condition compares only highlighter.language() with config.name. Updating a grammar/query under the same name, or adding a parser factory to a previously registered plain configuration, leaves the old parser/query installed. Identical text also returns early from update_edits. A new outer Arc does not correct the stale inner parser.
- Reasoned case: register GrammarConfig::plain("custom"), highlight once, then register a grammar/query or factory for that same name and highlight again. The cached inert parser still has language "custom", so the comparison is false and it remains inert.
- Scope: public vendor dynamic-language API. No current Desktop grammar/factory registrations were found in the targeted src search; present-app reachability is not established.
- Correction direction: include registry generation/config identity in the inner cache invalidation, not only in the outer closure cache; add same-name replacement and plain-to-factory regression cases.

VC 03 P2 performance risk the input s synchronous deadline does not cover injection parsing

- Source: vendor/gpui-component/src/highlighter/input_adapter.rs:30–32,145–163; vendor/gpui-component/src/highlighter/highlighter.rs:20–26,652–685,821–911,942–984.
- The editor requests a two-millisecond synchronous parse. The timed callback covers only the host parse. Once that succeeds, update_edits calls parse_injection_layers synchronously before returning. Each non-combined injection is allowed an independent 20 ms, with up to 512 parse attempts and no shared elapsed-time check. Query traversal and combined injections are also outside the host deadline. Thus the advertised foreground bound is not an aggregate bound and the background fallback is never selected just because injection work exceeds it.

- Scope: current Desktop source route exists for native code editors: src/ui/extensions/declarative/fields.rs:479,510 configures a language; code_block.rs:49 mounts component Editor; input/editor.rs:150 forwards to Input; input/input.rs:537 installs this factory. Root Cargo.toml enables HTML/JS/CSS grammars with injection support. Actual impact depends on document and grammar; no latency, freeze duration, CPU, or RSS measurement was performed.
- Correction direction: pass one shared deadline through host parsing and injections, and defer remaining work to the background parse. Preserve already valid host highlighting.

VC 04 P3 NumberInput placeholder is silently ignored

- Source: vendor/gpui-component/src/input/number_input.rs:23,40,51–55,123–225.
- The public placeholder builder only stores the argument. The complete render path never reads that field and constructs Input from the state without setting its placeholder. Passing a placeholder to this element therefore cannot alter the display. A separately configured InputState placeholder is unrelated to this dead builder.
- Scope: vendor API; current Desktop usage of this builder was not established. Fix by forwarding it to the appropriate presentation/state API, or clearly deprecating the inert builder without claiming it works.

Investigation candidate not yet a confirmed finding

Timed-out parser state may resume against a later edited buffer. highlighter/highlighter.rs:652–678 retains a Parser after its progress callback cancels, without reset; apply_background_tree:991–1004 replaces only the tree. input/syntax_context.rs:104–106 explicitly calls parser.reset() for the same cancelled-parse concern. The tree-sitter 0.26.13 parser-resumption contract still needs confirmation from dependency source/documentation before promoting this candidate. No test was run.

Not counted as newly discovered defects

- Link::disabled is behaviorally inert and the on_click documentation differs from its open-plus-callback behavior (src/link.rs), but dedicated compatibility tests explicitly preserve both. This pass does not recommend breaking those documented legacy expectations as an unrequested fix.
- AnyInputState::value's masking sentence conflicts with InputState::value's source-test expectation of the underlying value (input/state.rs:322–330 and input/input.rs:1129). This is a documentation issue, not proof of a new security exposure.

VC 05 P2 disabled popup actions can still be invoked with Enter Desktop reachable

- Source: vendor/gpui-component/src/menu/popup_menu.rs:878–906,943–980,1024–1029,1264–1275; vendor/gpui-component/src/menu/menu_item.rs:109–112.
- activate invokes Item/ElementItem handlers and actions without checking disabled. First Down and wraparound select index zero rather than the first clickable item; entering a submenu also selects zero. Additionally, MenuItemElement installs its hover listener even when disabled, and the popup listener selects that disabled item. The absence of a mouse-click handler therefore does not prevent keyboard confirmation.
- Reasoned sequences: (1) open a popup whose first item is disabled, press Down then Enter; (2) hover any greyed-out item while the popup has focus, then press Enter. Both paths reach the stored handler without a disabled guard. No sequence was executed in this read-only audit.

- Current Desktop sources: src/ui/extensions/declarative/menu.rs:99–109 attaches the process's disabled flag but the callback invokes send without checking it again; send at 40–43 invokes view.act directly when no prompt/confirmation is required. src/ui/tabs/menu.rs:40–45 similarly stores disabled and an unconditional handler; its Close others/right and split entries at 60–85 use this path. Downstream actions may independently reject invalid state, but the promised disabled menu behavior is already bypassed. No claim of unauthorized server-side state mutation is made.
- Fix direction: check item.is_clickable at the common activation boundary; make hover, first-item selection, wrapping and submenu entry skip disabled/header/separator entries. Test both Item and ElementItem, an all-disabled menu, hover+Enter, first Down+Enter, and submenu entry.

VC 06 P2 AppMenuBar and each AppMenu form a strong entity cycle

- Source: vendor/gpui-component/src/menu/app_menu_bar.rs:27–28,49–61,146–172.
- AppMenuBar owns every child as Entity<AppMenu>, while each AppMenu stores Entity<AppMenuBar>. reload supplies menu_bar.clone() to every child, and no release/Drop path breaks these backreferences. A nonempty menu bar therefore owns a cycle and survives dropping its external/window owner, retaining its menu data and any cached popup state. This follows ownership in the fully read file; no leak/RSS test was run.
- Scope: vendor application-menu API. A direct current Desktop AppMenuBar caller was not established. Use a WeakEntity backreference and upgrade only where needed; add a window-close release test analogous to the existing dropdown/context-menu release tests.

Additional alternate feature observations

- P3, syntect/no-tree-sitter theme compatibility: highlighter/wasm_stub.rs:423–429 lacks the serde renames for editor.background, editor.foreground, editor.active_line.background and other fields which are present in registry.rs:438–449. Themes using the normal dotted keys silently leave those fields None in the alternate backend. StatusColors is also an empty structure whose every color method returns Hsla::default (wasm_stub.rs:355–419), unlike the themed native values. Current Desktop explicitly uses tree-sitter, so this is an alternate-feature/vendor issue.
- P3, both theme backends: FontStyle::Underline is converted to gpui::FontStyle::Normal and no underline decoration is supplied (registry.rs:185–191,232–239; wasm_stub.rs:158–179). A theme explicitly requesting this supported enum value loses its underline.
- Investigation only: languages.rs:558–564 configures ERB using INJECTIONS_EJS_QUERY, exactly as EJS below it. Verify against the upstream embedded-template crate before changing the query constant. Current Desktop does not enable tree-sitter-erb.
- Investigation only: JavaScript and TypeScript queries place their generic identifier capture before function/type-specific captures, while highlighter.rs:1112–1119 preserves the earlier name for equal adjacent ranges. Kotlin's query comment explicitly documents that specific patterns must come first. Confirm precise QueryCursor ordering with a focused static dependency review or an authorized test before promoting this to a demonstrated coloring regression.

VC 07 P2 dialog callbacks can close a newly opened dialog instead of their own

- Source: vendor/gpui-component/src/dialog/dialog.rs:751–766; vendor/gpui-component/src/root.rs:415–430; cross-check vendor/gpui-base/src/dialog.rs:571–598,619–632.

- Base calls on_ok/on_cancel before invoking request_close. Component's request_close ignores the dialog's stable id and calls the global close-latest operation. A callback that opens dialog B while confirming dialog A and returns true therefore causes close_dialog_internal to close B, leaving A active. Likewise a callback that closes A itself but returns true can close the dialog below it. The on_close callback executes after closure and is not this particular trigger.
- Scope: vendor API reentrancy defect. Desktop uses these modal APIs, but the inspected current callbacks send asynchronous requests or replace the window; no current callback that synchronously opens another modal was established. Do not label the specific failure reproduced in current Desktop.
- Fix direction: capture the dialog id in request_close and close that identity, treating an already-closed identity as a no-op. Cover callbacks that open another dialog and callbacks that close their own dialog. No test was run.

VC 08 P2 delayed confirm focus restoration overwrites a user s newer focus choice

- Source: vendor/gpui-component/src/root.rs:455–487, especially 472–484; dialog/dialog.rs:756–765; vendor/gpui-base/src/dialog.rs:586–598.
- Confirm uses defer_close_dialog, which closes the modal logically now, then waits a fixed 250 ms and focuses the saved pre-dialog handle whenever the number of open dialogs is unchanged. It does not verify whether focus changed during the delay. A user who confirms, immediately clicks another input, and types has that newer focus replaced by the old input when the timer fires. Dialog render explicitly removes the closing overlay’s input occlusion and backdrop (dialog/dialog.rs:723–725,737–748), so background interaction can occur during this delay. The delay is unconditional even when reduced motion makes exit immediate. Independent delayed closings also have no generation/cancellation guard.
- Scope: source-reachable in current Desktop confirmation dialogs, e.g. src/ui/panels/session_menu.rs:137–151 and src/ui/extensions/declarative/prompt.rs:57–68. The particular rapid user interaction was reasoned from source, not executed. For a stacked-modal variant, replacing a remaining dialog while keeping the count unchanged also invalidates the count-only guard.
- Fix direction: restore immediately after logical close when allowed, or cancel restoration on any later focus/stack-generation change. Do not use dialog count as identity. Test a focus change before the timer and reduced-motion confirmation.

VC 09 P3 Sheet resizable is stored but never applied

- Source: vendor/gpui-component/src/sheet.rs:47,65,94–98,134–266.
- The public builder promises to toggle resizing (default true), but the complete render path never reads resizable or creates resize controls. Neither setting affects the resulting BaseSheet.
- Scope: vendor API only; a targeted search found no current Desktop open_sheet caller. Forward the setting to a supported resizing implementation or document/remove the inert API.

Lower priority behavior and compatibility notes from completed UI batches

- Notification custom icons are ignored when type_ is Some: notification.rs:255–260 promises the type icon only as a fallback, but render at 415–418 always picks the type icon. This affects info/success/warning/error constructors followed by .icon(...). P3 vendor API mismatch; a current Desktop use of the custom override was not established.

- Field::label_indent(false) suppresses an explicitly supplied label (form/field.rs:124–129,250,299–327), although its contract says indentation only applies when no label is set. P3 vendor API mismatch; current Desktop use was not established.
- ButtonGroup's keyboard group callback omission and builder-order-dependent disabled propagation are expressly preserved by compatibility tests at button/button_group.rs:373–424. ToggleGroup likewise intentionally overrides child callbacks even without a group callback and does not forward keyboard activation to the group (button/toggle.rs:556–583). These are limitations worth retaining in migration notes, rather than presenting as newly introduced regressions or silently changing compatibility. A targeted current Desktop src search found no ButtonGroup or ToggleGroup caller. Tests were read, not run.
- Full button directory, checkbox, accordion, alert, collapsible, form directory, group_box and notification were read line-by-line in this continuation. No additional high-confidence P1/P2 finding arose from those batches beyond the separately listed modal issues. This is not a defect-free guarantee.

VC 10 P2 vendor API selecting an empty carousel underflows with overflow checks

src/carousel/state.rs:722–724 uses (self.item_count > 0).then_some(index.min(self.item_count - 1)). then_some evaluates its argument eagerly, so a zero item count still evaluates 0usize - 1. The public CarouselState::new(0).with_selected_index(0) path at lines 91–118, or set_selected_index on an empty state at 173–175, therefore panics when integer overflow checks are enabled. With checks disabled, it wraps before ultimately returning None; no release-build crash is claimed. The nearby set_item_count uses saturating_sub at 210 and avoids this specific problem. Existing empty-state tests at 1292–1295 and 1436–1445 do not call a selection setter until the item count is positive. Use a lazy then(|| ...) or checked_sub/early empty return. No direct Carousel caller was found by searching current Desktop src; this is a vendor API defect, not established as current Desktop reachable. Source reasoning only; tests were read, not run.

VC 11 P2 vendor API zero chart tick margin panics during paint

src/chart/mod.rs:282 evaluates (i + 1) % tick_margin without guarding zero. Public builders accept zero unchanged: chart/area_chart.rs:216–218, bar_chart.rs:311–313, candlestick_chart.rs:197–199, and line_chart.rs:211–213. With at least one datum, valid accessors, the default visible category axis and no explicit tick count, .tick_margin(0) reaches modulo-by-zero during paint (for example line_chart.rs:416–420), including in release builds. Clamp to at least one or define zero as no labels. No current Desktop vendor chart route was found; Desktop's similarly named declarative BarChart is a separate div-based renderer at src/ui/extensions/declarative/render.rs:620–645. No runtime execution.

VC 12 P3 vendor API ColorPicker anchor setter has no effect

src/color_picker.rs:65,83,126–131 stores the requested anchor, but the full render at 469–530 constructs Popover::new without applying it. No other read of the anchor field exists in the fully read file. Any caller choosing a non-default anchor receives default placement. Forward self.anchor to the popup. No direct Desktop ColorPicker caller found; vendor-only.

VC 13 P2 vendor API Combobox filtered single selection silently changes its value

src/combobox.rs:184–197 and 460–470 compare selected IndexPath vectors rather than selected values. If an existing selection Rust has index0 and filtering puts Go at index0, the single-select Deselect+Select sequence can replace Rust by Go while indices remain [0]; Change/Confirm and closing are skipped at 202–210. The event callback still stores the new selection and synchronizes its snapshot. Cross-file review confirmed the default delegate: searchable_list/delegate.rs:186–199 resolves the filtered index to Go; since Go is not currently

selected, Deselect falls back to removing the old index0 Rust. Then Select at 174–183 inserts Go at index0. These source lines are included in the complete Desktop coverage. Therefore the silent single-select value replacement is confirmed by source flow. Compare selected values, not transient filtered positions. No runtime test was executed. Current Desktop src has no direct Combobox caller. Existing filtered tests 1401–1465 cover multi-select, not this single replacement.

Desktop vendor component settings tables speech и color

Proteus-desktop a608d93

V COMP TAIL 01 P2 Retained number setting subscriptions use obsolete bounds

Source: setting/fields/number.rs L67–139.

The state is retained by page/group/item position (71–79). Its Change subscription captures the initial num_options and setting setter (91–114). Subsequent render explicitly synchronizes changed options into the input engine (124–131), but the subscription still clamps through the original min/max (105–108). This contradicts the documented guarantee at 102–104 that the setting receives a value inside the configured range.

Source-level scenario: render a field with bounds 0..10, then re-render the same field/key with 0..100. Enter 50. The engine now accepts 50, but the retained subscription writes 10. In the reverse change, 0..100 to 0..10, out-of-range intermediate text can write 50 to the setting before blur clamps it. The retained setter can likewise target the old setting if a positional field is replaced, although that dynamic-replacement case is a separate usage assumption, not needed for the bounds finding. Search filtering itself preserves original item indices and is not claimed to cause this.

Fix direction: retain current options/setter in the state and refresh them before processing events, or rebuild the subscription when field identity/configuration changes. Add a keyed-state rerender test for both widening and narrowing bounds.

V COMP TAIL 02 P2 DataTable ignores nonselectable column constraints in cell and keyboard paths

Contract: table/column.rs L35–44, also L185–192: when selectable=false, the column and its cells cannot be selected.

Header mouse selection enforces both table and per-column flags (state.rs L876–889). Body-cell mouse selection checks only global cell_selectable and calls set_selected_cell without inspecting the column (892–929; handler attachment at 2317–2333 and 2427–2448). Arrow/tab column navigation likewise does not check col_selectable or a column's selectable flag (L1162–1237).

Thus an action column deliberately excluded from selection is still selected by body click or keyboard movement. With zero rows/columns, the same keyboard paths can manufacture (0,0) rather than no selection. This is a UI state/contract defect, not an authorization boundary.

Fix direction: centralize validation and navigation over selectable columns, handling empty dimensions, then test mouse and keyboard paths for the same policy.

V COMP TAIL 03 P3 Moving columns leaves grouped header geometry stale

Source: table/state.rs L738–785 computes and caches group widths from leaf columns. L1382–1399 moves the leaf group, emits MoveColumn and notifies, but does not recompute the header layout. Group rendering uses cached width at 2103 and 2188; leaf headers use current columns.

For a two-column group and leaf widths [100,200,300], moving the first leaf to the end changes that group's required width from 300 to 500 but the cached width remains 300. A delegate/caller that explicitly refreshes the table may repair it; there is no such mandatory refresh in the move path. Column resize does recalculate the layout. Recompute after column movement and test unequal widths plus grouped headers.

V COMP TAIL 04 P2 Optional speech paths buffer audio without a ceiling

Microphone capture L47–87 uses an unbounded channel. The device callback allocates and sends mono sample vectors at 119–129; the UI task drains the entire queue into another vector. UI stalls therefore accumulate audio without backpressure or a maximum batch size.

Separately, the macOS recognizer requests speech authorization while a Recognition exists (system/macos.rs L99–105); until the recognition task starts, every audio push extends `pending` (L334–342). SpeechState starts input while connecting (state.rs202–229); its deadline is for stopping, not starting (236–260). If microphone permission is already granted while the speech permission dialog remains pending, audio has no startup deadline/size ceiling. At the default 16kHz i16 format, pending audio alone grows approximately 32KB per second.

This is a boundedness concern in optional vendor speech functionality, not evidence that Desktop currently records or transmits audio. No direct Desktop consumer was found, and top-level Cargo.toml does not explicitly request the speech feature. Use bounded queues, maximum pre-recognition audio duration, and an authorization/start timeout.

V COMP TAIL 05 P2 Public hex parser panics on certain Unicode strings

theme/color.rs L306–317 accepts any string of byte length 6 or 8, then slices at byte offsets 2/4/6 without checking ASCII or character boundaries. `Hsla::parse_hex("€abc")` has six bytes, passes the length check and panics at `hex[0..2]` instead of returning Err. This is source-deterministic safe-Rust panic behavior, not an executed reproduction.

Only tests invoke this method in the pinned component tree, and no Desktop src/crates call was found. The production `try_parse_color` helper takes a different path through `gpui::Rgba::try_from`; do not extend this finding to theme-import or extension-color parsing without auditing that external implementation. Validate ASCII hex digits before slicing or parse bytes without string boundary indexing.

V COMP TAIL 06 P3 HSL mix uses opposite weights for hue and other channels

The trait documents factor as the weight of the first color (color.rs44–45). color.rs L209–224 uses factor as progress from self hue to other hue, while saturation/lightness/alpha use `self*factor + other*(1-factor)`. Consequently factor=0 keeps self hue but takes all other channels from the other color; factor=1 does the reverse. Even equal-saturation red/blue endpoints are reversed. `mix_oklab` handles endpoints consistently and is separately tested, but the HSL tests do not test endpoints.

Align hue interpolation weight with the other channels and add endpoint/non-equal-channel tests. No direct Desktop use was found.

Additional boundaries and rejected candidates

- Number-setting filtering enumerates before filtering, so search does not renumber field keys.
- `time/utils.rs` contains an old five-week calendar grid, but `time/mod.rs` does not include that module. The live Calendar facade delegates to gpui-base; do not report this unused source as the current calendar implementation.

- DatePickerState's number_of_months setter is not read by rendering, which uses DatePicker's independent value. This is an API inconsistency, not a current Desktop issue; it is lower priority than the findings above.
- Both component touch selection Copy (touch_selection/window_overlay.rs30–35) and base Root Copy trim selected text. The controls report owns the corresponding selection-copy candidate and its Desktop wrapper reachability, avoiding duplicate findings here.
- The text/window_selection.rs file is entirely regression tests. All 2,335 lines were read; test descriptions are not treated as fresh failures or proof those tests passed now.
- Public APIs, compatibility tests, select/search delegates, settings, speech, table state, sidebar/tab UI, tooltip lifecycle, touch selection, titlebar/window border, time controls and integration tests were read in full within the ledgered assignment. No absence-of-bugs guarantee is implied.

Desktop vendor component theme

Proteus-desktop a608d93

Findings

1 Bundled theme keys do not match the deserializer Desktop drag borders demonstrably receive a different alpha

Scope: current Desktop loading and painting path, not merely an unused vendor API.

ThemeConfigColors uses these names without aliases:

Supplied by bundled JSON	Name accepted by the schema
description_list_label.background	description_list.label.background
description_list_label.foreground	description_list.label.foreground
drag_border	drag.border
link.foreground	link
link.active.foreground	link.active
link.hover.foreground	link.hover
progress_bar.background	progress.bar.background
slider.bar.background	slider.background

Sources: schema names, lines 401–439, progress, lines 482–484, slider, lines 536–541.

The bundled vendor defaults use the incompatible names (light, dark). The Desktop's own Proteus themes use the same names. Serde ignores unknown fields, so these values become absent and the fallback branches run. A data-only comparison of JSON keys against the declared serde names found 8 unknown color keys in each dark theme and 7 in each light theme, where the label background is absent.

Exact Graphite comparison: Proteus Graphite sets drag_border to opaque #7c93ff, and sets primary.background to #7c93ff. The mapper substitutes self.primary.opacity(0.65). Opacity multiplies alpha.

Effective drag-border RGB is therefore #7c93ff with alpha **0.65**, rather than the configured **1.0**. This is not an ignored-but-equal case.

The loading chain is concrete: Desktop loads the bundled set and user files, selects a named theme, and preferences apply its mode. Interaction normalization does not repair drag_border (normalization). Desktop tab-drop borders and hovered/dragged divider grips paint the resulting field directly.

Other exact Graphite comparisons: normal link is configured #9db0ff, active link #7c93ff, and hover link #b6c4ff; all three fall back to opaque primary #7c93ff. Normal/hover differ; active is **ignored-but-equal**. Graphite progress and slider bars also request #7c93ff and fall back to that same primary, so those are **latent ignored-but-equal entries**, not proven current color differences. Graphite description-label foreground requests #e4e5e9 but falls back to muted foreground #9a9ca6. Direct Desktop consumers of the link/description-label fields were not established in this audit, so their value mismatch is not promoted to a demonstrated Desktop visual defect.

2 Applying semantic tokens drops the representable selection color

Scope: vendor API; no Desktop call to these semantic-application APIs was found in the targeted `src`/`crates` scan.

`color_tokens` exports selection: self.selection. However, `apply_semantic_tokens` copies the other generic colors and stops at ring, without updating either self.selection or its token. Selection is already represented by the legacy theme, so it is not covered by the documented exclusion of scale-only spacing/elevation details. A caller that changes tokens.colors.selection, applies those tokens and reads theme.color_tokens().selection gets the previous value. Theme::update cannot reconcile an assignment that never happened.

The standalone `SemanticColorConfig` and its application list also omit selection, so the JSON semantic-theme format cannot set it. The read tests cover primary/destructive and scale behavior but do not exercise selection application.

3 A custom default theme becomes stale across vendor directory reloads

Scope: vendor `ThemeRegistry::watch_dir`; not established as Desktop-reachable. Desktop explicitly loads files once at startup instead.

In `reload`, a custom theme with is_default: true replaces an entry in default_themes (284–286). On the next reload, themes is first repopulated from that retained map (273–277), then any newly parsed theme whose name is already present is skipped (279–282).

Source-derived sequence: load custom default A; edit A on disk; reload. The old A is seeded from default_themes, and the freshly parsed A is skipped, so its edits do not apply. Delete A; reload. The old A is still seeded from the retained map, so deletion does not remove it. The watcher does subscribe to modify/remove events (194–207); the defect is reconciliation, not lack of events.

Desktop documents and implements startup-only loading, and the targeted Desktop scan found no watch_dir call. Do not describe this as broken Desktop hot reload.

4 group_box title foreground is an accepted but inert schema field

Scope: vendor configuration surface.

The schema exposes `group_box_title_foreground`, but the configuration mapper only applies group_box and group_box_foreground (910–920). Neither `ThemeColor` nor the resolved token list has this title field. It is

deserialized but does not affect the resolved theme through the reviewed application path. No Desktop use of this setting is claimed.

5 Violet 950 s hex metadata disagrees with its RGB HSL metadata but the runtime uses HSL

Scope: vendor data inconsistency, not a demonstrated runtime color defect.

`default-colors.json`, 1831–1837 gives violet-950 hex #1e1b4b (RGB 30,27,75), while rgb and rgbChannel give 46,16,101 (hex #2e1065). The HSL data agrees with the latter to its published precision. Crucially, `ShadcnColor` and `from_hsl_channel` deserialize **`hslChannel`**, not hex or rgb. `ColorName::scale` returns that HSLA. No violet-950/violet_950 reference was found in Desktop src/crates or the built-in default theme. This should remain a low-priority metadata correction.

Audited architecture and boundaries

- Theme::update reconciles mutable legacy solid colors and gradient-capable tokens, reloads registered themes on mode changes, resolves default fonts when needed, rebuilds the Base projection and refreshes all windows. Theme::global_mut deliberately does not do this automatically. Update implementation; token reconciliation.
- Base receives semantic tokens, scrollbar appearance/motion, resize-handle colors and plot motion. Styled motion durations are 0/120/180/280 ms; scrollbar motion uses its separate idle/enter/exit/expand policy. Desktop's interaction projection sets scrollbars to Always and disables external focus rings, so vendor default scrollbar behavior should not be reported as current Desktop behavior. Base projection; motion tokens; Desktop projection.
- Theme files support per-mode colors, optional fonts/radii/shadows, highlighting and two-stop gradient-capable background tokens. Invalid/missing color values select fallbacks rather than yielding a theme-level error. The mapper caps list/table active alpha at 0.2 and selection alpha at 0.3, including individual gradient stops. Desktop subsequently replaces list/table selection with its own sidebar-accent policy. Fallback parsing; alpha policy.
- Default UI-font substitution probes GPUI's resolved .SystemUIFont; default monospace substitution checks platform alternatives and finally .SystemUIFont. Explicitly configured names are left untouched. Font names and monospace resolution are process-cached; the code does not promise live font-install refresh. Monospace resolution; UI-font resolution.
- Registry names are deduplicated on loading; sorted output puts defaults first, then light/dark, then case-insensitive name. Desktop sorts user files before loading, which gives deterministic first-name-wins behavior. Embedded tests were read for theme updates, gradients, mode and Base projection, semantic mapping, alpha clamping, fonts and motion. They were not run.

Desktop vendor component label list и inspector

Proteus-desktop a608d93

VC 14 P2 vendor API Label case insensitive highlighting can panic on ordinary Unicode

Source: vendor/gpui-component/src/label.rs:112–140, especially 123 and 131–135; render invokes this path at 207.

The search operates in full_text.to_lowercase(), but it advances and aligns its next byte offset against the original full_text. Unicode case conversion can change byte length and character boundaries. Reasoned

example: Label::new("İa").highlights("i"). The original UTF-8 has length 3 and boundaries 0,2,3; its lowercase is "i\u{0307}a", length 4, with boundaries 0,1,3,4. After the first match at zero, the original-string boundary loop moves search_start from 1 to 2. The next full_text_lower[2..] at line 123 slices inside the combining character and panics. This is independent of integer-overflow settings. Matching offsets/end lengths are also incorrect even where the next slice happens not to panic; prefix matching uses the query's original length without mapping into source coordinates.

Scope: public vendor Label::highlights API. Targeted searches found no direct current Desktop Label/highlights use and no non-test vendor call applying Label highlights, so no current Desktop trigger is established. Existing Unicode tests at 337–344 and 406–410 cover scripts whose lowercase does not shift offsets and miss this case.

Correction direction: preserve a folded-to-original byte-boundary mapping, or use a matching implementation returning ranges in the original text; advance within the same buffer being sliced. Add case-expanding and byte-length-changing Unicode examples. No test was run.

VC 15 P3 optional inspector the first user edit is discarded

Source: vendor/gpui-component/src/inspector.rs:171–174,181–185,203–207,241–246,252–272. Cross-check confirmed by cross-file review: vendor/gpui-base/src/input/base/state.rs:1299–1301,1307–1328, specifically emit_events=false at 1316 before replace_text and true at 1320; this base file is in the parent's ledger.

Selecting an inspected element sets each editor's editing flag to false and calls set_value. Each subsequent InputEvent::Change handler first sees false, sets it true and returns without applying the style. But set_value deliberately emits no Change, so that discarded event is the first actual user edit, not a programmatic update. The same loss happens after reset and after synchronizing the other editor. For example, changing one generated .p_1() to .p_2() in a single edit leaves the inspected style unchanged until another edit arrives.

Scope: vendor inspector, conditionally compiled/initialized only under the inspector feature or debug_assertions (component src/lib.rs:13–14,111–112,133–134). Do not label as a current release-Desktop path without confirming its build features. Remove the obsolete one-event suppression or explicitly distinguish programmatic changes. No interaction/test was executed.

VC 16 P3 vendor List API initial Down selects a nonexistent item when section zero is empty

Source: vendor/gpui-component/src/list/cache.rs:139–150; list/list.rs:427–438,401–410,380–398,303–311.

RowsCache::next(None) immediately returns IndexPath::default() rather than the first actual entry. For legal section counts [0,2], the cache contains only section 1, but the first Down selects section 0 row 0 and passes that nonexistent item to the delegate and ListEvent::Select. Enter then forwards it to confirm because it checks only that the overall cache is nonempty, not that the selected path exists. A second Down recovers through position_of(None)'s fallback, so the defect can appear as a swallowed first key or an invalid delegate selection. start_search similarly selects the default path from the pre-search cache. A delegate-specific panic is possible only if the application mishandles that invalid path and is not claimed here.

Scope: vendor sectioned List API. Targeted Desktop search found GPUI's different ListState and first-party lists, not a component ListDelegate caller. Existing empty-section navigation tests start with a nonempty section zero and supplied selections; the measurement test uses [0,2] but does not navigate. Use first_entry_position/actual RowEntry for an absent selection and validate selection before confirmation.

Other bounded observations

- Kbd::from_binding at kbd.rs:96–99 retains only the first keystroke of a multi-stroke binding. This is an incomplete hint for chord bindings, not a demonstrated current Desktop binding problem.

- ListState's parent row registers its own click/confirmation listener at list/list.rs:515–525 regardless of a returned ListItem's disabled/separator state; those flags gate only the child's own listeners at list_item.rs:192–234. Applications needing unselectable rows cannot rely solely on ListItem::disabled. ListDelegate has no selectable-row predicate. This is a vendor API limitation; no current Desktop caller was established.
- No additional high-confidence finding from the remaining Kbd, Marker, list loading/module/separator files. Reading coverage is not a defect-free assurance.

Desktop vendor component overlay и Find ownership

Proteus-desktop a608d93

Confirmed findings

C1 P2 Desktop reachable completion code action menus retain themselves through their list delegates

`CompletionMenu` owns `Entity<ListState<ContextMenuDelegate>>`, whose delegate owns `Entity<CompletionMenu>`. Construction stores `cx.entity()` in that delegate, so dropping every outside menu handle leaves a strong entity cycle. `CodeActionMenu` repeats the same ownership. The weak editor field does not break the menu-to-list-to-menu cycle. The pair retains the lists/delegates and their item/documentation vectors for the lifetime of the app's entity storage, rather than releasing them when an overlay is discarded.

Evidence:

- completion delegate owns menu, lines 62–68
- completion menu owns list and creates back-reference, lines 219–275
- code-action delegate, lines 24–28, and owner/construction, lines 140–188.

Desktop route does not require an LSP provider. Declarative Code creates EditorState, and renders the readonly Editor. EditorState::new enables search; the Find action opens it without a readonly exclusion. Input rendering materializes overlays. Creating an editor overlay host always builds both menus, including a host created just for search 315–320.

Exact repetition condition: close Find with no other overlay open, allow the next render, then reopen Find. The closed render removes the host; reopening creates a new one. Each such cycle creates another pair of self-retaining menus. This is not a claim that every close/open gesture does so without an intervening closed render, nor that a retained host allocates on every frame. The tests verify the registry entry/editor owner disappear, not that menu/list weak references expire. Fix direction: weak menu handles in both delegates, and release tests for menus/lists; lazy creation can reduce unnecessary allocation but does not fix the ownership cycle. Static finding, no RSS or runtime measurement claimed.

C2 P2 vendor API only InputGroup silently clears readonly on its supplied control

The group defaults `readonly` to false. It preserves a child's disabled state by OR-ing it with the group's setting, but sets `GroupPresentation.readonly` only from the group and unconditionally calls `input.readonly(presentation.readonly)`. Therefore `InputGroup::new("g").input(Input::new(&state).readonly(true))` becomes editable unless the caller redundantly marks the group readonly. The same conversion applies to Textarea. This contradicts the documented preservation of input capabilities and changes an edit restriction, not merely the border.

Evidence: capability contract/default, group.rs 30–68; disabled preserved but readonly discarded, 140–162; overwrite, 272–291; Input setter, 349–357, state mutation, 610–611. No InputGroup call found in Desktop src/ or crates/. Do not describe this as a current Desktop editor bypass.

C3 P2 vendor API only CodeActionMenu synchronization can dismiss a newer result

CodeActionMenu::hide schedules an unconditional dismiss_code_action_overlay for a later turn. Overlay synchronization calls that same method when mirroring a closed old session. If new actions arrive after this sync but before the deferred closure runs, the closure closes the new session. There is no revision/request identity check. Minimal ordering: keep another overlay (e.g. search) open, show actions A, mark actions closed and render/sync (queues hide), present actions B, then drain pending work. B becomes closed. This is the same stale-hide pattern for which CompletionMenu has separate non-mutating hide versus user dismiss and a focused regression test; CodeActionMenu did not receive that separation.

Evidence: sync closed state, overlay.rs 250–263; queued dismissal, code_action_menu.rs 257–266; new result opens current state, base overlay.rs 86–95; dismiss affects any current result, 159–164; completion's corresponding protection/test, 343–357 and 518–566. No Desktop code-action provider or present_code_actions call found in src/ or crates/, so the functional failure is vendor-only even though empty menu allocation reaches Desktop.

C4 P3 vendor API only updating hover content for an unchanged range leaves stale text

The facade snapshots hover solely by symbol_range and copies contents only when that range changes. Public present_hover replaces both range and contents and notifies, but does not require clearing an old hover first. Calling it twice with 0..3 and different documentation, with a rendered frame between, leaves the first text visible. Use a revision or identity that changes with content. Normal pointer handling avoids repeatedly asking within an existing symbol, so do not present this as a proven ordinary Desktop hover path. Desktop does not configure a hover provider in the searched source.

Evidence: range-only snapshot, overlay.rs 169–187; range equality skips content, 266–277; public content replacement, base overlay.rs 98–109.

Additional lifetime issue same ownership repair area

An active hover uses a strong Entity<EditorState> in HoverPopover, lines 15–35, while the global registry retains its host. Dropping the containing view while that hover is still registered leaves the editor retained by the global host; pruning based on owner.upgrade().is_some() cannot recognize it as dead. SearchPanel, completion, code-action and diagnostic objects use weak editor references; hover does not. This is another vendor lifecycle concern, not an independently demonstrated Desktop hover usage. It does not imply SearchPanel has a strong editor cycle.

Review and validation boundaries

Read group composition and every group test; checked addon alignment/style preservation, disabled propagation, child focus, textarea behavior, copy, popup actions and scaling. Read overlay revision/echo checks and all overlay tests; inspected the caret-anchored popup's prepaint positioning; read completion highlight range filtering and its tests; read search/replace query, focus, navigation and regression tests. Existing tests were read, never run. No source edits, dependency install, upstream code execution or benchmark. Supplementary source snippets/searches establishing callers and state behavior are not counted as full-file audit coverage.

Desktop vendor component остальные controls и границы

Proteus-desktop a608d93

Results and reachability

No confirmed critical/high-severity Desktop defect established in this slice. Static source review is not runtime, accessibility or platform validation. Distinguish available vendor controls from controls Desktop actually instantiates.

- NativeMenu is genuinely reachable indirectly through Input context menus: vendor input/input.rs:614–662 builds Cut/Copy/Paste/Select All and calls show; Desktop uses Input directly in src/ui/chat/question_card.rs:118 and through src/ui/kit.rs:116. OS routing is native_menu/mod.rs:201–222: AppKit on macOS, Win32 on Windows, drawn PopupMenu fallback elsewhere. The fallback is window-clipped, not an OS popup. Absolute icon paths read filesystem; relative paths use AssetSource (mod.rs:226–257); tests cover CWD-shadow prevention. AppKit refreshes the window after dismissal (macos.rs:67–80); Windows returns without updating on no selection (windows.rs:64–80). This asymmetry is a platform-test target, not a proven Windows failure.
- Scrollable is directly used by composer drafts/queue (src/ui/chat/composer.rs:134–150), settings, catalog details and extension surfaces. Existing-handle gutter is used by session list (src/ui/panels/sessions.rs:188–202). Diff has intentionally different embedded versus standalone handling (src/ui/diff/file.rs:179–196): standalone horizontal wrapper remaps vertical wheel to X, embedded diff preserves wheel propagation. Wrapper preserves outer size/flex constraints, moves the source content into a tracked area, clips the scrolled axis and retains explicit minimums. IDs default to caller location; repeated dynamic call sites must provide unique IDs, demonstrated by tests. Tests cover intrinsic/max/flex sizing, padding/gap, overflow, wheel direction and state independence; they were read, not run.
- ProgressCircle is directly instantiated for declarative Face::Ring (src/ui/extensions/declarative/badge.rs:169–181), with percent, color, numeric child; that path does NOT set loading. Both progress controls provide numeric/a11y state through BaseProgress, optional accessible labels and determinate transitions. Linear Progress explicitly supplies a static reduced-motion loading segment (progress.rs:149–164); ProgressCircle loading always uses repeating Animation (progress_circle.rs:183–193). At delta=1 its arc start=end=100. The reduced-motion endpoint behavior needs upstream/runtime validation; do not report this as an observed Desktop loading bug, because the verified Desktop path is determinate.
- Message / MessageGroup compose avatar/header/body/footer and allow stable ID+role; ghost bubbles suppress inherited insets. MessageScroller owns only ListState, not message payload; tail following, splice validation/remeasurement, jump button and fade are implemented. No direct uses of these vendor classes were found in src/ or crates/. Do not equate their presence with the actual Desktop transcript renderer.
- Questionnaire is a skin over gpui_base state, validation, navigation and shortcuts. Root captures keys; active items/choices/input use state definitions; choice controls delegate behavior to base; action buttons delegate previous/skip/next/submit. The global size lookup is bounded at 128 entries. Extensive tests cover keyboard guards, text input, radio movement, single-choice delayed advance, multiple-choice toggling and double-submit prevention. No direct Desktop Questionnaire usage found; Desktop question_card uses its own composition and Input.
- Radio / RadioGroup project controlled values, names, disabled state, focus and set position through base controls. Rating differs: internally retained value/hover, pointer handlers on star divs, no keyboard/focus/a11y role wiring in this implementation. No direct Desktop usages found for either family; Rating's accessibility limitation is a vendor-API concern rather than a demonstrated Desktop screen defect.

- Pagination delegates bounds/navigation to base and bounds ellipsis menus to 100 nearby hidden pages; tests verify long and short gaps. No direct Desktop Pagination use found.
- Plot module reexports base primitives; tooltip supplies crosshair, halo dots, cursor-relative positioning, hover progress/spring and structured rows. Structured title/rows replace the freeform base entirely (tooltip.rs:518–557), so styles set on base are also lost in that branch; custom minimum-width documentation below is therefore accurate only for freeform content. No direct Desktop plot tooltip use established.
- Popover wraps BasePopover, forwards controlled/open/focus/dismissal, and draws arrows. Tests cover all anchors, offset, trigger width, window clamping, arrow geometry, dismissal and repeated dropdown enter motion. Dropdown helper documents no scale/exit and always-from-above slide. Indirect vendor use includes dropdown menus and input diagnostic/hover popovers; direct Desktop Popover constructor was not found. API documentation inconsistency: appearance(false) says outside click will not dismiss (264–270), but render forwards overlay_closable independently (343–350); callers wanting that behavior should set overlay_closable(false), as dropdown_menu.rs:112–114 does.
- Resizable reexports base behavior and supplies themed animated handle appearance with idle/hover/pressed/drag states; dock/dock.rs consumes render_resize_handle. No direct Desktop h_resizable/v_resizable/resize_handle call found in src/ or crates/. This does not exclude indirect reach through a dock consumer.

Evidence links

- vendor/gpui-component/src/input/input.rs:614–662
- src/ui/chat/question_card.rs:110–120
- src/ui/kit.rs:112–118
- src/ui/chat/composer.rs:134–150
- src/ui/panels/sessions.rs:188–202
- src/ui/diff/file.rs:179–196
- src/ui/extensions/declarative/view.rs:535–546
- src/ui/extensions/declarative/badge.rs:169–181
- vendor/gpui-component/src/menu/dropdown_menu.rs:112–125

10 Desktop fixtures performance и доказательность

Proteus-desktop a608d93

Narrow harness findings

R DESK 01 persisted synthetic Codex history can reuse turn item identities after restart P3 high confidence fixture only

examples/codex/mock-app-server.py:25 initializes sequence = 0 on every process start. The file persists data, including history (98–118), but does not persist or restore that counter. turn/start increments the process-local counter and uses fixture:turn/{sequence} (234–244). Started user, reasoning, command and assistant item ids derive solely from that turn (130, 144, 179, 189).

Concrete source-derived scenario: resume a persistent fixture thread after restarting the peer, then send its first new turn. If the thread already contains the matching numeric turn from an earlier run, new canonical history records repeat its turn and item identities. This does not require a collision in the new-thread generator: thread creation uses time_ns, but a resumed existing thread uses the preserved history. The README expressly offers state-file persistence for restart checks (12–15). Existing tests cover persistence during deletion but not turn identity across process restarts.

Effect: a fixture meant to validate resumed-history behavior can generate invalid duplicate identities, distorting UI/history results. This is not evidence that production Codex reuses identities or that Desktop loses data. Use a UUID/run nonce or persisted monotonic sequence for fixture turn ids, and add a restart/resume/send case that verifies item identities remain unique.

Source: Источник examples/codex/mock-app-server.py#L234-L244

R DESK 02 shimmer isolation check lacks a positive activity readiness assertion test gap not production defect

examples/feedback_caption.rs:55–67 sleeps, reads the parent's render count twice, and emits passed whenever the counts agree. It does not assert that an initial real layout/paint happened, that the child actually received any animation ticks in non-reduced mode, or that the window is active. A broken/stopped ticker would therefore satisfy the same no-parent-render assertion. By contrast chat_scrolling/probe.rs:203–216 explicitly verifies a laid-out active window, and its motion scenarios verify physical movement before testing quiescence.

Improve the oracle with positive child animation evidence before checking parent isolation, retaining a separate expected-zero test in reduced-motion mode. No native false pass was executed here.

Source: Источник examples/feedback_caption.rs#L55-L67

What this residual review establishes

- The additional renderer shader test validates WGSL parsing/type validation for the three assembled variants; it does not assert GPU pixel output or platform rendering compatibility.
- Scrolling harnesses expressly distinguish resident rows, synthetic paged state, physical prepaint geometry and actual backend history. Their read-source coverage should not be described as a native run.
- The Codex-internals research report identifies exact older upstream and Proteus revisions, offline empty-thread scope, modified harness provenance, and limitations. Its historical memory figures do not establish current or comparable production memory.
- docs/agent-drivers.md includes a dated/archived exploratory section after its current-state introduction; that section is not evidence that present terminal-broker or chat-workspace work is still absent. docs/chat-workspaces.md is the more specific current workspace contract.
- Archived DCP helper uses old imports/paths, as its README explicitly says it needs adaptation before revival. It is not a current portable-build failure.
- assets/proteus-logo.svg was read as XML source; the Noto font attribution/license documents were read. Binary font/icon rendering was not inspected. Nonvendor location does not imply all assets are authored by this project.

Findings

PERF 1 P2 Creating a chat after deleting an earlier one can overwrite a surviving synthetic chat

- Source: examples/performance/chat-spaces/fixture.py:185–205, especially 189 and 196–197; assignment into the maps is at 84–89.
- number = len(self.sessions) + 1 is used as the new name/UUID allocator. Deletion decreases that count while later identities remain allocated.
- Deterministic sequence from the source: start with A/B/childA; create C (number 4), create NEW5 (number 5); delete C; create another chat. Length is four, so the new session is again NEW5, with the same UUID, root-thread ID and session directory as the surviving NEW5.
- add() silently replaces self.sessions[state.session] and self.subscription_stop[state.session]. The existing chat's identity and stop-event are overwritten rather than allocating an independent new chat. A subscriber that already captured the old stop-event can also continue waiting on an event no longer reachable through the map.
- Impact: repeated create/delete GUI tests can misdiagnose Desktop, hide state-reuse regressions, or fail to model independent chat identities. No production data is affected by this synthetic fixture.
- Suggested fix: use a monotonically increasing allocator (or fresh UUID/path), reject collisions, and keep subscription ownership tied to the specific session generation.
- Missing regression: test_create_independent_c_then_delete_without_mutating_other_histories (test_fixture.py:81–94) tests only one create/delete. Add C → NEW5 → delete C → create, with history/identity/stop-event checks for the original NEW5. Also cover deletion of one initial chat before creation.
- Confidence: high; directly derived from map mutation and the allocator, without running fixture code.

PERF 2 P2 validation oracle Native history does not detect newly created canonical blob files

- Source: examples/performance/native-history/run.py:164–167,236,255.
- The runner enumerates the original journal and blobs once, then only checks sha(journal.parent / path) == original for those original keys. It never compares the set of canonical paths after teardown with the original set.
- A regression that adds a new file below blobs/, while leaving existing files and journal.jsonl unchanged, passes this check. The report still records journal.unchanged = True and the original canonical file map.
- This leaves the promised read-only canonical-storage invariant incomplete (native-history/README.md:65–69,88–89). It is a test-oracle defect; this audit does not claim that the actual application writes such files during startup.
- Suggested fix: re-enumerate canonical paths after teardown and compare the full path-to-hash maps, including added and deleted files.
- Missing regression: the only local test_runner.py case covers process classification, not canonical verification. Add synthetic unchanged/add/delete/modify cases for the verification helper.
- Confidence: high for the missing detection, no claim of an observed production mutation.

Additional validation and failure path gaps

Terminal replay timeout has no guaranteed owned process cleanup

examples/performance/terminal/run.py:47–63 launches without a new process session and without a surrounding try/finally. At the 100-second timeout it calls only process.kill() and raises immediately, without wait()/communicate() or owned-descendant cleanup. Exceptions during sampling and user interruption also lack cleanup. A killed replay process cannot run the normal explicit PTY shutdown in crates/terminal/examples/replay.rs:71–83.

This is a real failure-path omission. Whether a shell/awk survives depends on PTY hangup behavior and the failure, so no surviving-process claim is made without running it. The remedy is an owned-process lifecycle guard with bounded reap and descendant handling, patterned after the other harnesses. No local Python tests accompany this terminal runner.

Terminal replay silently treats unreadable process memory as absent

examples/performance/terminal/run.py:26–44 swallows PermissionError, process races and missing /proc files without recording incomplete samples. A memory-read failure can therefore lower the sampled tree peak, including producing zero metrics. This is unlike native-history's explicit memory_incomplete_pids check at run.py:210. Exited children between samples are an inherent sampling limit, but permission failures on a live process should be separately surfaced rather than silently reported as complete memory measurements.

The runner also accepts --runs 0 or negative values, then fails later at selected[0] (78,86,95–97), and writes an existing report path with write_text (111–112) rather than refusing overwrite. These are lower-priority CLI/evidence preservation issues, not production defects.

Teardown failures can disagree with saved status

native-terminal/run.py sets state['status'] = 'stopped' at 124, then performs teardown in 128–144. A failure in stop_native, reap_adopted or fixture.close is outside the preceding except, so the final receipt can retain stopped despite a nonzero runner exit; it does not attach a teardown error. native-svg/run.py:228–236 explicitly handles its teardown error, providing an existing local model. Add fault-injection tests for teardown failures, and require consumers to check process exit as well as receipt status.

For native-images/native-svg, final sampler operations occur before the try protecting fixture closure and report serialization (native-images/run.py:155–161; native-svg/run.py:238–244). An exception there can skip final server cleanup/report writing. Normal lifecycle tests are present but do not cover these instrumentation-failure paths. This is a reliability gap in failure reporting, not evidence of normal-run leakage.

Identity and proof limits of optional mapping capture

native-svg/tools/mappings.py:20–40 verifies the executable hash and that /proc/PID/stat start ticks do not change *during* capture, but does not compare those ticks with a recorded launch identity. A stale running receipt plus PID reuse by another instance of the same executable is not rejected at the start. The helper is read-only and no signal is sent, but it could attribute another process's mappings to this run. Compare the initial identity to the original sampler identity before reading mappings.

The capture/compare helpers have no dedicated tests in this directory. They intentionally verify solid-color paint and bounds, not complete SVG geometry or GPU cleanup; that limitation is correctly documented.

Coverage and observations by group

Agent terminal 6 files 513 lines

Read README, nested proxy, password prompt, launcher, wire verifier and workflow. The command is fixed to the prepared password demo, text input does not choose a shell command, the policy is intentionally permissive within an isolated synthetic scenario, and the password is generated locally for the wire probe. The verifier uses its owned Desktop/Core relation, exact fixture config, loopback listener and direct no-proxy HTTP. It waits for the tool yield before supplying input, deliberately avoids retrying uncertain input, drains output and checks plaintext absence in output/history/session files. Those are meaningful checks, not production hardening.

Limits: this directory has no local automated tests. Nested attribution, holding/releasing the response, UI tab lifetime, stop semantics and visible no-echo remain separate native checks. The README correctly says its direct HTTP verify.py is not a nested UI proof. The final tool did not settle polling uses the same truthy result condition already observed before input, so it adds no distinct proof that a new tool-result state appeared after command exit; final terminal exit/drain is checked independently. Plaintext scanning is not a guarantee against every possible encoded storage representation.

Build cost 2 files 182 lines

Reads scope from selected components, preserves caches, samples an owned build tree, records GNU time separately from sampled tree peaks, and records source revisions/dirty paths. Full builds are explicit. Documentation correctly distinguishes existing-cache rebuilds from cold builds, resolve graph size from actually compiled crates, and single-process RSS from tree PSS. No in-scope tests exercise this runner. Static review cannot validate the recorded old build measurements or external sampler accuracy.

Chat spaces 5 files 693 lines

Per-session immutable transcript/snapshot state, independent SSE baselines, addressed workspace resolution, exact allowed-query handling, isolated output, and no native-verified claim. Tests exercise concurrent baseline/reconnect, foreign child cwd, create/delete, startup badges, auth and preparation. PERF-1 affects lifecycle scenarios beyond their single-create coverage. /delete-session deliberately does not model cascade deletion, per README, so that is not reported as a production flaw.

Native history 3 files 453 lines

Separates first application-cache and restart runs, requires explicit ready executables/seeder, validates live session/tail after idle measurement, uses layout readiness by default, limits memory, and samples owned process identities. Readiness is honestly distinguished from a GPU-presented frame. PERF-2 limits storage immutability evidence. Only process classification is unit-tested in this directory. HOME and XDG_STATE_HOME are not replaced in the launched environment; the explicitly used Proteus/config/data/cache paths are private. Do not describe this runner as fully sanitizing every ambient environment variable.

Native images 5 files 937 lines

Bounded deterministic PNG generation, exact image-ID lookup, encoded and aggregate decoded corpus limits, staged immutable snapshots, direct lifecycle tests and no native GUI acceptance inference. Tests cover PNG CRCs/dimensions, corpus bounds, image byte equality, counters, auth/unknown ID, one malformed-query case, immutability, oversized files, preparation and normal/abnormal runner stop. The test named test_extra_duplicate_or_wrong_session_query_rejected exercises only the extra-query variant; duplicate keys and wrong session are validated in source but not individually tested in that method.

The dated result correctly limits itself to its pinned version and PNG corpus, acknowledges changed runner guards and unequal durations, does not compare latency from that pair, and does not equate freeing CPU owners with a resident-PSS/VRAM decrease. The README explicitly updates the current retained-scene lease model without rewriting the historical measurement.

Native SVG 10 files 1 415 lines

Read corpus, renderer, wire adapter, launcher, test suite, capture/compare/mapping helpers, README and dated report. Source stage matching is exact, SVG output uses real native nodes, the immutable snapshot is compared as a complete object, progression is sequential, callback evidence is distinguished from paint, named samples wait for acknowledgment in capture, and launch/renderer provenance is recorded. The runner includes subreaper cleanup for independent extension process groups and redacts the fixture token from process samples.

The historical report rejects invalid geometry and unacknowledged measurements, reports increased PSS rather than inventing savings, and separates true bitmap/PTY/stdio lifecycle behavior from a global memory claim. Matching region bounds does not prove full SVG fidelity, correctly stated. Read-only findings do not reverify old PNGs, ELF hashes, test counts or local receipts, which are not part of this assignment.

Native terminal 5 files 667 lines

Strict synthetic settings DTOs, one copied ready Terminal package, isolated working paths, live shell PID/start-ticks proof with ancestry checks, and child-subreaper cleanup are present. Tests cover protocol shape, auth/scope, copy isolation, normal stop, nonzero exit, owned/unowned shell proof and separate process-group cleanup. It intentionally measures functional lifecycle, not RAM. The README says even zero unsolicited exit is a failure; source implements that, though this directory's unsolicited-exit test uses code 7 only.

Terminal replay 2 files 173 lines

Reads an already-built shared replay executable and alternates engines within workloads. Report separates self memory from sampled process-tree peaks and states retention policies differ. Auxiliary replay source confirms marker matching is against a complete line, uses the same grid/workload/idle interval, and explicitly shuts down the PTY before emitting success. Failure-path limitations are above. This is headless engine/PTY evidence, never Native GUI or full-app performance evidence.

Other dated reports 3 files 435 lines

Read dot-2026-11-03.md, session-lifecycle-2026-11-03.md, and subagent-view-2026-11-03.md completely. Each records concrete revisions, scope and limitations; failed/terminated attempts are not presented as passed final gates. They distinguish unknown original incident cause from synthetic regression evidence, and implementation savings from unmeasured runtime benefit. No definite stale behavioral claim was established from this assigned subset. Dated counts/revisions are historical claims, not current snapshot test results, and were not rerun or independently reproduced.

Findings

NS 1 P2 harness correctness live append followed by finish duplicates a child message ID

- examples/performance/native-stream/subagent_demo.py:104-107 chooses the next live message number as 10000 + len(child["messages"]). The initial A transcript has 150 messages (:38-46), so the first appendA creates message number 10150.

- subagent_demo.py:147-159, specifically :154, subsequently appends the final response with the hard-coded number 10150 again.
- Both controls are supported in one live run (:300-310); appendA can also be repeated while the child runs. Therefore appendA followed by finishA produces two different messages with identical message_id (00000000-0000-0000-0000-0000000027a6). Multiple prior appends still collide with the first live append.
- Consequence: the fixture claiming exact child identities emits an internally invalid identity sequence on its live-to-finished path. A client using message IDs to merge, retain, or identify rows may conflate the two rows, producing a misleading UI failure or masking identity-handling defects. This report does not assert that a specific production reducer actually conflates them.
- Tests miss the composition: test_subagent.py:24-47 checks binding/live append but never finishes; :102-126 finishes only the original 150-message transcript. Initial uniqueness is checked in :71-100, before any appended messages.
- Suggested correction: derive the final message number from the current child length or a shared monotonic allocator; add one test for bind → append → finish and another with multiple appends, asserting all message IDs remain unique, final text is preserved, total count is correct, and base.common_prefix matches the previous revision length.
- Confidence: high, direct deterministic arithmetic/data-flow proof from source. Not executed.

NS 2 P3 harness oracle gap deferred A is readable before binding

- subagent_demo.py:35-46 always creates both child records, including A's absolute path and all 150 saved messages.
- In --deferred-a mode, :69-72 sets bound_a=False and removes A's session from the live parent row. :77-79 also hides A from the catalog until binding.
- However /history/page checks only that the requested path appears in self.children (:123-126) and then serves it (:127-145). It does not check bound_a. Thus a request for the exact preallocated A directory succeeds even before bindA, although the error text says the target must be an “exact bound child directory.”
- Consequence: the deferred-binding fixture fails to reject premature/guessed A-history reads. Its general audit accepts those reads; only an explicitly requested check-unopened checkpoint catches them. This weakens the intended binding/laziness oracle rather than exposing production data: the server is a local synthetic fixture.
- test_subagent.py:24-47 confirms only that A is absent from /sessions before bind_a; it does not try A's history endpoint before binding. The invalid-target matrix at :167-205 also omits this case.
- Suggested correction: reject A history while bound_a is false (and define whether finishing an unbound A is allowed), then test pre-bind rejection and post-bind successful tail loading.
- Confidence: high, directly follows the handler branch. Not executed.

Documentation and coverage notes

DOC 1 Low priority documentation gap the new deferred live controls have no walkthrough

README.md:487-555 describes only the initially bound 150-message setup and direct finishA path, including a 151-message total, common_prefix=150, and exact refresh counts (:522-528). The implementation additionally

exposes --deferred-a, control/bindA, and control/appendA (subagent_demo.py:300-310,345-346), but the README has no instructions for them. Its existing numbers are correct for the documented default scenario; they are not evidence of incorrect default behavior. Extend the walkthrough for deferred binding and repeated live appends, with revision/prefix/count expectations appropriate to that sequence. This is a gap, not a claim that the current default instructions fail.

Other observed coverage limits, not separately promoted to defects:

- The inspected tests intentionally exercise wire payloads, fixture controls, provenance helpers, and exact source/clipboard receipts. They explicitly do not prove rendering, layout, animation, real Native process cleanup, or actual clipboard origin.
- capture_motion.py has no directly assigned dedicated automated test; capture timing, helper failure, and receipt recovery require separate coverage. The README correctly calls its timings command bounds, not exact frame timestamps, and warns helper serialization can miss intermediate frames.
- HistoryFixture tests cover valid pages, custom short first pages, long-tool data, dates, and unpaged content, but do not include a malformed/auth/stale-revision request matrix for the specialized /history/page branch. SubagentFixture has a more extensive negative matrix.
- No GUI screenshots, performance metrics, or runtime pass claims were generated during this read-only audit.

Notable boundaries preserved

The general fixture is loopback-only with authentication and bounded payload/trace/request queues. The stream oracle disallows healing completion and post-subscription replacement snapshots, while the separate status demo explicitly opts into completion. The README repeatedly distinguishes SSE delivery, smoke checks, exact byte checks, and real UI verification. Renderer selection is intentionally limited to a ready Markdown extension for the relevant demos. Synthetic input handling is not treated as an attacker-facing production boundary, consistent with repository instructions.

Заключение

11 План проверки после исправлений

1. Зафиксировать актуальные SHA и оценить изменения относительно четырёх снимков. Не переносить вывод о зависимости Core 23883bd на 9b07d03 без отдельной сверки.
2. Исправлять локальные границы малыми проверяемыми изменениями. Для каждой находки сохранить precondition, ожидаемый invariant и regression, который реально достигает проблемного пути.
3. Сначала проверить отмену A/B и pool reuse, protocol stdin, Git hooks/config, plan retention после durable ACK, canonical path policy и profile Save → new/cold resume.
4. Для всех bounded APIs проверить не только payload одного сообщения, но admitted work, retained bytes, число ожидающих threads, вторичные очереди, caches и освобождение после cancel/close.
5. Для Desktop отдельно проверить реальные GUI/IME/accessibility действия, два окна, закрытие и повторное открытие Find, полный approval payload, переполнение frame и неторопливый extension handler.
6. Для vendor safe FFI APIs нужны focused Rust safety tests и проверка native implementation соответствующей сборки. Отсутствие текущего Desktop caller не снимает дефект публичного safe API.

7. Для experimental native renderer сначала установить budgets dimensions/depth/work и тестировать parser containment. Не выдавать native исправления за исправления production Pele.
8. Запустить тесты, сборки, platform/GUI проверки и benchmark только отдельным разрешённым этапом. Настоящий отчёт не содержит таких результатов. Для resource claims заранее определить метрики и сценарии, а не выводить RSS или latency из числа строк.

Как использовать доказательства

В каждом разделе отделены подтверждённый control flow, условие достижимости, API-only или experimental область, кандидат и отклонённая гипотеза. Ссылки ведут к неизменному source snapshot; прямой доступ зависит от обычных GitHub прав пользователя. Они не являются публичной публикацией отчёта и не расширяют доступ.

Шесть Desktop binary hash-only позиций перечислены в разделе покрытия. Ни чтение лицензий, ни проверка hashes, ни исторический PASS в документации не подтверждают безопасность, правовую совместимость, текущую работоспособность или успешное выполнение тестов.